



Article

Advanced Hybrid Transformer-CNN Deep Learning Model for Effective Intrusion Detection Systems with Class Imbalance Mitigation Using Resampling Techniques

Hesham Kamal * and Maggie Mashaly *

Department of Information Engineering and Technology, German University in Cairo, Cairo 11835, Egypt

* Correspondence: hesham.khalil@student.guc.edu.eg (H.K.); maggie.ezzat@guc.edu.eg (M.M.)

Abstract: Network and cloud environments must be fortified against a dynamic array of threats, and intrusion detection systems (IDSs) are critical tools for identifying and thwarting hostile activities. IDSs, classified as anomaly-based or signature-based, have increasingly incorporated deep learning models into their framework. Recently, significant advancements have been made in anomaly-based IDSs, particularly those using machine learning, where attack detection accuracy has been notably high. Our proposed method demonstrates that deep learning models can achieve unprecedented success in identifying both known and unknown threats within cloud environments. However, existing benchmark datasets for intrusion detection typically contain more normal traffic samples than attack samples to reflect real-world network traffic. This imbalance in the training data makes it more challenging for IDSs to accurately detect specific types of attacks. Thus, our challenges arise from two key factors, unbalanced training data and the emergence of new, unidentified threats. To address these issues, we present a hybrid transformer-convolutional neural network (Transformer-CNN) deep learning model, which leverages data resampling techniques such as adaptive synthetic (ADASYN), synthetic minority oversampling technique (SMOTE), edited nearest neighbors (ENN), and class weights to overcome class imbalance. The transformer component of our model is employed for contextual feature extraction, enabling the system to analyze relationships and patterns in the data effectively. In contrast, the CNN is responsible for final classification, processing the extracted features to accurately identify specific attack types. The Transformer-CNN model focuses on three primary objectives to enhance detection accuracy and performance: (1) reducing false positives and false negatives, (2) enabling real-time intrusion detection in high-speed networks, and (3) detecting zero-day attacks. We evaluate our proposed model, Transformer-CNN, using the NF-UNSW-NB15-v2 and CICIDS2017 benchmark datasets, and assess its performance with metrics such as accuracy, precision, recall, and F1-score. The results demonstrate that our method achieves an impressive 99.71% accuracy in binary classification and 99.02% in multi-class classification on the NF-UNSW-NB15-v2 dataset, while for the CICIDS2017 dataset, it reaches 99.93% in binary classification and 99.13% in multi-class classification, significantly outperforming existing models. This proves the enhanced capability of our IDS in defending cloud environments against intrusions, including zero-day attacks.

Keywords: ADASYN; data resampling; deep learning; ENN; IDS; multi-class classification; Transformer-CNN

Citation: Kamal, H.; Mashaly, M. Advanced Hybrid Transformer-CNN Deep Learning Model for Effective Intrusion Detection Systems with Class Imbalance Mitigation Using Resampling Techniques. *Future Internet* **2024**, *16*, 481. <https://doi.org/10.3390/fi16120481>

Academic Editor: Ugo Fiore

Received: 31 October 2024

Revised: 13 December 2024

Accepted: 16 December 2024

Published: 23 December 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

As the internet has evolved and expanded over time, it now offers a wide array of valuable services that significantly improve people's lives. Nevertheless, these services are accompanied by various security threats. The increasing prevalence of network infections, eavesdropping, and malicious attacks complicates detection efforts and contributes to a rise in false alarms. Consequently, network security has become a paramount concern for a growing number of internet users, including in critical sectors such as banking, corporations, and government agencies.

Cyber-attacks typically initiate with reconnaissance efforts aimed at identifying system vulnerabilities, which are subsequently exploited to execute harmful actions [1]. Unauthorized access to computer systems threatens their confidentiality, integrity, and availability (CIA), resulting in what is classified as an “intrusion” [2]. In recent years, a plethora of sophisticated cyber-attack methods has emerged, including brute force attacks, botnets, distributed denial of service (DDoS) attacks, and cross-site scripting [3]. These developments have heightened concerns regarding cyber security. Cybercriminals are increasingly leveraging numerous hosts and cloud servers as vehicles for deploying malware and botnets, including Bitcoin Trojans. According to the internet security threat report (ISTR), malware is detected, on average, every 13 s during web searches. There has been a marked rise in incidents of ransomware, email spam, and other online threats, as noted by CNBC [4,5]. In this context, intrusion detection systems are crucial for enhancing network security and alleviating the growing risks associated with cyber-attacks [6].

Real-time intrusion detection is essential for maintaining the security and integrity of network infrastructures. Deep learning models have demonstrated remarkable effectiveness in analyzing network traffic instantaneously, facilitating the rapid identification of potential intrusions [7]. Various machine learning strategies contribute to enhancing the agility of intrusion detection systems (IDS), particularly in their ability to adapt to newly emerging threats [8]. Moreover, the incorporation of real-time functionalities within IDS significantly bolsters network security by enabling the swift detection and mitigation of attacks [9].

IDS are among the most commonly implemented security mechanisms, designed to detect and prevent unauthorized access while safeguarding both individual computers and broader network infrastructures from malicious threats. These systems can be classified into two primary categories, based on their method of identifying intrusions:

- **Signature-based IDS:** This approach involves scrutinizing network traffic or host activity by matching it against a repository of known malicious patterns. While it excels at detecting familiar threats, its efficacy hinges on continuous updates to remain vigilant against evolving attacks. However, its dependence on established signatures renders it less effective in confronting unknown or zero-day threats, as it lacks the capacity to detect new intrusions that fall outside its predefined dataset.
- **Anomaly-based IDS:** These systems detect threats by recognizing deviations from established behavioral norms, rather than relying on predefined attack signatures. This makes them particularly adept at identifying zero-day attacks that exploit previously undiscovered vulnerabilities. By utilizing machine learning and deep learning algorithms, anomaly-based IDS can analyze extensive datasets, learn patterns of normal system behavior, and detect anomalies with exceptional precision. This method not only enhances adaptability to emerging threats but also minimizes false positives. In our research, we adopted this approach to improve the accuracy and responsiveness of intrusion detection.

In this study, we introduce an advanced hybrid deep learning model combining Transformer and convolutional neural network (CNN) architectures for a robust intrusion detection system. Our methodology tackles class imbalance by employing various data resampling techniques, such as adaptive synthetic (ADASYN) and synthetic minority oversampling technique (SMOTE), for binary and multi-class classification, along with edited nearest neighbors (ENN) and class weighting strategies to enhance model robustness. The findings reveal that our Transformer-CNN model significantly outperforms prior methods, achieving an impressive 99.71% accuracy in binary classification and 99.02% in multi-class classification on the NF-UNSW-NB15-v2 dataset [10,11], as well as 99.93% accuracy in binary classification and 99.13% in multi-class classification on the CICIDS2017 dataset [12,13], highlighting its efficacy in diverse operational contexts. Below, we outline the key contributions of our research:

- We create a highly efficient intrusion detection system using an advanced hybrid Transformer-CNN model, integrated with techniques such as ADASYN, SMOTE, ENN, and class weights to effectively tackle class imbalance challenges.
- An enhanced data preprocessing pipeline is applied, which first utilizes a combined outlier detection approach using Z-score and local outlier factor (LOF) to identify and handle outliers, followed by correlation-based feature selection. This structured approach refines model input, enhancing accuracy and reducing computational complexity.
- Using the NF-UNSW-NB15-v2 and CICIDS2017 datasets, this study highlights the exceptional performance of the proposed model, demonstrating its superiority compared to current state-of-the-art models in the field.

This paper is organized into several sections: Section 2 delivers an extensive overview of the relevant literature, offering insights into existing research in the field. Section 3 outlines the methodology utilized in this study, detailing the approaches and techniques employed. Section 4 showcases the results derived from the experimental procedures, providing an analysis of the data obtained. Following this, Section 5 engages in a thorough discussion of the findings, interpreting their significance and implications. Section 6 highlights the limitations encountered within the proposed methodology, providing a critical assessment of its scope. Section 7 concludes the study by summarizing the primary contributions and key insights gained. Lastly, Section 8 presents potential avenues for future research, suggesting directions for further exploration and investigation.

2. Related Work

IDSs have become vital safeguards for national, economic, and personal security due to the rapid expansion of data collection and the increasing interconnectedness of global internet infrastructures. The concept of intrusion detection was pioneered by James P. Anderson in 1980 [14], aimed at mitigating vulnerabilities in computer systems and enhancing monitoring capabilities. Over the years, as security professionals have continued to refine the effectiveness and accuracy of IDSs, their widespread adoption has followed. This section delves into the various machine learning and deep learning techniques that have been explored in the literature for intrusion detection. Given the extensive applications and remarkable performance of deep learning in fields such as image recognition and natural language processing, it has emerged as a compelling choice for detecting traffic anomalies within IDSs. Academic publications have primarily focused on utilizing deep learning methodologies for the classification of attack types in intrusion detection systems.

2.1. Binary Classification

In the context of binary classification for IDS, the integration of the Transformer-CNN model presents a highly effective solution. The Transformer component is employed to extract key contextual features, allowing the model to thoroughly analyze relationships and dependencies within network traffic. This process captures vital insights into the data. Once these features are extracted, the CNN layer further processes them, showcasing exceptional ability in detecting intricate patterns and distinguishing between normal and malicious traffic. This combined Transformer-CNN approach enhances the model's accuracy in identifying attacks, thus bolstering the system's detection capabilities and improving overall cyber security defenses.

In ref. [15], the authors introduce the range-optimized attention convolutional scattered technique (ROAST-IoT), an innovative AI model specifically tailored for efficient intrusion detection in IoT environments. This model utilizes a range-optimized attention mechanism coupled with a multi-modal approach to uncover intricate relationships across diverse network traffic data. By leveraging sensors to monitor system behavior, the data is subsequently stored in a cloud-based infrastructure for in-depth analysis. The effectiveness of ROAST-IoT is evaluated using benchmark datasets, including IoT-23 [16], Edge-IIoT [17], ToN-IoT [18], and UNSW-NB15 [19]. In ref. [16], a new classifier algorithm is proposed, focusing on the identification of malicious network traffic within IoT environments through

advanced machine learning techniques. The study employs a real-world IoT dataset, assessing the performance of various classification algorithms to determine their efficacy in detecting harmful traffic. In ref. [20], the authors establish key constraints for realistic adversarial cyber-attack scenarios and introduce a robust framework for adversarial analysis, centered on an evasion attack vector. This framework is utilized to evaluate the performance of three supervised learning algorithms: random forest (RF), extreme gradient boosting (XGB), and light gradient boosting machine (LGBM), alongside one unsupervised algorithm, isolation forest (IFOR). In ref. [21], the study focuses on three primary machine learning techniques, applied for both binary and multi-class classification within an IDS designed to detect IoT-based attacks. The IoT-23 dataset [16], a comprehensive and up-to-date collection, serves as the foundation for developing an intelligent IDS capable of identifying and categorizing attack types in IoT environments. In ref. [18], the authors address the challenge of creating an IoT/IoT dataset that includes labeled ground truth differentiating between normal and attack classes. The dataset also features attack sub-classes for more detailed multi-classification tasks. Known as ToN-IoT, the dataset encompasses telemetry data from IoT/IoT services, OS logs, and network traffic, gathered from a realistic simulation of a medium-scale IoT network, conducted at the University of New South Wales (UNSW) Canberra's Cyber Range and IoT Labs. In ref. [22], the study utilizes PySpark with Apache Spark within the Google Colaboratory (Colab) environment, incorporating Keras and Scikit-learn libraries. The authors employ the 'CICIoT2023' and 'TON_IoT' datasets for model training and testing, refining the features through correlation analysis to reduce dimensionality. A hybrid deep learning model, integrating one-dimensional CNN and LSTM, is then proposed to optimize performance. Additionally, ref. [20] explores adversarial robustness by defining constraints for realistic cyber-attacks and introducing a comprehensive evaluation method. This approach is used to test the effectiveness of RF, XGB, LGBM, and IFOR algorithms under adversarial conditions.

This study conducts an extensive evaluation of deep learning models, demonstrating their potential when integrated with big data analytics to optimize IDS. In ref. [2], a deep neural network (DNN) model achieves an outstanding 99.99% accuracy in binary classification, underscoring the synergy between deep learning techniques and big data in enhancing IDS effectiveness. The research utilizes three classifiers, random forest, gradient boosting tree (GBT), and a deep feed-forward neural network, to classify network traffic, while employing a homogeneity measure to extract the most relevant features from the datasets. In ref. [23], a DNN model is proposed, achieving 93.1% accuracy for binary classification, with a focus on developing a robust and adaptable IDS capable of detecting both known and emerging cyber threats. Recognizing the ever-evolving nature of network environments and the rapid emergence of new attack vectors, the study evaluates various datasets using both static and dynamic analysis techniques to identify optimal methods for detecting novel threats. It compares the performance of DNN models with traditional machine learning classifiers on a variety of publicly available malware datasets. The authors in ref. [24] present a DNN-based IDS model that achieves a 99% accuracy rate, applied to a newly constructed dataset containing both packet-based and flow-based data, as well as associated metadata. Despite the dataset's imbalance and the inclusion of 79 attributes, some representing classes with minimal training samples, the research highlights the capacity of deep learning to mitigate the issues inherent in imbalanced datasets. Meanwhile, ref. [25] introduces a stacked auto encoder (SAE) model, achieving a remarkable 99.92% accuracy. The study outlines an innovative IDS framework comprising five core components: data preprocessing, auto encoder compression, database storage, classification, and feedback. By compressing the preprocessed data to extract lower-dimensional features, the auto encoder enables more efficient classification while storing the compressed data in a database for future forensic analysis, post-attack evaluations, and model retraining. In ref. [26], a long short-term memory (LSTM) model is proposed, achieving 92.2% accuracy in binary classification by incorporating attention mechanisms to enhance the capture of both temporal and spatial features in network traffic data. This model is tested on the UNSW-NB15 dataset,

which offers diverse patterns and significant disparities between training and testing sets, making it an ideal challenge for evaluating model performance. In ref. [27], the authors propose a hybrid CNN-BiLSTM model, which achieves 97.90% accuracy in binary classification. This model combines bidirectional LSTMs with a lightweight CNN architecture and utilizes feature selection methods to reduce complexity while maintaining robust detection performance. Similarly, in ref. [28], a random forest model is presented, achieving 98.6% accuracy in detecting network attacks on the UNSW-NB15 dataset. This comprehensive study employs advanced machine learning and deep learning techniques to create a highly effective attack detection strategy. Finally, in ref. [2], another DNN model reaches 99.16% accuracy in classifying network traffic, utilizing five-fold cross-validation and incorporating ensemble learning methods. The model leverages the Apache Spark MLlib alongside the Keras deep learning framework, illustrating the powerful capabilities of deep learning and big data technologies in addressing complex network security challenges.

In ref. [29], a LSTM-based recurrent neural network (RNN) was trained as a category classifier, utilizing a dataset with 122 features. This model achieved a test accuracy of 82.68%, demonstrating its ability to manage complex classification tasks. The authors in ref. [30] addressed the issue of class imbalance by combining a CNN with a Bidirectional LSTM (BiLSTM) and integrating ADASYN, resulting in a notable accuracy of 90.73% on the test set. In ref. [31], performance enhancement was achieved by optimizing an auto encoder network for anomaly detection, yielding a test accuracy of 90.61%. Meanwhile, ref. [32] introduced a multi-CNN model with discrete data preprocessing steps, which effectively classified attacks on the test set, achieving an accuracy of 83%. In ref. [33], advancements in IDS for cloud environments were explored by developing and evaluating two cutting-edge deep neural network models. The first model was a multi-layer perceptron (MLP) trained using backpropagation (BP), while the second incorporated particle swarm optimization (PSO) into the MLP training process. Both models demonstrated a significant improvement in IDS performance and efficiency, achieving an impressive accuracy of 98.97%. This underscores their effectiveness in both intrusion detection and prevention. In ref. [34], the efficacy of deep learning algorithms for network intrusion detection was evaluated by comparing frameworks such as Keras, TensorFlow, Theano, fast.ai, and PyTorch. The researchers employed an MLP model and achieved a test accuracy of 98.68% in identifying network intrusion traffic and classifying various attack types, utilizing the CSE-CIC-IDS2018 dataset for validation. Similarly, [35] presented an innovative IDS model that employed a custom-designed recurrent convolutional neural network (RCNN) optimized through the ant lion optimization algorithm, achieving a test accuracy of 94%. This approach significantly improved IDS performance, particularly in detecting and mitigating network intrusions. In ref. [36], a deep learning framework was proposed to enhance IDS by utilizing a denoising auto encoder (DAE) as the central component of the methodology. The DAE was trained using a layer-wise greedy approach to prevent overfitting and avoid local optima, achieving a robust test accuracy of 96.53%. This strategy ensured higher reliability in detecting network intrusions. In ref. [37], a hidden naïve Bayes (HNB) classifier was introduced, specifically tailored to counter denial of service (DoS) attacks. By relaxing the traditional naïve Bayes assumption of conditional independence, the model incorporated discretization and feature selection techniques, achieving a test accuracy of 97%. This approach not only enhanced performance but also minimized processing time by prioritizing the most relevant features. Lastly, in ref. [38], the authors developed a novel classifier by employing a cascade of boosting-based artificial neural networks (ANNs) on two prominent intrusion detection datasets. Their method, which achieved a test accuracy of 98.25%, significantly improved upon the traditional one-vs-remaining strategy by introducing an additional example filtering step, ultimately boosting the model's overall effectiveness.

In ref. [39], the authors present the design of an IDS tailored for IIoT networks, employing the RF model for classification. The methodology incorporates PCC to identify and select critical features, alongside IF to detect outliers. Both PCC and IF are applied both

independently and in an interchangeable sequence, allowing PCC to process features that are then refined by IF, and alternatively, IF processes initial data for PCC to further optimize feature selection. This iterative application enhances the robustness and precision of the IDS model. In ref. [10], the paper addresses the identified limitation by proposing and assessing standardized feature sets for network intrusion detection systems (NIDS) that are derived from the NetFlow network metadata collection protocol and system. We conduct a thorough evaluation and comparison of two distinct variants of NetFlow-based feature sets, one comprising 12 features and the other encompassing 43 features. For our analysis, we transformed four widely utilized NIDS datasets into new versions that incorporate our proposed NetFlow-based feature sets. Utilizing an Extra Tree classifier, we systematically compared the classification performance of these NetFlow-based feature sets against the proprietary feature sets originally provided with the datasets.

Our proposed Transformer-CNN model unequivocally outperforms existing models, as evidenced by a thorough analysis of previous research. On the NF-UNSW-NB15-v2 dataset, the model demonstrates superior performance, achieving an outstanding 99.71% accuracy in binary classification. Similarly, on the CICIDS2017 dataset, the model achieves an impressive 99.93% accuracy in binary classification, further underscoring its effectiveness across diverse datasets. These results mark a significant advancement over those reported in prior studies, highlighting the robustness and adaptability of the proposed model. A comprehensive comparison of our model's performance against other relevant approaches is provided in Table 1, highlighting its exceptional capabilities.

2.2. Multi-Class Classification

In the domain of multi-class classification for IDS, the integration of the Transformer-CNN model offers a powerful solution. The Transformer component is utilized for contextual feature extraction, effectively analyzing relationships and patterns within the network traffic data. This enables the model to capture critical information and context surrounding the data. Following this, the CNN processes the extracted features, demonstrating exceptional proficiency in identifying complex patterns and anomalies. By leveraging this combined approach within the Transformer-CNN framework, the IDS significantly enhances its capability to accurately differentiate between various attack types, thereby improving detection precision and strengthening the overall security posture of the system.

In ref. [16], the authors propose a novel classification algorithm aimed at identifying malicious traffic within IoT environments through the application of machine learning techniques. This method employs a real-world IoT dataset that accurately reflects actual traffic conditions and evaluates the effectiveness of various classification algorithms. In ref. [40], the authors investigate IoT network security by analyzing the performance of machine learning algorithms for anomaly detection in network data. The study conducts a comprehensive comparative analysis of several machine learning algorithms that have demonstrated efficacy in similar contexts, utilizing a range of parameters and methodologies. In ref. [41], the authors delve into the application of different machine learning and deep learning techniques, alongside established datasets, to bolster IoT security. This research emphasizes the creation of a deep learning-based algorithm specifically tailored for detecting DoS attacks. In ref. [42], the authors examine strategies for addressing missing values in real-world computational intelligence applications. They conducted two experimental campaigns to evaluate various imputation methods for missing data, focusing on their influence on classifiers based on random forests. These classifiers were trained using contemporary cybersecurity benchmark datasets, such as CICIDS2017 and IoT-23.

Table 1. Related work in binary classification.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
Anandaraj Mahalingam et al. [15]	IoT-23	2023	ROAST-IoT	99.15%	This paper introduces ROAST-IoT, an AI-based model designed for efficient intrusion detection in IoT environments. It employs a multi-modal architecture to capture complex relationships in diverse network traffic data. System behavior is continuously monitored by sensors and stored on a cloud server for analysis. The model’s performance is thoroughly evaluated using benchmark datasets, including IoT-23, Edge-IIoT, ToN-IoT, and UNSW-NB15.	The study acknowledges its limitations, particularly the necessity to integrate a broader range of deep learning models to enhance the security of IIoT networks against cyber threats.
Mohamed ElKashlan et al. [16]	IoT-23	2023	Filtered classifier	99.2%	This paper presents a classifier algorithm specifically developed to identify malicious traffic within IoT environments through the application of machine learning techniques. The proposed system leverages an authentic IoT dataset derived from actual IoT traffic, evaluating the performance of multiple classification algorithms in the process.	The primary limitation of this study lies in its reliance solely on the IoT-23 dataset, which may not encompass the full spectrum of attack scenarios within IoT EVCS environments. Future research should incorporate a broader range of datasets and advanced deep learning techniques to enable a more comprehensive evaluation.
João Vitorino et al. [20]	IoT-23	2023	RF, XGB, LGBM, and IFOR	99%	This study delineates the essential constraints required for the development of a realistic adversarial cyber-attack and introduces a methodology for performing a reliable robustness analysis through a practical adversarial evasion attack vector. The proposed approach was employed to evaluate the robustness of three supervised machine learning algorithms: RF, XGB, and LGBM, along with one unsupervised algorithm, IFOR.	A significant limitation of this research is that, although adversarial training improves model resilience, certain models like LGBM remain highly vulnerable to adversarial examples, especially in imbalanced multi-class classification scenarios. This highlights the necessity for further exploration of effective defense strategies and the assessment of these models using new datasets and diverse attack methods.

Table 1. Cont.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
Trifa S. Othman and Saman M. Abdullah [21]	IoT-23	2023	ANN	99%	This research presents three leading machine learning methodologies employed for binary and multi-class classification, serving as the foundation of an intrusion detection system designed to safeguard Internet of Things environments. These approaches are utilized to identify a range of cyber threats targeting IoT devices while effectively categorizing their respective types. By harnessing the cutting-edge IoT-23 dataset, the study constructs a sophisticated intelligent intrusion detection system capable of detecting malicious behaviors and classifying attack vectors in real-time, thereby bolstering the security posture of IoT networks.	A significant limitation highlighted in this research is the failure of the SMOTE to enhance the accuracy of the proposed intelligent intrusion detection system model on the IoT-23 dataset, despite its usual efficacy in addressing issues related to imbalanced datasets.
Abdallah R. Gad et al. [18]	ToN-IoT	2020	XGBoost	98.2%	This paper presents a new dataset, TON_IoT, designed for the IoT and IIoT, which includes labeled ground truth to differentiate between normal operations and various attack classes. The dataset features attributes for identifying attack subclasses, supporting multi-class classification. It contains telemetry data, operating system logs, and network traffic, collected from a realistic medium-scale network simulation at UNSW Canberra, Australia. Overall, the study significantly enhances the effectiveness of intrusion detection systems in IoT environments by providing a comprehensive dataset for improved classification accuracy.	This study acknowledges several limitations, particularly the presence of class imbalance and missing values within the ToN-IoT dataset. Although techniques like Chi-squared for feature selection and SMOTE for class balancing were employed, these issues could hinder the model's ability to generalize effectively and scale in practical, real-world scenarios. Consequently, the findings may not fully reflect the model's performance under diverse operational conditions, highlighting a need for further refinement and validation in future research.

Table 1. Cont.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
Sami Yaras and Murat Dener [22]	ToN-IoT	2024	CNN-LSTM	98.75%	This study employed PySpark with Apache Spark in Google Colaboratory, utilizing Keras and Scikit-Learn to analyze the ‘CI-CIoT2023’ and ‘TON_IoT’ datasets. It focused on feature reduction via correlation to enhance model relevance and developed a hybrid deep learning algorithm combining one-dimensional CNN and LSTM for better performance. Overall, the research showcases advanced deep learning applications for improving IoT intrusion detection.	• A significant limitation of this study is that, although it attained high accuracy levels, the extensive data volumes resulted in prolonged training and testing durations. This emphasizes the necessity for future optimization efforts to achieve a balance between accuracy, computational efficiency, and cost-effectiveness.
João Vitorino et al [20].	ToN-IoT	2023	RF, XGB, LGBM, and IFOR	85%	This research outlines the critical requirements for developing a credible adversarial cyber-attack and presents a framework for conducting a reliable robustness analysis with a practical adversarial evasion attack vector. The framework was employed to assess the robustness of three supervised machine learning algorithms: random forest, XGBoost, and LightGBM, alongside one unsupervised algorithm, isolation forest.	• The primary limitation of this study is that, despite the improvements in model resilience achieved through adversarial training, specific models such as LightGBM remain highly susceptible to adversarial examples in the context of imbalanced multi-class classification. This underscores the necessity for additional research into defense strategies and the evaluation of new datasets and attack methods.
Osama Faker and Erdogan Dogdu [2].	CICIDS2017	2019	DNN	99.9%	This research improves intrusion detection by integrating deep learning with big data techniques, employing random forest, gradient boosting trees, and a deep feed-forward neural network. It assesses feature importance and evaluates performance on the UNSW-NB15 and CICIDS2017 datasets using five-fold cross-validation. The approach combines Keras with Apache Spark and ensemble methods for enhanced analysis.	• The paper inadequately tackles scalability issues linked to distributed processing and provides minimal investigation into advanced feature selection methods.

Table 1. Cont.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
R. Vinaya-Kumar et al. [23]	CICIDS2017	2019	DNN	93.1%	This study explores deep neural networks for a versatile intrusion detection system capable of identifying and categorizing new cyber-attacks, evaluating their performance against conventional machine learning classifiers using standard benchmark datasets.	Limited scalability and performance evaluation of distributed systems and advanced deep neural network architectures.
Kaniz Farhana et al. [24]	CICIDS2017	2020	DNN	99%	This research presents a deep neural network-based intrusion detection system developed with Keras in the TensorFlow environment. The model utilizes a recent imbalanced dataset containing 79 features, comprising packet-level, flow-level data, and metadata, with notable underrepresentation of specific classes.	The model's inability to accurately classify 'Heartbleed,' 'Infiltration,' and 'Web Attack SQL Injection' highlights issues related to class imbalance, stemming from the insufficient number of records for these specific attack types.
Chongzhen Zhang et al. [25]	CICIDS2017	2021	SAE	99.92%	This research proposes a robust intrusion detection system framework consisting of five interconnected modules: pre-processing, autoencoder, database, classification, and feedback. The autoencoder reduces data size, while the classification module produces results, and the database retains compressed features for future analysis and model retraining.	The framework's restoration and retraining capabilities require improvements to enhance its adaptability and overall performance.
Mohammad A. Alsharaiah et al. [26]	UNSW-NB15	2024	AT-LSTM	92.2%	This study introduces a novel network intrusion detection system that employs LSTM networks and attention mechanisms to analyze the temporal and spatial characteristics of network traffic. Utilizing the UNSW-NB15 dataset, the approach evaluates different training and testing set sizes.	- Intricate architecture. - Although the AT-LSTM model demonstrates impressive accuracy on the UNSW-NB15 dataset, it fails to tackle class imbalance and lacks evaluation on alternative datasets, such as NSL-KDD. This limitation may hinder its applicability and effectiveness across various contexts.

Table 1. Cont.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
Mohammed Jouhari et al. [27]	UNSW-NB15	2024	CNN-BiLSTM	97.90%	This research presents a robust intrusion detection system model that combines BiLSTM with a lightweight CNN. The approach incorporates feature selection techniques to streamline the model, enhancing its efficiency and effectiveness in detecting threats.	• Intricate architecture. • This research focused on enhancing the intrusion detection system model to address computational limitations, potentially overlooking important factors like broader generalization and robustness across various datasets.
Fuat Türk [28]	UNSW-NB15	2023	RF	98.6%	This study achieved high attack detection rates on the UNSW-NB15 dataset, recording 98.6% accuracy in binary classification and 98.3% in multi-class classification by employing sophisticated machine learning and deep learning methods.	• Misclassification of attack classes suggests the necessity for improved dataset balancing and the implementation of real-time model updates to boost overall performance.
Osama Faker and Erdogan Dogdu [2]	UNSW-NB15	2019	DNN	99.16%	This study assesses machine learning models through 5-fold cross-validation, employs ensemble techniques alongside Apache Spark, and integrates deep learning by merging Apache Spark with Keras.	• The paper fails to address scalability concerning distributed processing and does not explore advanced feature selection methods.
Pramita Sree Muhuri et al. [29]	NSL KDD	2020	Long Short-Term Memory Recurrent Neural Network (LSTM-RNN)	96.51%	This research presents a novel intrusion detection approach that integrates recurrent neural networks with long short-term memory, utilizing a genetic algorithm for optimal feature selection. The findings indicate that LSTM-RNN classifiers enhance intrusion detection effectiveness on the NSL-KDD dataset when supplied with suitable features.	• Restricted to binary classification. • Intricate architecture. • The study did not evaluate training duration or test the model on a GPU-based system, relying solely on the NSL-KDD dataset. This limitation may prevent a comprehensive understanding of the LSTM-RNN model's performance on more current datasets or real-time network traffic.

Table 1. Cont.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
Yanfang Fu et al. [30]	NSL KDD	2022	CNN and BiLSTMs	90.73%	This research presents DLNID, an advanced model for detecting traffic anomalies that combines an attention mechanism with Bi-LSTM to improve accuracy. The model employs CNN for feature extraction, enhances channel weights through attention, and utilizes Bi-LSTM to learn sequence features effectively.	• Restricted to binary classification. • Intricate architecture. • While the DLNID model performs well on the KDDTest+ dataset, it has not been validated in real-world scenarios or for online intrusion detection applications.
Wen Xu et al. [31]	NSL KDD	2021	Auto Encoder	90.61%	This research introduces a novel five-layer auto encoder architecture for detecting network anomalies, along with a thorough assessment of its performance metrics.	• Restricted to binary classification. • Although the five-layer auto encoder model achieves strong results on the NSL-KDD dataset, its effectiveness in real-world settings and against various intrusion types and datasets has yet to be validated.
Jihoon Yoo et al. [32]	NSL KDD	2021	CNN	83%	This research explores a convolutional neural network classifier aimed at mitigating class imbalance in network traffic data. It employs a preprocessing technique that transforms one-dimensional packet vectors into two-dimensional images and uses discretization to enhance relational analysis and overall model generalization.	• Inadequate accuracy. • Restricted to binary classification. • While the proposed approach improves performance for certain classes and simplifies the model, it falls short for others and may not effectively resolve class imbalance issues.

Table 1. Cont.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
Saud Alzughaibi and Salim El Khediri [33]	CSE-CIC-IDS2018	2023	MLP-BP, MLP-PSO	98.97%	This research enhances intrusion detection systems for cloud environments by developing and assessing two deep neural network models: one based on a multilayer perceptron with backpropagation and the other utilizing particle swarm optimization. These models aim to improve the efficiency and effectiveness of detecting and responding to intrusions.	• Inadequate accuracy. • Intricate architecture. • The study reports high accuracy for both the multilayer perceptron with backpropagation and the multilayer perceptron with particle swarm optimization models. However, these models have yet to be evaluated in real-time or cloud environments. Exploring additional optimization algorithms may further improve the outcomes.
Ram B. Basnet et al. [34]	CSE-CIC-IDS2018	2019	MLP	98.68%	This article assesses various deep learning algorithms for network intrusion detection by comparing frameworks including Keras, TensorFlow, Theano, fast.ai, and PyTorch, utilizing the CSE-CIC-IDS2018 dataset for evaluation.	• The study showcases fast.ai's outstanding performance and efficiency; however, it does not evaluate this framework with different datasets or in real-world settings. Future research should prioritize hyperparameter tuning and investigate alternative deep learning algorithms.
T. Thilagam and R. Aruna [35]	CSE-CIC-IDS2018	2021	RC-NN-IDS	94%	This paper presents a sophisticated IDS that utilizes a customized RC-NN enhanced by the ALO algorithm, with the goal of markedly improving the system's effectiveness.	• Intricate architecture. • The proposed RC-NN model outperforms existing classifiers in intrusion detection; however, it is missing a management module to initiate preventive measures following detection.

Table 1. Cont.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
Fahimeh Farahnakian and Jukka Heikkonen [36]	KDD-CUP'99	2018	DAE	96.53%	To address this challenge, the authors propose an IDS that employs the widely recognized DAE model. By training the DAE through a greedy layer-wise method, they aim to reduce overfitting and avoid local optima, resulting in a more resilient and efficient detection system.	• KDD99 employed this dataset, which displays a greater level of redundancy. • The deep DAE model demonstrates impressive accuracy; however, it does not explore the potential benefits of applying sparsity constraints or consider other deep learning approaches for additional improvement.
Hafza A. Mahmood and Soukaena H. Hashem [37]	KDD-CUP'99	2017	HNB	97%	This paper advocates for the use of a HNB classifier to address DoS attacks. The HNB model, which improves upon traditional naive Bayes by easing its conditional independence assumption, combines discretization and feature selection techniques. This approach aims to enhance detection performance while reducing processing time through optimized feature relevance.	• KDD99 employed this dataset, which displays a greater level of redundancy. • The proposed system effectively detects DoS attacks with high accuracy by utilizing targeted feature selections from the KDD Cup 99 dataset. However, it fails to consider limitations associated with the NSL KDD dataset and does not explore how different feature selections might impact performance in various cloud environments.
Mirza M. Baig et al. [38]	KDD-CUP'99	2017	ANNs	98.25%	The authors present a robust classifier development method utilizing a cascade of boosting-based ANNs, validated on two intrusion detection datasets. This technique, similar to the one-vs-remaining strategy but enhanced with extra example filtering, enhances classifier performance.	• KDD99 employed this dataset, which displays a greater level of redundancy. • The proposed method performs well with adequately represented classes in the KDD'99 dataset but struggles with sparse classes, showing comparatively lower effectiveness on the UNSW-NB15 dataset. This underscores the necessity for improved management of sparse classes and broader testing.

Table 1. Cont.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
Mouaad Mohy-Eddine et al. [39]	NF-UNSW-NB15-v2	2022	RF	99.30%	In this study, we design an IDS for IIoT networks utilizing the RF model for classification. The approach integrates PCC for selecting relevant features and IF as an outlier detection mechanism. PCC and IF are applied independently as well as interchangeably, with PCC feeding its output to IF and, conversely, IF supplying its output to PCC in different iterations.	A limitation of this study is that the IDS model's effectiveness has only been validated on a limited set of datasets, suggesting a need for broader evaluation across diverse IIoT and IoT datasets to ensure generalized applicability.
Mohanad Sarhan et al. [10]	NF-UNSW-NB15-v2	2022	Extra Tree classifier	99.7	This study addresses limitations in NIDS by introducing and evaluating standardized feature sets based on the NetFlow metadata collection protocol. It systematically compares two variants of these feature sets, one with 12 features and another with 43 features. The study reformulates four well-known NIDS datasets to incorporate these NetFlow-based feature sets. Utilizing an Extra Tree classifier, it assesses the classification performance of the NetFlow-derived feature sets against the original proprietary feature sets included in the datasets.	Although this study significantly contributes to the establishment of a standardized NetFlow-based feature set for NIDS, it is constrained by its dependence on existing benchmark datasets, which may not comprehensively capture the full spectrum of real-world network environments and diverse attack scenarios encountered in practice.

In ref. [18], the authors tackle the challenge of identifying malicious activities in IoT/IIoT environments by introducing a new dataset, named TON_IoT, which features labeled ground truth to differentiate between normal operations and attack classes. This dataset further enriches the classification process by including a feature that categorizes various attack subclasses, thereby facilitating multi-class classification. TON_IoT comprises telemetry data from IoT/IIoT services, operating system logs, and network traffic, all gathered from a realistic medium-scale network simulation conducted at the Cyber Range and IoT Labs at UNSW Canberra, Australia. In ref. [43], the authors utilized both machine learning and deep learning algorithms to investigate DoS and DDoS attacks. The analysis was conducted using the Bot-IoT dataset, developed by the UNSW Canberra Cyber Centre, with relevant features extracted from the pcap files of the UNSW dataset using ARGUS software, allowing for an in-depth examination of attack patterns. Additionally, in ref. [43], the authors propose a novel framework known as the privacy-preserving intrusion detection framework (P2IDF), specifically designed for traffic in Software-Defined IoT and Fog networks. This framework employs a SAE technique to transform raw data into an encoded format, effectively safeguarding against inference attacks. Subsequently, an IDS based on ANN is integrated and evaluated using the ToN-IoT dataset to discern normal from malicious traffic before and after the data transformation. This dual approach enhances the security of IoT-Fog networks while maintaining data confidentiality. In ref. [44], the authors conducted a thorough assessment of feature importance across six network NIDS datasets. They employed three feature selection techniques: Chi-square, information gain (IG), and correlation to rank features according to their predictive significance. These features were then assessed using deep feed forward networks (DFF) and RF classifiers, leading to a total of 414 experiments. A major finding from this study is that a carefully selected subset of features can deliver equal or even superior detection performance compared to using the full feature set, highlighting the efficiency of feature reduction in NIDS performance. In ref. [45], the authors present a novel, comprehensive cyber security dataset tailored for IoT and IIoT applications, named Edge-IIoTset. This dataset is designed for use with machine learning-based intrusion detection systems, supporting both centralized and federated learning modes. It was created using a purpose-built IoT/IIoT testbed, which incorporates a wide array of representative devices, sensors, protocols, and cloud/edge configurations, ensuring its relevance and applicability in real-world scenarios.

In ref. [23], a versatile DNN model reaches 95.6% accuracy, emphasizing the evaluation of diverse datasets via static and dynamic methodologies to effectively detect emerging cyber threats. The research in ref. [24] presents a DNN model achieving 99% accuracy while addressing imbalances in labeled datasets through a comprehensive analysis of packet-based and flow-based data. In ref. [46], the authors suggest a CNN-gated recurrent unit (GRU) approach, achieving an accuracy of 98.73%. This method optimizes network parameters by combining CNN and GRU, demonstrating various CNN-GRU combinations. The CICIDS-2017 benchmark dataset is used, and evaluation metrics such as recall, precision, false positive rate (FPR), and true positive rate (TPR) are employed. Another study [2] reports a DNN model achieving 97.01% accuracy with five-fold cross-validation and Apache Spark for distributed computing. The work of ref. [47] showcases an ANN model with a notable 99.59% accuracy, employing a holistic dataset approach to enhance deep learning performance. Similarly, an RF model in ref. [48] achieves 97.37% accuracy by addressing dataset imbalance and dimensionality through feature clustering techniques. In ref. [28], an RF model extends its attack detection methodology to the UNSW-NB15 dataset, reaching 98.3% accuracy, while an RNN model in ref. [49] achieves 94% accuracy, utilizing recursive feature elimination to improve classification across various attack categories. Furthermore, [50] introduces a multilayer CNN combined with LSTM networks, achieving 99.5% accuracy, and [51] presents a method utilizing sparse stacked auto encoders, achieving 98.5% accuracy through a three-stage process. Finally, ref. [52] introduces an LSTM model with a commendable accuracy of 96.9%.

In ref. [34], the research investigates the effectiveness of several deep learning frameworks for network intrusion detection, comparing notable options such as Keras, TensorFlow, Theano, fast.ai, and PyTorch, along with MLP integration. The study reports an impressive accuracy of 98.31% in identifying and classifying network intrusion traffic and various attack types using the CSE-CIC-IDS2018 dataset. Additionally, the study in ref. [53] underscores the critical importance of cyber security in safeguarding network infrastructures against vulnerabilities and intrusions, highlighting advancements in machine learning and deep learning techniques that facilitate early detection and prevention of attacks through advanced self-learning and feature extraction methods. Leveraging the CSE-CIC-IDS2018 dataset, which encompasses normal network behaviors and diverse attacks, an LSTM model achieved an outstanding detection accuracy of 99%. In ref. [54], the authors evaluate a DNN model that demonstrates approximately 90% accuracy, emphasizing its capability to effectively identify network intrusions. The work presented in ref. [55] introduces a dynamic network anomaly detection system aimed at bolstering network security through deep learning techniques, specifically a deep neural network based on LSTM integrated with an attention mechanism (AM) to enhance performance; this approach addresses class imbalance in the CSE-CIC-IDS2018 dataset using SMOTE and an enhanced loss function, resulting in 96.2% accuracy. The study detailed in ref. [38] proposes an advanced classifier development technique that employs a cascade of boosting-based ANNs to construct a highly effective multi-class classifier, utilizing a one-vs-remaining strategy refined with example filtering, ultimately achieving an impressive accuracy of 99.36%. Furthermore, in ref. [23], the authors focus on developing a DNN for a flexible and effective IDS capable of detecting and classifying novel cyber-attacks. Recognizing the dynamic nature of network behaviors and attack strategies, they emphasize the necessity of evaluating datasets generated through both static and dynamic methods over time; the proposed DNN model achieves robust performance with an accuracy of 93.5%, demonstrating its adaptability for real-time threat detection. Lastly, the authors in ref. [56] execute a multi-class classification experiment for network intrusion detection utilizing the KDD-CUP 99 and NSL-KDD datasets, employing a CNN to achieve a remarkable accuracy of 98.2%, thereby showcasing its efficacy in accurately identifying various network attack types.

In ref. [10], the study seeks to address the identified limitation by proposing and systematically evaluating standardized feature sets for NIDS that are derived from the NetFlow network metadata collection protocol and system. We conduct a detailed assessment of two distinct variants of NetFlow-based feature sets, one comprising 12 features and the other encompassing 43 features. For our evaluation, we transformed four widely recognized NIDS datasets into revised versions that integrate these proposed NetFlow-based feature sets. Utilizing an Extra Tree classifier as the analytical framework, we meticulously compare the classification performance of the NetFlow-derived feature sets with the proprietary feature sets that accompany the original datasets, thereby providing a comprehensive analysis of their relative effectiveness in detecting network intrusions. In ref. [57], the paper introduces a conditional generative adversarial network (CGAN) enhanced by bidirectional encoder representations from transformers (BERT), a sophisticated pre-trained language model, aimed at improving multi-class intrusion detection. The proposed method leverages CGAN to augment minority attack data, effectively addressing the issue of class imbalance. Additionally, BERT is incorporated into the CGAN discriminator, facilitating robust feature extraction that strengthens input-output dependencies and enhances detection capabilities through adversarial training.

Through an extensive review of prior research, we firmly establish that our proposed Transformer-CNN model markedly surpasses current methodologies in performance. This cutting-edge model attains an impressive accuracy of 99.02% in multi-class classification on the NF-UNSW-NB15-v2 dataset and 99.13% on the CICIDS2017 dataset, underscoring its effectiveness across diverse datasets. These outcomes not only demonstrate the efficacy of our model but also highlight the significant advancements it brings to the field of intrusion detection. A comprehensive comparison of our findings with relevant studies is presented in Table 2.

Table 2. Related work in multi-class classification.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
Mohamed ElKashlan et al. [16]	IoT-23	2023	Filtered classifier	99.2%	This paper proposes a new machine learning-based classifier for detecting malicious traffic in IoT networks, using a real-world IoT dataset to assess the performance of different algorithms.	The study's reliance on the IoT-23 dataset limits its coverage of all IoT EVCS attack scenarios. Future work should explore diverse datasets and advanced deep learning for more comprehensive results.
Nicolas-Alin Stoian [40]	IoT-23	2020	RF	99.5%	This paper explores IoT network security by evaluating the effectiveness of various ML algorithms for anomaly detection through comparative analysis.	The study faces challenges with data handling, correlation removal, and MLP performance. Future work should utilize the full dataset, optimize data needs, analyze decision tree accuracy, and explore advanced neural networks.
Bambang Susilo and Riri Fitri Sari [41]	IoT-23	2020	CNN	91.24%	This research employs ML and DL techniques with standard datasets to enhance IoT security, developing a DL-based algorithm for DoS attack detection.	The study is limited by its focus on RF, CNN, and MLP, with further research needed to optimize batch sizes and integrate multiple ML or DL models for real-time intrusion detection.
Mateusz Szczepański et al. [42]	IoT-23	2022	RF	96.30%	This paper tackles the challenge of handling missing values in computational intelligence applications. It presents two experiments assessing different imputation methods for missing values in random forest classifiers trained on modern cybersecurity benchmark datasets like CICIDS2017 and IoT-23.	The study is limited by its emphasis on comparing imputation methods without examining the impact of deep learning imputation on various ML classifiers. Future research should fill this gap by exploring explainability techniques and the latent representations of autoencoders.

Table 2. Cont.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
Abdallah R. Gad et al. [18]	ToN-IoT	2020	XGBoost	97.8%	This paper addresses the challenge by introducing a novel data-driven IoT/IIoT dataset called TON_IoT, which includes ground truth labels to distinguish between normal and attack classes. It features an additional attribute for various attack subclasses, allowing for multi-class classification. The dataset comprises telemetry data from IoT/IIoT services, operating system logs, and network traffic, all collected from a realistic medium-scale network environment at the Cyber Range and IoT Labs at UNSW Canberra, Australia.	This study's limitations include class imbalance and missing values in the ToN-IoT dataset. Although Chi2 was used for feature selection and SMOTE for class balancing, these issues may affect the model's scalability and its ability to generalize to real-world scenarios.
Prahlad Kumar et al. [43]	Bot-IOT	2021	Decision trees (DT), RF, KNN, NB, and ANN	99.6%	This paper employs machine learning and deep learning techniques to conduct a thorough analysis of DoS and DDoS attacks. It utilizes the Bot-IoT dataset from the UNSW Canberra Cyber Centre as the main training resource. To achieve precise feature extraction, ARGUS software was used to process and derive features from the pcap files of the UNSW dataset. This methodology enables a detailed investigation of attack behaviors, aiding in the detection and classification of malicious activities within IoT environments.	The study concludes that both machine learning and deep learning models are effective in detecting DoS and DDoS attacks; however, deep learning models require more resources and are best suited for systems with ample resources. In contrast, machine learning models are more appropriate for environments with constrained resources and lower data traffic.
Prabhat Kumar et al. [43]	ToN-IoT	2021	ANN	99.44%	This paper presents a P2IDF for Software-Defined IoT-Fog networks, utilizing a SAE for data encoding to mitigate inference attacks. It assesses an ANN-based intrusion detection system on the ToN-IoT dataset, comparing performance before and after data transformation. The framework successfully identifies attacks while ensuring data privacy.	The study emphasizes that the P2IDF framework surpasses recent methods in terms of detection accuracy and precision. Future research will concentrate on creating a real-time prototype to tackle privacy and security issues in Software-Defined IoT-Fog networks.

Table 2. Cont.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
Mohanad Sarhan et al. [44]	ToN-IoT	2022	DFF, RF	96.10%, 97.35%	This paper assesses feature importance across six NIDS datasets by employing three feature selection techniques: Chi-square, information gain, and correlation analysis. The chosen features were evaluated using deep feed-forward networks and random forest classifiers, resulting in a total of 414 experiments. A significant finding is that a streamlined subset of features can achieve detection performance comparable to or better than that of the complete feature set, underscoring the value of feature selection in enhancing the efficiency and accuracy of NIDS.	A major limitation noted in this paper is the absence of a universal guideline for selecting optimal feature sets, as the importance of features varies considerably among different datasets and classifiers. This necessitates thorough analysis for each specific scenario. Additionally, some unrealistic features in synthetic datasets, such as TTL-based attributes in the UNSW-NB15 dataset, should be omitted to guarantee reliable evaluation outcomes.
Mohamed Amine Ferrag et al. [45]	Edge-IIoT	2022	DNN	94.67%	This paper presents Edge-IIoTset, an extensive cybersecurity dataset tailored for IoT and IIoT applications, specifically aimed at machine learning-based intrusion detection systems. It accommodates both centralized and federated learning models and was developed using a custom IoT/IIoT testbed featuring a wide range of devices, sensors, protocols, and cloud/edge configurations, thus ensuring its relevance in real-world scenarios.	The primary limitation of the proposed dataset is that, while it seeks to overcome the shortcomings of existing datasets by integrating new technologies and additional layers, its effectiveness and representativeness for evaluating machine learning-based intrusion detection systems across various real-world scenarios and emerging technologies still require comprehensive validation.

Table 2. Cont.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
R. Vinaya-Kumar et al. [23]	CICIDS2017	2019	DNN	95.6%	This research aims to develop a versatile intrusion detection system (IDS) by utilizing deep neural networks to detect and classify emerging cyber threats. It evaluates various datasets and algorithms, comparing DNNs with traditional classifiers using benchmark malware datasets to determine the most effective approach for identifying new threats.	There is a lack of scalability and performance analysis for distributed systems, as well as for advanced deep neural networks.
Kaniz Farhana et al. [24]	CICIDS2017	2020	DNN	99%	This study introduces an IDS based on deep neural networks, evaluated on a contemporary imbalanced dataset featuring 79 attributes. Built using Keras and TensorFlow, the model analyzes packet-based, flow-based data, and associated metadata.	The model's failure to classify 'Heartbleed,' 'Infiltration,' and 'Web Attack SQL Injection' underscores the challenges posed by class imbalance, stemming from an inadequate number of records for these specific attacks.
Azriel Henry et al. [46]	CICIDS2017	2023	CNN-GRU	98.73%	The study presents a method combining CNN and GRU for optimizing network parameters, evaluated using the CICIDS-2017 dataset and metrics such as recall, precision, FPR, and TPR.	- Complex architecture. - The proposed IDS model, despite its high accuracy, could benefit from improvements in handling imbalanced data, optimizing training for all attack types, and addressing accuracy, false alarms, and execution time in large-scale systems
Osama Faker and Erdogan Dogdu [2]	UNSW-NB15	2019	DNN	97.01%	This work evaluates machine learning models using five-fold cross-validation, employing Keras with Apache Spark for deep learning and leveraging Apache Spark MLlib for ensemble methods.	The paper fails to include an analysis of scalability concerning distributed processing and does not cover advanced feature selection techniques.

Table 2. Cont.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
A. M. Aleesa et al. [47]	UNSW-NB15	2021	ANN	99.59%	They evaluated the effectiveness of deep learning for both binary and multi-class classification using an updated dataset, consolidating all data into a single file and creating new multi-class labels based on different attack families.	The research is confined to controlled experiments utilizing the UNSW-NB15 dataset and does not consider the deployment of deep learning models in real-world environments.
Muhammad Ahmad et al. [48]	UNSW-NB15	2021	RF	97.37%	They introduce feature clusters for Flow, TCP, and MQTT derived from the UNSW-NB15 dataset to address issues of imbalance, dimensionality, and overfitting, using ANN, SVM, and RF for classification.	The model's ability to generalize to other IoT protocols and datasets may be constrained by the study's emphasis on particular protocols and imputation techniques.
Fuat Türk [28]	UNSW-NB15	2023	RF	98.3%	This article utilizes advanced machine learning and deep learning techniques for attack detection on the UNSW-NB15 and NSL-KDD datasets, achieving an accuracy of 98.6% in binary classification and 98.3% in multi-class classification for the UNSW-NB15 dataset.	Attack classes are occasionally misclassified, highlighting the necessity for improved dataset balancing and real-time model updates to enhance performance.
Bilal Mohammed, Ekhlās K. Gbashi [49]	NSL KDD	2021	RNN	94%	The study employs RFE for feature selection and utilizes DNN and RNN for classification, achieving an accuracy of 94% across five classes with the RNN model.	- Restricted to multi-class classification. - The system performs exceptionally well on the NSL-KDD dataset; however, it requires assessment on additional datasets, exploration of alternative feature selection methods, and implementation for real-time deployment.

Table 2. Cont.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
Muhammad Basit Umair et al. [50]	NSL KDD	2022	Multilayer CNN-LSTM	99.5%	To overcome the limitations of traditional methods, this paper presents a statistical approach for intrusion detection. It includes feature extraction, classification using a multilayer CNN with softmax activation, and additional classification through a multilayer DNN.	- Restricted to multi-class classification. - Intricate architecture. - The proposed IDS exhibits high accuracy and robust performance metrics; however, it has not been evaluated on diverse datasets or in real-world scenarios, which may impact its generalizability.
Padideh Choobdar et al. [51]	NSL KDD	2021	Sparse Stacked Auto-Encoders	98.5%	This work introduces a controller module for a SDN-based IDS, which includes pre-training with sparse stacking autoencoders, training with a softmax classifier, and parameter optimization.	- Restricted to multi-class classification. - The proposed IDS demonstrates high accuracy; however, it has not been tested on distributed SDN networks or advanced deep learning techniques such as GANs. Additionally, it could benefit from improved hardware to accelerate the training process.
Supriya Shende, Samrat Thorat [52]	NSL KDD	2020	LSTM	96.9%	The model, developed and evaluated with the NSL-KDD dataset, employs LSTM for efficient intrusion detection.	- Restricted to multi-class classification. - The LSTM-based intrusion detection method demonstrates high accuracy but may face challenges in generalizing to datasets beyond NSL-KDD.
Ram B. Basnet et al. [34]	CSE-CIC-IDS2018	2019	MLP	98.31%	In this article, the authors assess deep learning algorithms for network intrusion detection by exploring various frameworks, including Keras, TensorFlow, Theano, fast.ai, and PyTorch, utilizing the CSE-CIC-IDS2018 dataset.	The study emphasizes fast.ai's superior performance and efficiency but has not been tested on additional datasets or in real-world environments. Future research should concentrate on hyperparameter tuning and the investigation of other deep learning algorithms.

Table 2. Cont.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
Baraa Ismael Farhan and Ammar D. Jasim [53]	CSE-CIC-IDS2018	2022	LSTM	99%	The increasing demand for cybersecurity highlights the significance of effective network monitoring. This study employs deep learning techniques on the CSE-CIC-IDS2018 dataset, attaining 99% detection accuracy using an LSTM model to identify network attacks.	• Restricted to multi-class classification. • The proposed LSTM-based intrusion detection system achieves high accuracy but faces challenges due to dataset imbalance and its large size, which may impact accuracy and complicate model design.
Rawaa Ismael Farhan et al. [54]	CSE-CIC-IDS2018	2020	DNN	90%	In this paper, we evaluate our DNN model, which has achieved a significant detection accuracy of around 90%.	• Restricted to multi-class classification. • The proposed DNN model for flow-based intrusion detection reaches 90% accuracy but faces challenges related to large data size, high dimensionality, and data preprocessing. Tackling these issues will necessitate feature selection and hyperparameter tuning to improve performance.
Peng Lin et al. [55].	CSE-CIC-IDS2018	2019	LSTM	96.2%	To enhance network security, we developed a dynamic anomaly detection system utilizing deep learning techniques. This system employs an LSTM-based DNN model, augmented with an AM to boost performance. Additionally, the SMOTE algorithm and an advanced loss function are used to effectively tackle class imbalance in the CSE-CIC-IDS2018 dataset.	• Restricted to multi-class classification. • The system achieves high accuracy and recall but depends on pre-processed features, potentially limiting its ability to learn and adapt directly from raw network traffic data.

Table 2. Cont.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
Mirza M. Baig et al. [38]	KDD-CUP'99	2017	ANNs	99.36%	The authors propose a method that employs a cascade of boosting-based ANNs to develop an effective classifier. Tested on two intrusion detection datasets, this approach enhances the one-vs-remaining strategy with additional example filtering to boost accuracy.	• KDD99 employed this dataset, which displays a greater level of redundancy. • The proposed method performs effectively with well-represented classes in the KDD'99 dataset but struggles with sparse classes and exhibits lower performance on the UNSW-NB15 dataset, highlighting the need for better handling of sparse classes and broader testing.
R. Vinaya-Kumar et al. [23]	KDD-CUP'99	2019	DNN	93%	The authors develop a DNN-based IDS that attains 93% accuracy in detecting and classifying new cyber-attacks by analyzing various static and dynamic datasets.	• KDD99 employed this dataset, which displays a greater level of redundancy. • Inadequate analysis of scalability and performance in distributed systems and advanced DNNs.
Guojie Liu and Jianbiao Zhang [56]	KDD-CUP'99	2020	CNN	98.2%	This study performed multi-class network intrusion detection using the KDD-CUP 99 and NSL-KDD datasets. The CNN model achieved an impressive accuracy of 98.2%, showcasing its effectiveness in detecting various types of network attacks.	• KDD99 employed this dataset, which displays a greater level of redundancy. • Restricted to multi-class classification • The proposed model improves accuracy and recall but needs further refinement for better classification of unknown attacks and validation with real network traffic data.

Table 2. Cont.

Author	Dataset	Year	Utilized Technique	Accuracy	Contribution	Limitations
Mohanad Sarhan et al. [10]	NF-UNSW-NB15-v2	2022	Extra Tree classifier	98.9%	This paper addresses limitations in NIDS by proposing and evaluating standardized feature sets based on the NetFlow metadata collection protocol. It compares two variants of these feature sets, one with 12 features and another with 43 features, by reformulating four well-known NIDS datasets to include the proposed sets. Using an Extra Tree classifier, the study rigorously assesses and contrasts the classification performance of the NetFlow-based feature sets with the original proprietary feature sets, highlighting their effectiveness in intrusion detection.	While this study advances the development of a standardized NetFlow-based feature set for NIDS, it is limited by the reliance on existing benchmark datasets, which may not fully represent the diverse range of real-world network environments and attack scenarios encountered in practice.
Fang Li [57]	NF-UNSW-NB15-v2	2024	CGAN-BERT	87.40%	This study presents an innovative method that combines a CGAN with BERT to tackle multi-class intrusion detection challenges. The approach focuses on augmenting data for minority attack classes, addressing class imbalance issues. By integrating BERT into the CGAN’s discriminator, the framework strengthens input-output relationships and enhances detection capabilities through adversarial training, resulting in improved feature extraction and a more robust cybersecurity detection mechanism.	A key limitation of this study is the difficulty in accurately distinguishing between attacks with similar characteristics or high concealment, such as Analysis, Backdoor, and DoS in the NF-UNSW-NB15-v2 dataset.

2.3. Class Imbalances

When a class is underrepresented in a dataset, it leads to an imbalance that poses challenges in detecting the minority class, ultimately reducing the performance of intrusion detection systems [58]. In ref. [21], the IoT23 dataset is introduced, highlighting significant class imbalances due to the disparity between malicious and benign behaviors of IoT-connected devices. This work addresses the class imbalance challenge by applying preprocessing techniques like the SMOTE method to enhance the detection of minority class instances in the proposed IDSs. In ref. [18], the ToN-IoT dataset, derived from a large-scale heterogeneous IoT network, was utilized to address class imbalance in both binary and multi-class classification tasks. The study applied SMOTE for class balancing, demonstrating that XGBoost outperformed other ML methods. In ref. [30], ADASYN was employed to address data imbalance by expanding minority class samples, creating a more balanced NSL-KDD dataset. The study focused on mitigating class imbalance to improve model performance and utilized a modified stacked auto encoder for dimensionality reduction to enhance information fusion. In ref. [55], to address the class imbalance issue in the CSE-CIC-IDS2018 dataset, the SMOTE algorithm was applied, along with an improved loss function. These techniques helped balance the dataset, ensuring better model performance, with a LSTM network enhanced by an AM for improved detection. In ref. [57], to tackle the class imbalance issue, a CGAN was utilized to augment minority attack data for multi-class intrusion detection. This approach helped balance the dataset and improve model performance by addressing the underrepresentation of certain attack classes. In ref. [59], a data resampling technique combining the ADASYN and Tomek Links algorithms was proposed to address the class imbalance problem. This approach was applied alongside various deep learning models to improve performance on the NSL-KDD dataset. In ref. [60], a resampling method based on self-paced ensemble and auxiliary classifier generative adversarial networks (SPE-ACGAN) was proposed to address class imbalance. This method mitigated the imbalance by oversampling minority class samples using ACGAN and undersampling majority class samples through SPE. In ref. [61], class imbalance was addressed by using ADASYN to oversample minority-class samples. This technique enhanced the model's ability to detect underrepresented classes, ensuring better performance in classification tasks. In ref. [62], a hybrid approach combining SMOTE and Tomek Links was proposed to address class imbalance in the CICDDoS2019 and Edge-IIoT datasets, effectively mitigating the unbalanced distribution of classes. In ref. [63], the Jaya optimization technique was combined with the SMOTE-ENN method to address class imbalance in the UNSW-NB15 and NSL-KDD datasets, providing an effective solution for improving intrusion detection. In ref. [64], SMOTE and NearMiss-1 were utilized to address class imbalance. The model's performance in multi-class classification was evaluated on the UNSW-NB15 dataset and further validated for robustness using the NSL-KDD dataset. In ref. [65], the imbalance in training data leading to low detection rates of minority attacks was addressed by enriching minority samples using an improved generative adversarial network (IGAN). In ref. [66], a novel model-based generative adversarial network called TDCGAN was proposed to tackle class imbalance in datasets. This approach focused on improving the detection rate of minority class instances while ensuring efficiency in handling imbalanced data. In ref. [67], SMOTE was employed to address class imbalance by oversampling the minority class, thereby reducing the impact of data imbalance on classification performance. In ref. [68], a solution to class imbalance was proposed using the ADASYN algorithm for oversampling the minority class, and class weights for undersampling the majority class, ensuring a more balanced dataset for training a multi-stage CNN1D deep learning model. In ref. [69], the proposed system was compared with other techniques for addressing class imbalance, such as random over-sampling (ROS), SMOTE, and ADASYN. Benchmarking on the NSL-KDD and BoT-IoT datasets showed that the proposed system performed effectively in detecting minority class instances in binary classification tasks. In ref. [70], SMOTE was applied to address the class imbalance in the UNSW-NB15 dataset, enhancing the classification performance by balancing the distribution of classes. In ref. [71], an ensemble method was

used to tackle class imbalance across multiple datasets, including CICIDS2017, KDD99, and UNSW-NB15, improving the classification performance by effectively handling the under-representation of minority classes. In ref. [72], the performance of DT and RF was enhanced by applying CatBoost alongside random oversampling and undersampling techniques on the CIC-IDS-2018 dataset. In ref. [73], a novel feature selection algorithm, improved non-dominated sorting genetic algorithm III (I-NSGA-III), was proposed to address the imbalance issue, resulting in a better detection rate, though it did not lead to higher accuracy. In ref. [74], random oversampling of the minority classes and random undersampling of the majority class were applied to improve intrusion detection performance. However, random oversampling is known to induce overfitting [75], and only accuracy was reported in this study. A study in ref. [76] evaluated two tree-based classifiers and one deep learning-based classifier under various sampling rates, showing that sampling techniques improve the detection of both majority and minority classes. In ref. [77], Zhang et al. proposed a combination of SMOTE with ENN (SMOTE-ENN) and a DNN for the NSL-KDD dataset. In ref. [78], a cost-sensitive deep learning model combined with ensemble techniques was used to tackle class imbalance in intrusion detection. The model enhanced detection of both majority and minority attacks, but required high computational resources and time. In ref. [79], to address class imbalance, the study utilized SMOTE for oversampling the minority class and Tomek Links for undersampling the majority class. This combination of techniques helped balance the dataset before applying an LSTM model for classification, improving the model's ability to detect both majority and minority classes effectively.

2.4. Challenges

State-of-the-art IDSs that utilize deep learning models encounter several significant obstacles. A primary concern is the challenge of achieving high accuracy, which is frequently impeded by class imbalance within benchmark datasets. In these datasets, normal traffic typically outnumbers attack traffic, complicating the detection of rare yet critical attack types. This imbalance results in higher false alarm rates and diminishes overall detection effectiveness. Moreover, while deep learning has the potential to enhance detection performance, it also introduces considerable computational complexity and resource demands. This raises important concerns regarding scalability and efficiency, especially in large-scale, real-time operational environments. Another notable challenge is the generalizability of these models; they often struggle to adapt to varying network conditions or to detect new attack types that were not present in the training data, thereby limiting their robustness in practical, real-world applications. Additionally, many existing studies tend to emphasize theoretical and experimental aspects of deep learning, often overlooking essential practical deployment issues such as data privacy, system latency, and the integration of these systems with existing security infrastructures. Lastly, a narrow focus on accuracy can obscure other critical performance metrics, including precision, recall, F1-score, and the impacts of false positives and negatives. To address these multifaceted challenges, a comprehensive approach is required, one that carefully balances data handling, scalability, adaptability, and practical implementation.

Our proposed Transformer-CNN model addresses several key limitations of contemporary intrusion detection systems, offering superior performance in terms of accuracy and other critical metrics compared to traditional approaches. By incorporating advanced techniques such as ADASYN, SMOTE, ENN, and class weights, the model effectively mitigates class imbalance, significantly improving its ability to detect rare attack types. The transformer's contextual feature extraction capability enables the system to analyze complex relationships and patterns within the data with exceptional efficacy. Simultaneously, the CNN processes these extracted features to accurately classify specific attack types. Designed for scalability and efficiency, the Transformer-CNN model excels in handling large-scale datasets, optimizing computational resources, and ensuring real-time processing capabilities. Extensive testing on the NF-UNSW-NB15-v2 and CICIDS2017 datasets demonstrates the model's robustness, validating its effectiveness across diverse network

environments and attack scenarios. The model also addresses practical deployment challenges by minimizing false positives and false negatives, ensuring dependable performance in real-world applications. Moreover, the evaluation framework for the Transformer-CNN model goes beyond accuracy, incorporating a wide range of metrics to provide a thorough performance assessment and address potential limitations in detection reliability and practical implementation.

3. Proposed Approach

The Transformer-CNN model embodies a cutting-edge deep learning architecture that fuses the strengths of Transformer and CNN to achieve exceptional performance in both binary and multi-class classification tasks. This innovative framework proficiently addresses critical challenges faced by IDS, particularly in enhancing classification accuracy and mitigating class imbalances, with a primary emphasis on the NF-UNSW-NB15-v2 and CICIDS2017 datasets. In this section, we outline the detailed steps involved in the model, including comprehensive preprocessing procedures applied to the NF-UNSW-NB15-v2 dataset, followed by an evaluation of its performance on both the NF-UNSW-NB15-v2 and CICIDS2017 datasets. To tackle the issue of class imbalance, the model incorporates a suite of advanced data preprocessing techniques. It employs ADASYN and SMOTE to effectively oversample minority classes, thereby bolstering the model's capacity to learn from underrepresented yet crucial instances. Additionally, the model utilizes ENN for strategic undersampling, while also applying class weights to recalibrate the importance of each class during the training phase. This dual strategy not only ensures that challenging cases receive adequate focus but also preserves a balanced class distribution throughout the training process. The transformer component of the model is dedicated to contextual feature extraction, empowering the system to adeptly analyze relationships and patterns within the data. Meanwhile, the CNN efficiently processes the extracted features to accurately classify specific attack types. This synergistic architecture significantly reduces the incidence of false positives and false negatives, thereby enhancing the model's ability to detect both known threats and previously unseen (zero-day) attacks. The model's outstanding performance is underscored by its remarkable results on the NF-UNSW-NB15-v2 dataset, where it achieved an impressive 99.71% accuracy in binary classification and 99.02% accuracy in multi-class classification, as well as on the CICIDS2017 dataset, achieving 99.93% accuracy in binary classification and 99.13% accuracy in multi-class classification. Figure 1 illustrates the model architecture and its application to various classification tasks using the NF-UNSW-NB15-v2 dataset, providing a clear visual representation of its capabilities.

3.1. Description of Dataset

The UNSW-NB15 dataset [19], released in 2015 by the Cyber Range Lab of the Australian Centre for Cyber Security (ACCS), is a widely recognized and utilized NIDS dataset. It generates benign network activities and premeditated attack scenarios, comprising a total of 2,540,044 network samples, with 2,218,761 (87.35%) benign and 321,283 (12.65%) attack samples. This dataset includes 49 features, including twelve derived using SQL algorithms. Recently, the NF-UNSW-NB15-v2 dataset was generated and released in 2021, based on the original UNSW-NB15 dataset. This NetFlow dataset includes 43 NetFlow-based features extracted from the pcap files of the UNSW-NB15 dataset using the nprobe feature extraction tool, with the data flows labeled appropriately. The NF-UNSW-NB15-v2 dataset contains a total of 2,390,275 flows, of which 95,053 (3.98%) are attack samples and 2,295,222 (96.02%) are benign [10]. Additionally, this dataset is classified into ten classes, comprising nine for different types of attacks and one for benign traffic. Table 3 provides a detailed overview of the different attack categories, including comprehensive descriptions of each, along with the distribution of samples across various classes within the datasets. Figure 1 depicts the architecture developed for binary and multi-class classification using the NF-UNSW-NB15-v2 dataset.

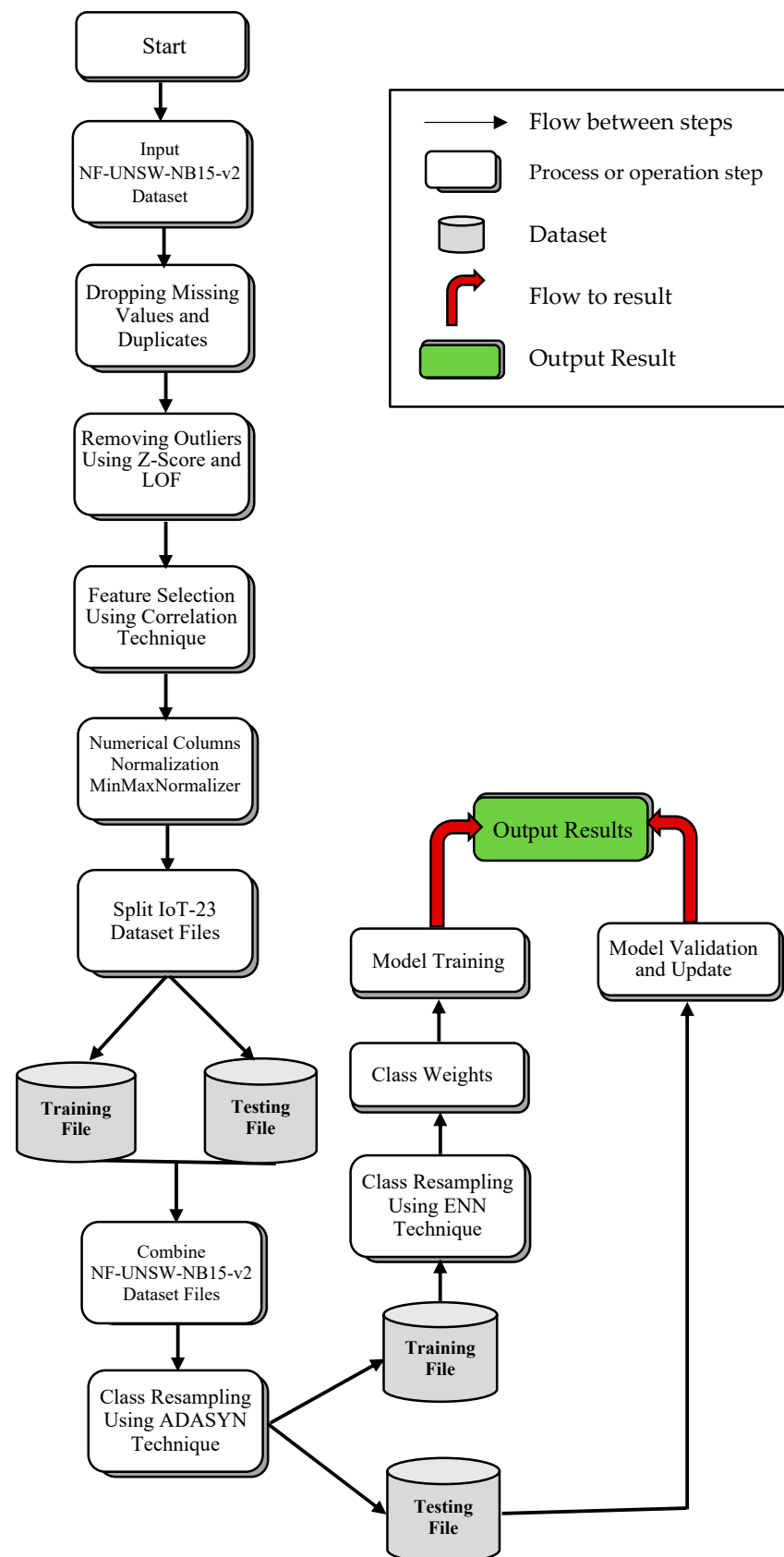


Figure 1. Architectural design for binary classification and multi-class classification using NF-UNSW-NB15-v2 dataset.

Table 3. Types of attacks in the NF-UNSW-NB15-v2 dataset.

Type of Attack	Samples Counts	Description
Benign	99,000	Normal, non-malicious flows.
Fuzzers	20,645	An attack in which the attacker transmits significant volumes of random data, resulting in system crashes while also seeking to identify security vulnerabilities within the system.
Analysis	770	A category that encompasses various threats aimed at web applications via ports, emails, and scripts.
Backdoor	833	A method designed to circumvent security measures by responding to specifically crafted client applications.
DoS	4172	Denial of service refers to an attempt to overwhelm a computer system's resources, aiming to impede access to or availability of its data.
Exploits	29,905	Sequences of commands that manipulate the behavior of a host by exploiting a known vulnerability.
Generic	5992	A technique that targets cryptographic systems, resulting in a collision with each block cipher.
Reconnaissance	11,171	A method used to collect information about a network host, also referred to as probing.
Shellcode	1427	Malware that infiltrates code to take control of a victim's host.
Worms	164	Attacks that self-replicate and propagate to other computers.

3.2. Data Preprocessing

Data preprocessing is a vital phase in both data analysis and machine learning workflows, where raw data are transformed into a refined, structured format, ready for effective analysis. This process encompasses a range of tasks, including handling missing values, removing duplicates, eliminating outliers or irrelevant data, selecting meaningful features, and applying normalization or standardization to numerical features. Additionally, class resampling techniques are employed to address imbalanced data. Proper data preprocessing significantly enhances data quality, minimizes noise, and optimizes the performance of machine learning models by enabling them to learn efficiently from the processed dataset. The specific steps and techniques required for preprocessing vary depending on the dataset. The NF-UNSW-NB15-v2 dataset, despite its comprehensiveness, contains missing or NaN values that must be addressed as part of the initial preprocessing step by removing them. Following this, any duplicate entries are eliminated. Outliers are then identified and removed using both the Z-Score and LOF methods. Next, a feature selection technique, based on correlation, is applied to reduce dimensionality. Numerical features are normalized using the MinMaxScaler 1.2.2 to achieve consistent scaling across the dataset. Once these preprocessing steps are completed, the dataset is split into training and testing subsets. Subsequently, the training and testing sets are recombined, and the ADASYN technique is applied to the combined dataset. This approach enhances learning from both the training and testing data. After ADASYN generates synthetic samples based on the combined training and testing datasets, the synthetic samples are added to the dataset. The dataset is then split again, where the new training set consists of the original training data along with the ADASYN-generated samples, while the test set remains unchanged. This strategy allows the model to learn more effectively from the augmented data and improves its overall performance. This strategy helps mitigate class imbalance, ultimately improving model accuracy and performance. Additionally, the ENN method is employed to under-sample the training data, and class weights are adjusted during model training to further balance the dataset. Our comprehensive data preparation process, encompassing outlier removal, feature selection, normalization, resampling, and model development, is depicted

in Figure 1, which illustrates the full workflow for both binary and multi-class classification tasks on the NF-UNSW-NB15-v2 dataset.

3.2.1. Removing Outliers Using Z-Score and Local Outlier Factor (LOF)

Z-score was applied to detect and filter out extreme outliers in the dataset. Specifically, the `zscore` function from the `scipy.stats` module calculated the z-scores for all features in the DataFrame. Z-scores represent how far a data point is from the mean in terms of standard deviations. A threshold of 6 was set, meaning any data point with a z-score greater than 6 in any feature was considered an outlier and removed. This process was applied for both binary and multi-class classification to ensure that the dataset remained clean and free of extreme outliers.

Following the z-score, the LOF method was implemented to further detect and eliminate outliers. LOF identifies data points with significantly lower density compared to their neighbors, making it particularly effective for datasets with varying density distributions. The LOF was configured with `n_neighbors` set to 20 and `contamination` set to 0.1, indicating that 10% of the data were expected to be outliers. After fitting the LOF model, samples were classified as either outliers (labeled `-1`) or inliers (labeled `1`). Only the inlier samples were retained, resulting in a cleaner dataset for subsequent analysis. This dual approach of outlier removal enhanced the performance and reliability of the classification models for both binary and multi-class tasks.

(i) Binary Classification

The NF-UNSW-NB15-v2 dataset underwent z-score to remove extreme outliers, ensuring higher data quality for binary classification. Outliers were identified and removed based on how far data points deviated from the mean, improving the dataset's reliability. As shown in Table 4, the Benign class was reduced from 96,432 to 93,653 samples after outlier removal. Similarly, other attack categories, such as Exploits and Fuzzers, decreased from 18,804 to 17,576 and 12,999 to 11,695 samples, respectively. Smaller classes, including Shellcode, Backdoor, and Worms, also experienced slight reductions. This filtering process ensured that both majority and minority classes remained balanced while minimizing the impact of noise and outliers on the classification model's performance.

Table 4. Sample distribution of NF-UNSW-NB15-v2 dataset in binary classification using z-score.

Class Type	Number of Samples Before Z-Score	Number of Samples After Z-Score
Benign	96,432	93,653
Exploits	18,804	17,576
Fuzzers	12,999	11,695
Reconnaissance	7121	6883
Generic	3810	3211
DoS	2677	2180
Shellcode	900	886
Backdoor	547	322
Analysis	490	324
Worms	104	89

The sample distribution of the NF-UNSW-NB15-v2 dataset in binary classification is presented in Table 5, highlighting the impact of the LOF method on the dataset. Before applying LOF, the class Benign consisted of 93,653 samples, which decreased to 85,680 samples after outlier removal. Other classes also experienced significant reductions; for instance, Exploits reduced from 17,576 to 14,969, while Fuzzers dropped from 11,695 to 10,116. Smaller classes, such as Shellcode, decreased from 886 to 605, and Backdoor saw

a reduction from 322 to 233 samples. The Worms class remained relatively stable, with a slight decline from 89 to 87 samples. This filtering process ensured a cleaner dataset, facilitating more accurate classification while reducing the influence of outliers on model performance. The adjustments made through LOF provide a balanced representation of both majority and minority classes, enhancing the reliability of the classification tasks.

Table 5. Sample distribution of NF-UNSW-NB15-v2 dataset in binary classification using LOF.

Class Type	Number of Samples Before LOF	Number of Samples After LOF
Benign	93,653	85,680
Exploits	17,576	14,969
Fuzzers	11,695	10,116
Reconnaissance	6883	6759
Generic	3211	2668
DoS	2180	1716
Shellcode	886	605
Backdoor	322	233
Analysis	324	304
Worms	89	87

(ii) Multi-Class Classification

The sample distribution of the NF-UNSW-NB15-v2 dataset in multi-class classification is summarized in Table 6, which illustrates the effects of z-score on the dataset. Prior to the application of z-score, the Benign class contained 96,432 samples, which decreased to 93,530 samples following outlier removal. Similar reductions were observed in other classes; for example, Exploits dropped from 18,804 to 17,492, and Fuzzers declined from 12,999 to 11,730. The Reconnaissance class saw a reduction from 7121 to 6881, while Generic samples decreased from 3810 to 3234. The DoS class also experienced a notable decrease, from 2677 to 2195 samples. In contrast, some smaller classes exhibited minimal changes, such as Shellcode, which went from 900 to 886, and Analysis, which decreased slightly from 490 to 330. The Worms class experienced a reduction from 104 to 92 samples. These modifications, achieved through z-score, helped maintain the integrity of the dataset by ensuring that extreme outliers were removed, thereby enhancing the robustness and accuracy of the subsequent classification models.

Table 6. Sample distribution of NF-UNSW-NB15-v2 dataset in multi-class classification using z-score.

Class Type	Number of Samples Before Z-Score	Number of Samples After Z-Score
Benign	96,432	93,530
Exploits	18,804	17,492
Fuzzers	12,999	11,730
Reconnaissance	7121	6881
Generic	3810	3234
DoS	2677	2195
Shellcode	900	886
Backdoor	547	327
Analysis	490	330
Worms	104	92

The sample distribution of the NF-UNSW-NB15-v2 dataset in multi-class classification is detailed in Table 7, highlighting the impact of the LOF method on the dataset. Initially, the Benign class comprised 93,530 samples, which was reduced to 85,510 samples after outlier removal. Similarly, the Exploits class decreased from 17,492 to 14,933, while the Fuzzers class saw a decline from 11,730 to 10,131. The Reconnaissance class experienced a minor reduction from 6881 to 6774, and the Generic class dropped from 3234 to 2688 samples. The DoS class also faced a significant decrease, with numbers falling from 2195 to 1730. In the smaller classes, Shellcode decreased from 886 to 614, and Backdoor went from 327 to 243 samples. The Analysis class experienced a slight reduction from 330 to 316, while the Worms class saw minimal change, dropping from 92 to 88 samples. These adjustments, facilitated by the LOF method, contributed to a more balanced dataset by removing outliers, thereby improving the reliability and effectiveness of classification tasks.

Table 7. Sample distribution of NF-UNSW-NB15-v2 dataset in multi-class classification using LOF.

Class Type	Number of Samples Before LOF	Number of Samples After LOF
Benign	93,530	85,510
Exploits	17,492	14,933
Fuzzers	11,730	10,131
Reconnaissance	6881	6774
Generic	3234	2688
DoS	2195	1730
Shellcode	886	614
Backdoor	327	243
Analysis	330	316
Worms	92	88

3.2.2. Feature Selection Using Correlation Technique

Feature selection was conducted based on the correlation of features with the target variable, ‘Target’, in the dataset, applicable to both binary and multi-class classification tasks. Initially, the features and target variable were separated, and a correlation matrix was computed by concatenating the features and the target. The absolute correlation values were extracted and sorted to identify the strength of relationships between each feature and the target variable. A correlation threshold of 0.01 was set to filter out features with weak correlations, allowing only those with significant relationships to be retained. The target variable, ‘Target’, was subsequently removed from the list of selected features. To address potential multicollinearity, the code calculated the absolute correlation matrix for the selected features and examined the upper triangle to avoid redundancy. Any features exhibiting a correlation greater than 0.9 were flagged for removal. The final dataset comprised the selected features that demonstrated meaningful correlations with the target variable while mitigating issues related to multicollinearity. This process ensured a robust input for subsequent analyses or model training in both binary and multi-class classification contexts.

(i) Binary Classification

The selected features from the NF-UNSW-NB15-v2 dataset for binary classification, identified using a correlation technique, are detailed in Table 8. These features were carefully chosen based on their significant correlation with the target variable, ensuring their relevance for the classification task. The selected features include metrics such as MAX_TTL and MIN_IP_PKT_LEN, which provide insights into packet attributes, as well as network behaviors like SRC_TO_DST_AVG_THROUGHPUT and DST_TO_SRC_AVG_THROUGHPUT. Additionally, protocol-specific characteristics are captured through features like PROTO-

COL and L4_DST_PORT, while metrics like FLOW_DURATION_MILLISECONDS and NUM_PKTS_128_TO_256_BYTES offer a deeper understanding of traffic patterns. Overall, this selection aims to enhance the model's ability to accurately differentiate between benign and malicious activities in network traffic.

Table 8. Selected features of NF-UNSW-NB15-v2 dataset in binary classification using correlation technique.

Selected Features	Selected Features	Selected Features	Selected Features
MAX_TTL	TCP_WIN_MAX_IN	NUM_PKTS_128_TO_256_BYTES	SRC_TO_DST_AVG_THROUGHPUT
MIN_IP_PKT_LEN	OUT_PKTS	ICMP_TYPE	L7_PROTO
SERVER_TCP_FLAGS	DNS_QUERY_TYPE	FTP_COMMAND_RET_CODE	DST_TO_SRC_SECOND_BYTES
NUM_PKTS_UP_TO_128_BYTES	FLOW_DURATION_MILLISECONDS	NUM_PKTS_256_TO_512_BYTES	DNS_TTL_ANSWER
L4_DST_PORT	NUM_PKTS_512_TO_1024_BYTES	DNS_QUERY_ID	
MAX_IP_PKT_LEN	PROTOCOL	SHORTEST_FLOW_PKT	
DST_TO_SRC_AVG_THROUGHPUT	RETRANSMITTED_IN_PKTS	RETRANSMITTED_IN_BYTES	

(ii) Multi-Class Classification

The selected features from the NF-UNSW-NB15-v2 dataset for multi-class classification, identified using a correlation technique, are outlined in Table 9. These features were chosen for their significant correlation with the target variable, enhancing the model's ability to distinguish among various classes effectively. Key features include MIN_TTL and MIN_IP_PKT_LEN, which provide important information about packet characteristics, alongside metrics like DST_TO_SRC_AVG_THROUGHPUT and SRC_TO_DST_AVG_THROUGHPUT, which capture network traffic flow dynamics. Additionally, protocol-related attributes are represented through features such as PROTOCOL and L4_SRC_PORT, while metrics like DURATION_IN and LONGEST_FLOW_PKT offer insights into session behavior. This selection aims to improve the classification performance by retaining features that exhibit strong correlations with the output variable across different classes.

Table 9. Selected features of NF-UNSW-NB15-v2 dataset in multi-class classification using correlation technique.

Selected Features	Selected Features	Selected Features	Selected Features
MIN_TTL	PROTOCOL	NUM_PKTS_128_TO_256_BYTES	DNS_QUERY_ID
MIN_IP_PKT_LEN	TCP_WIN_MAX_IN	NUM_PKTS_512_TO_1024_BYTES	L4_SRC_PORT
SERVER_TCP_FLAGS	DST_TO_SRC_AVG_THROUGHPUT	TCP_WIN_MAX_OUT	L7_PROTO
NUM_PKTS_UP_TO_128_BYTES	OUT_PKTS	NUM_PKTS_256_TO_512_BYTES	DST_TO_SRC_SECOND_BYTES
LONGEST_FLOW_PKT	FTP_COMMAND_RET_CODE	SRC_TO_DST_AVG_THROUGHPUT	DNS_TTL_ANSWER
L4_DST_PORT	SHORTEST_FLOW_PKT	DURATION_IN	RETRANSMITTED_IN_BYTES
DNS_QUERY_TYPE	RETRANSMITTED_IN_PKTS	ICMP_TYPE	

3.2.3. Normalization

Data scaling, a crucial preprocessing step in machine and deep learning, involves adjusting numerical values to a specific range, thereby enhancing the efficiency and effectiveness of model. This standardization process is applied across all columns, ensuring consistent data representation. Among various normalization techniques, the MinMaxScaler, a widely used tool in the scikit-learn library, stands out as the most effective for our study. The normalization formula, as depicted in Equation (1) [80], calculates each value by subtracting the minimum value in the column and dividing by the range (the difference between the maximum and minimum values). In this context, X represents the original values, $\min(X)$ is the minimum value in the column, and $\max(X)$ is the maximum value in the column. After evaluating multiple normalization methods, MinMaxScaler was chosen for its superior performance. This normalization technique was applied to the

selected features in the dataset, ensuring consistent scaling for both binary and multi-class classification tasks.

$$X(\text{scaled}) = \frac{X - \min(x)}{\max(x) - \min(x)} \quad (1)$$

The testing file was employed for evaluating the NF-UNSW-NB15-v2 dataset, while the complete training file was used for training in the initial approach.

3.2.4. Train-Test Dataset Split

The division of the dataset into training and testing subsets plays a crucial role in achieving rigorous evaluation and ensuring model generalization. The training subset allows the model to learn intricate patterns and relationships within the data, while the testing subset, isolated from the training phase, serves as an unbiased benchmark for assessing performance on unseen instances. This approach minimizes overfitting and provides meaningful insights into the model's adaptability and effectiveness in binary and multi-class classification scenarios.

(i) Binary Classification

The NF-UNSW-NB15-v2 dataset used for binary classification was divided into training and testing sets, as detailed in Table 10. The Normal class included 85,680 samples, of which 81,320 were utilized for training and 4360 for testing. Similarly, the Attack class consisted of 37,457 samples, with 35,660 used for training and 1797 for testing. This distribution ensured that both classes were adequately represented in both the training and testing phases, facilitating effective model evaluation and generalization.

Table 10. Sample distribution of the NF-UNSW-NB15-v2 dataset in binary classification.

Class Type	Train	Test
Normal	81,320	4360
Attack	35,660	1797

(ii) Multi-Class Classification

The NF-UNSW-NB15-v2 dataset, used for multi-class classification, includes ten classes, comprising the Benign class and nine attack types, such as Exploits, Fuzzers, Reconnaissance, Generic, DoS, Shellcode, Backdoor, Analysis, and Worms. Table 11 outlines the sample distribution across these classes, highlighting the class-wise split between training and testing sets. The largest class, Benign, comprises 81,200 training samples and 4310 testing samples. Exploits and Fuzzers follow with 14,190 and 9653 training samples, respectively. Smaller attack categories, such as Worms and Analysis, include 83 and 306 training samples, respectively, with a limited number of testing samples. This distribution ensures representation of both frequent and rare attack types, enabling comprehensive evaluation of the model's ability to detect diverse attacks effectively.

Table 11. Sample distribution of the NF-UNSW-NB15-v2 dataset in multi-class classification.

Class Type	Train	Test
Benign	81,200	4310
Exploits	14,190	743
Fuzzers	9653	478
Reconnaissance	6428	346
Generic	2547	141
DoS	1652	78

Table 11. Cont.

Class Type	Train	Test
Shellcode	589	25
Backdoor	227	16
Analysis	306	10
Worms	83	5

3.2.5. Class Balancing

Class imbalance is a significant challenge in the NF-UNSW-NB15-v2 dataset, potentially reducing the performance of machine learning models. To mitigate this issue, a robust class balancing strategy was implemented, leveraging a combination of oversampling and undersampling techniques, well-established methods for addressing class imbalance [81]. Following this, the training and testing sets were merged, and ADASYN was employed on the combined dataset. This technique enhances learning by allowing the model to benefit from both training and testing data. After generating synthetic samples, the dataset was split again, with the new training set comprising the original training data plus the ADASYN-generated samples, while the test set remained unchanged. ADASYN was applied to oversample both binary and multi-class classification tasks by generating synthetic samples to improve the representation of minority classes. This approach improves model performance by augmenting the dataset with additional samples. Furthermore, ENN was applied to the training dataset for undersampling, removing noisy or redundant instances from the majority class. Class weights were adjusted during model training to balance the influence of each class, ensuring that the models did not become biased toward the majority class. By employing this combination of ADASYN for oversampling, ENN for undersampling, and class weight adjustments, the model's ability to accurately detect and classify minority classes was significantly improved, leading to enhanced performance and reliability. However, despite achieving high accuracy, models can still suffer from the accuracy paradox, where minority class predictions are weak [82]. To counter this, an improved strategy, inspired by [79], was introduced, integrating ADASYN for oversampling, ENN for undersampling, and class weights to provide a more effective solution to class imbalance. This approach ensures more balanced performance across all classes, ultimately improving the model's effectiveness in handling imbalanced datasets.

1. ADASYN

ADASYN is an advanced technique designed to address the challenges of class imbalance in datasets. By generating synthetic samples for the minority class, ADASYN focuses on regions of the feature space where instances of the minority class are underrepresented. This method enhances the representation of the minority class while preserving the distribution of the majority class, leading to improved model performance. In our approach, an enhanced cascaded ADASYN technique was applied twice for binary classification tasks and nine times for multi-class classification tasks. This approach ensured that the dataset remained balanced at each stage, progressively improving model training by handling class imbalance more effectively in both binary and multi-class scenarios [83].

Let X_i represent the minority class samples, and $N(X_i, k)$ denote the k -nearest neighbors of X_i . The number of synthetic samples n_i to generate for each minority instance is defined as presented in Equation (2) [84].

$$n_i = \frac{N_{Maj} - N_{Min}}{N_{Min}} \cdot \left(1 - \frac{N_i}{k}\right) \quad (2)$$

In this context, N_{Maj} and N_{Min} represent the sample counts of the majority and minority classes, respectively, highlighting the imbalance between them. The term N_i denotes the number of minority class samples that fall within the radius defined by the k -

nearest neighbors, which helps in identifying minority instances near decision boundaries, where synthetic samples are often generated to improve model performance.

For each minority instance X_i , synthetic samples are generated using the following equation, as presented in Equation (3) [84].

$$X_{syn} = X_i + \gamma \cdot (X_j - X_i) \quad (3)$$

where X_{syn} denotes the synthetic sample created to address class imbalance, X_j represents a randomly selected neighbor from the k-nearest neighbors of X_i , the minority sample, and the term γ is a random number between 0 and 1, ensuring that the synthetic sample is generated along the line segment between X_i and X_j .

(i) Binary Classification

The sample distribution in each class before and after applying the ADASYN resampling technique for binary classification on the NF-UNSW-NB15-v2 dataset is presented in Table 12. Initially, the dataset comprised 85,680 samples for the ‘Normal’ class and 37,457 samples for the ‘Attack’ class. After applying ADASYN, the number of samples for the ‘Attack’ class increased to 85,777, while the count for the ‘Normal’ class remained unchanged at 85,680. This adjustment underscores the effectiveness of ADASYN in addressing class imbalance by generating synthetic samples for the minority class, ultimately enhancing the model’s ability to learn from a more balanced dataset.

Table 12. Sample distribution in each class before/after resampling using ADASYN for binary classification on NF-UNSW-NB15-v2 dataset.

Class Type	Number of Samples Before Resampling (ADASYN)	Number of Samples After Resampling (ADASYN)
Normal	85,680	85,680
Attack	37,457	85,777

(ii) Multi-Class Classification

The sample distribution in each class before and after applying the ADASYN resampling technique for multi-class classification on the NF-UNSW-NB15-v2 dataset is detailed in Table 13. Initially, the ‘Benign’ class consisted of 85,510 samples, while other classes had varying sample sizes. Following the application of ADASYN, the number of samples significantly increased for the minority classes. For instance, the ‘Exploits’ class rose from 14,933 to 86,104 samples, and the ‘Reconnaissance’ class grew from 6774 to 85,734 samples. Meanwhile, the ‘Benign’ class remained stable at 85,510 samples. This resampling process effectively enhances the representation of underrepresented classes, thus improving the dataset’s balance and aiding in the development of more robust classification models.

Table 13. Sample distribution in each class before/after resampling using ADASYN for multiclass classification on NF-UNSW-NB15-v2 dataset.

Class Type	Number of Samples Before Resampling (ADASYN)	Number of Samples After Resampling (ADASYN)
Benign	85,510	85,510
Exploits	14,933	86,104
Fuzzers	10,131	85,737
Reconnaissance	6774	85,734
Generic	2714	85,504
DoS	2688	85,642

Table 13. Cont.

Class Type	Number of Samples Before Resampling (ADASYN)	Number of Samples After Resampling (ADASYN)
Shellcode	1730	85,587
Backdoor	243	85,462
Analysis	316	85,572
Worms	88	85,507

2. ENN

ENN is a data preprocessing technique aimed at refining training datasets by removing noisy instances and improving class boundaries. This method examines the nearest neighbors of each instance and eliminates those that are misclassified, thereby enhancing the overall quality of the training data. ENN effectively reduces class overlap and helps maintain a balanced representation of the classes, making it particularly useful in both binary and multi-class classification scenarios. In this study, ENN was applied once for binary classification and three times for multi-class classification. By applying ENN to the training data, models can achieve better generalization and improved performance on unseen data.

For each instance X_i in the dataset, determine the k -nearest neighbors. The set of neighbors is defined as using Equation (4) [85].

$$N(X_i) = \{X_{j1}, X_{j2}, \dots, X_{jk}\} \quad (4)$$

where X_{jk} are the nearest neighbors of X_i in terms of a distance metric (e.g., Euclidean distance).

To calculate the majority class among the nearest neighbors, one can use the formula represented in Equation (5) [85]. This involves determining the class labels of the nearest neighbors and identifying which class occurs most frequently. By applying this method, one can ensure that the predicted class for an instance is based on the most common class among its neighbors, thereby enhancing the classification accuracy.

$$C(X_i) = \operatorname{argmax}_c \left(\sum_{j=1}^k \prod (y_j = c) \right) \quad (5)$$

Here, $C(X_i)$ denotes the predicted class for instance X_i , with y_j representing the class label of its j -th neighbor. The indicator function $\prod$ outputs 1 if the condition holds true, otherwise returning 0.

An instance X_i is removed if its predicted class $C(X_i)$ does not match its actual class y_j , using the formula in Equation (6) [85].

$$\text{If } C(X_i) \neq y_j, \text{ then remove } X_i \quad (6)$$

(i) Binary Classification

The sample distribution in each training class before and after applying ENN for resampling in the binary classification of the NF-UNSW-NB15-v2 dataset is summarized in Table 14. Initially, the number of samples for the 'Normal' class stood at 81,320, which remained unchanged after the resampling process. Conversely, the 'Attack' class had 83,980 samples before resampling, which slightly decreased to 83,754 following the application of ENN. This adjustment highlights ENN's role in refining the dataset by effectively managing class distribution while maintaining the integrity of the 'Normal' class samples.

Table 14. Sample distribution in each Train class before/after resampling using ENN for binary classification on NF-UNSW-NB15-v2 dataset.

Class Type	Number of Samples Before Resampling (ENN)	Number of Samples After Resampling (ENN)
Normal	81,320	81,320
Attack	83,980	83,754

(ii) Multi-Class Classification

The sample distribution in each training class before and after applying ENN for resampling in the multi-class classification of the NF-UNSW-NB15-v2 dataset is presented in Table 15. The ‘Benign’ class began with 81,200 samples, which slightly decreased to 80,713 following resampling. The ‘Exploits’ class experienced a more substantial reduction, dropping from 85,361 to 72,076 samples. Similar trends were observed in other classes, such as ‘Fuzzers’, which decreased from 85,259 to 80,288 samples, and ‘Reconnaissance’, which dropped from 85,388 to 74,094. In contrast, the ‘Shellcode’ class saw minimal change, with a slight decrease from 85,562 to 85,531 samples. These adjustments highlight ENN’s effectiveness in refining the dataset by eliminating noisy instances and improving class boundaries, ultimately contributing to a more balanced representation of each class.

Table 15. Sample distribution in each Train class before/after resampling using ENN for multi-class classification on NF-UNSW-NB15-v2 dataset.

Class Type	Number of Samples Before Resampling (ENN)	Number of Samples After Resampling (ENN)
Benign	81,200	80,713
Exploits	85,361	72,076
Fuzzers	85,259	80,288
Reconnaissance	85,388	74,094
Generic	85,363	77,164
DoS	85,564	75,066
Shellcode	85,562	85,531
Backdoor	85,446	61,014
Analysis	85,562	66,110
Worms	85,502	85,318

3. Class Weights

Class weights are a valuable technique used to address class imbalance in datasets by assigning different weights to each class during model training. This approach ensures that the model pays more attention to minority classes, thereby improving its ability to correctly classify instances from these groups. By applying class weights to the training data, both binary and multi-class classification tasks benefit from enhanced model performance and generalization. This method helps mitigate the risks associated with biased predictions, ensuring a more balanced representation of all classes throughout the learning process.

The class weights can be calculated using the following formula to address class imbalance within the dataset. This approach assigns different weights to each class, ensuring that the model pays more attention to minority classes during training. The formula for calculating class weights is provided in Equation (7) [86].

$$Weight_c = \frac{N}{k.n_c} \quad (7)$$

In this context, $Weight_c$ denotes the weight assigned to class c . The total number of instances in the dataset is represented by N , while k indicates the total number of classes. Additionally, n_c signifies the number of instances belonging to class c .

(i) Binary Classification

The weights assigned to each class in the training data for binary classification using class weights on the NF-UNSW-NB15-v2 dataset are presented in Table 16. The ‘Normal’ class is assigned a weight of 1.0150, while the ‘Attack’ class receives a weight of 0.9855. These weights reflect the importance of each class during model training, with the goal of addressing any imbalances in the dataset. By incorporating these class weights, the model can enhance its performance and improve the accuracy of its predictions, particularly for the minority class.

Table 16. Weight in each train class using class weights for binary classification on NF-UNSW-NB15-v2 dataset.

Class Type	Weight Using Class Weights
Normal	1.0150
Attack	0.9855

(ii) Multi-Class Classification

The weights assigned to each class in the training data for multi-class classification using class weights on the NF-UNSW-NB15-v2 dataset are shown in Table 17. The ‘Benign’ class has a weight of 0.9384, while ‘Exploits’ is assigned a weight of 1.0508. Other classes, such as ‘Fuzzers’ and ‘Reconnaissance,’ receive weights of 0.9433 and 1.0222, respectively. The ‘Generic’ class has a weight of 0.9815, and ‘DoS’ is given a weight of 1.0089. Notably, the ‘Backdoor’ class receives the highest weight at 1.2413, followed closely by ‘Analysis’ with a weight of 1.1456. These weights are designed to improve the model’s performance by addressing class imbalances, allowing for a more effective and accurate representation of each class during training.

Table 17. Weight in each Train class using class weights for multi-class classification on NF-UNSW-NB15-v2 dataset.

Class Type	Weight Using Class Weights
Benign	0.9384
Exploits	1.0508
Fuzzers	0.9433
Reconnaissance	1.0222
Generic	0.9815
DoS	1.0089
Shellcode	0.8855
Backdoor	1.2413
Analysis	1.1456
Worms	0.8877

3.3. Architectures of Models

In this study, a variety of model architectures were utilized, encompassing CNN, auto encoder, DNN, and Transformer-CNN. These models were selected due to their outstanding performance across multiple evaluation metrics [83,87,88].

3.3.1. Convolutional Neural Networks (CNN)

The given model architecture integrates CNN and a MLP for both binary and multi-class classification tasks. It begins with an input layer designed to accept sequential data structured as a one-dimensional array. The first CNN block applies a convolutional layer followed by batch normalization, ReLU activation, max pooling, and dropout to extract and regularize features effectively. This process is repeated in subsequent CNN blocks with varying kernel sizes to capture different patterns and features in the data. After the CNN blocks, the flattened output is fed into an MLP block that consists of a dense layer with L2 regularization, batch normalization, ReLU activation, and dropout to enhance the model's representation capabilities while mitigating overfitting. The outputs from the CNN and MLP components are then concatenated to form a comprehensive feature set. Finally, the output layer uses a sigmoid activation function for binary classification, or a softmax activation for multi-class classification, combined with binary cross-entropy or categorical cross-entropy as the loss function, respectively, to optimize model performance based on the specific classification task. The model is compiled using the Adam optimizer, which aids in efficient learning and convergence during training.

The convolution operation in the CNN layers plays a crucial role in feature extraction by applying filters to the input data. This operation involves sliding the convolution kernel over the input feature map, computing the dot product at each position to produce a feature map. The mathematical representation of the convolution operation can be defined as shown in Equation (8) [89].

$$Z_{i,j} = (X * K)_{i,j} = \sum_m \sum_n Z_{i+m,j+n} k_{m,n} \quad (8)$$

In this context, Z represents the output feature map, while X denotes the input feature map. The convolution kernel is indicated by K , which is utilized in the convolution operation to transform the input features into the output features.

The ReLU activation function, shown in Equation (9) [90], is a simple yet powerful non-linear transformation. It outputs zero for negative inputs, retains positive values, mitigates the vanishing gradient issue, and promotes sparse activations, enhancing both training efficiency and model performance.

$$\text{ReLU}(x) = \max(0, x) \quad (9)$$

The max pooling operation reduces the spatial dimensions of the input feature map while retaining the most important features. This operation selects the maximum value from a specified pooling window, effectively downsampling the input. The equation for the max pooling operation can be expressed as shown in Equation (10) [89].

$$P_{i,j} = \max(X_{i:i+p, j:j+q}) \quad (10)$$

In this context, P refers to the pooled output generated from the pooling operation, while p and q represent the dimensions of the pooling window used to aggregate the input features into the pooled output.

The dropout layer randomly sets a fraction p of input units to zero during training to prevent overfitting. This technique helps to improve the model's generalization by ensuring that it does not rely too heavily on any single input feature, as detailed in Equation (11) [91].

$$\text{Dropout}(x) = \begin{cases} x & \text{with probability } 1 - p \\ 0 & \text{with probability } p \end{cases} \quad (11)$$

For binary classification tasks, the output layer utilizes the sigmoid function, which outputs a probability score indicating the likelihood of an instance belonging to the positive class. This is mathematically expressed in Equation (12) [92].

$$\sigma(Z) = \frac{1}{1 + e^{-z}} \quad (12)$$

where Z is the output from the last dense layer.

The output layer employs the softmax function for multi-class classification, allowing the model to produce a probability distribution across multiple classes. This can be mathematically represented as shown in Equation (13) [92].

$$\text{Softmax}(Z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (13)$$

where Z_i is the output for class i , and Z_j represents the raw score for class j .

(i) Binary Classification

The architecture of the CNN model designed for binary classification is detailed in Table 18. The model architecture is shared across both the NF-UNSW-NB15-v2 and CICIDS2017 datasets, with the input block differing to accommodate the specific features of each dataset. For the NF-UNSW-NB15-v2 dataset, the input layer processes 25 distinct features, while the CICIDS2017 dataset input layer accommodates 69 features. This input layer serves as the foundation for subsequent computations. The CNN model for binary classification begins with the first hidden block, which includes a one-dimensional (1D) CNN layer with 256 filters, using the ReLU activation function to introduce non-linearity and enhance feature extraction. Following this, a 1D max pooling layer with a pool size of 2 is employed to downsample the data, preserving critical features. A dropout layer with a very low rate of 0.0000001 is incorporated to mitigate the risk of overfitting. The second hidden block replicates this structure, incorporating another 1D CNN layer with 256 filters and ReLU activation, followed by a 1D max pooling layer with a pool size of 4, and another dropout layer with the same low rate to maintain generalization. In the third hidden block, a dense layer with 1024 neurons is used, employing ReLU activation to facilitate complex feature interactions. This is followed by another dropout layer to further enhance robustness against overfitting. The final output block consists of a single neuron configured with a sigmoid activation function, which is critical for producing binary classification outputs for both datasets. This carefully structured architecture, as summarized in Table 18, is optimized to effectively process the unique characteristics of the NF-UNSW-NB15-v2 and CICIDS2017 datasets, ensuring reliable and accurate binary classification performance.

(ii) Multi-Class Classification

The CNN model designed for multi-class classification features a comprehensive architecture, as detailed in Table 19. This architecture is tailored to process the distinct features of both the NF-UNSW-NB15-v2 and CICIDS2017 datasets, with the input block configured specifically for each dataset. For the NF-UNSW-NB15-v2 dataset, the input layer processes 27 features, while the CICIDS2017 dataset input layer handles 35 features. These input layers provide the foundation for the model to effectively capture dataset-specific information relevant to the classification task. The model begins with the first hidden block, which includes a one-dimensional (1D) CNN layer with 256 filters, employing the ReLU activation function to enable effective feature extraction. This is followed by a 1D max pooling layer with a pool size of 2, which reduces dimensionality while retaining critical information. To minimize overfitting, a dropout layer with an extremely low rate of 0.0000001 is applied. The second hidden block mirrors this structure, featuring another 1D CNN layer with 256 filters and ReLU activation, followed by a 1D max pooling layer with a pool size of 4 and a dropout layer with the same low rate to maintain generalization. In the third hidden block, a dense layer with 1024 neurons is used, employing ReLU activation

to enhance the model's ability to learn complex feature relationships. This is followed by another dropout layer to further strengthen the model's capacity to generalize well to unseen data. The output block varies depending on the dataset. For the NF-UNSW-NB15-v2 dataset, the output layer consists of 10 neurons with a softmax activation function, allowing the model to output probabilities across 10 classes. For the CICIDS2017 dataset, the output layer comprises 15 neurons, also using a softmax activation function to accommodate its multi-class structure. This carefully designed architecture, as summarized in Table 19, is optimized to handle the unique characteristics of both datasets, ensuring effective learning and high-performance multi-class classification.

Table 18. CNN model layers for binary classification.

Dataset	Block	Layers	Layer Size	Activation
NF-UNSW-NB15-v2	Input block	Input layer	25	-
CICIDS2017	Input block	Input layer	69	-
Shared Structure	Hidden block 1	1D CNN layer	256	ReLU
		1D Max Pooling layer	2	-
		Dropout layer	0.0000001	-
	Hidden block 2	1D CNN layer	256	ReLU
		1D Max Pooling layer	4	-
		Dropout layer	0.0000001	-
	Hidden block 3	Dense layer	1024	ReLU
		Dropout layer	0.0000001	-
	Output block	Output layer	1	Sigmoid

Table 19. CNN model layers for multi-class classification.

Dataset	Block	Layers	Layer Size	Activation
NF-UNSW-NB15-v2	Input block	Input layer	27	-
CICIDS2017	Input block	Input layer	35	-
Shared Structure	Hidden block 1	1D CNN layer	256	ReLU
		1D Max Pooling layer	2	-
		Dropout layer	0.0000001	-
	Hidden block 2	1D CNN layer	256	ReLU
		1D Max Pooling layer	4	-
		Dropout layer	0.0000001	-
	Hidden block 3	Dense layer	1024	ReLU
		Dropout layer	0.0000001	-
NF-UNSW-NB15-v2	Output block	Output layer	10	Softmax
CICIDS2017	Output block	Output layer	15	Softmax

(iii) Hyperparameter Configuration for the CNN Model

The hyperparameters for the CNN model, as outlined in Table 20, are meticulously tuned for both binary and multi-class classification tasks. In both classifier configurations, a batch size of 128 is consistently utilized, ensuring efficient processing of data during training. The learning rate for both the binary and multi-class classifiers is adaptively managed through the ReduceLROnPlateau scheduler. If the validation loss shows no improvement over a set number of epochs (patience), the learning rate is halved. This

approach allows the model to make finer adjustments during training, which can help accelerate convergence. To avoid excessively small updates, the learning rate is capped at a minimum value of 1×10^{-5} . This method ensures more efficient and stable training, enabling the model to converge steadily without overshooting the optimal solution. Across both classifier types, the Adam optimizer is employed, known for its adaptive learning rate capabilities, which enhances training performance. The choice of loss function is tailored to the nature of the classification task. Binary cross-entropy is adopted for the binary classification scenario, while categorical cross-entropy is utilized in multi-class classification, ensuring appropriate measurement of model performance based on the output format. Lastly, accuracy is designated as the evaluation metric for both classifiers, providing a straightforward assessment of their performance in correctly classifying the input data. This careful selection and configuration of hyperparameters are essential for optimizing the effectiveness of the CNN models in their respective classification tasks.

Table 20. Hyperparameters for CNN model.

Parameter	Binary Classifier	Multi-Class Classifier
Batch size	128	128
Learning rate	Scheduled: Initial = 0.001, Factor = 0.5, Min = 1×10^{-5} (ReduceLROnPlateau)	Scheduled: Initial = 0.001, Factor = 0.5, Min = 1×10^{-5} (ReduceLROnPlateau)
Optimizer	Adam	Adam
Loss function	Binary cross-entropy	Categorical_crossentropy
Metric	Accuracy	Accuracy

3.3.2. Auto Encoder (AE)

Auto encoder is tailored for both binary and multi-class classification tasks, starting with an input layer that accepts feature vectors. It features an encoder composed of several dense layers that progressively reduce the input's dimensionality while applying the ReLU activation function, effectively extracting important features from the data. For binary classification, a classification layer follows, using the sigmoid activation function, while for multi-class classification, it uses the softmax activation function, enabling the model to output class probabilities for the respective scenarios. The model is compiled with the Adam optimizer and employs binary cross-entropy loss for binary tasks and categorical cross-entropy loss for multi-class tasks, ensuring appropriate loss calculations for each classification type. Additionally, a callback is implemented to adjust the learning rate based on validation loss, facilitating improved convergence and minimizing the risk of overfitting. The training and validation accuracies are plotted across epochs to evaluate the model's performance in both classification contexts.

The encoder layers progressively reduce the dimensionality of the input data, extracting important features through dense layers. This dimensionality reduction and feature extraction process can be mathematically expressed as shown in Equation (14) [89].

$$h^{(l)} = f \left(W^{(l)} a^{(l-1)} + b^{(l)} \right) \quad (14)$$

In this formulation, $h^{(l)}$ denotes the output of the encoder layer l , while $a^{(l-1)}$ represents the output from the previous layer, serving as the input for the first layer. The weight matrix for layer l is indicated by $W^{(l)}$, and $b^{(l)}$ denotes the bias vector for that layer. The activation function applied is denoted as f , which is specifically the ReLU function in this context.

In a standard auto encoder, the decoder layer reconstructs the input from the compressed representation learned by the encoder. This reconstruction process can be mathematically represented as detailed in Equation (15) [89].

$$\hat{\alpha} = g \left(W^{(d)} h^{(l)} + b^{(d)} \right) \quad (15)$$

In this context, $\hat{\alpha}$ represents the reconstructed output, while $h^{(l)}$ is the output from the last encoder layer. The weight matrix for the decoder layer is denoted as $W^{(d)}$, and $b^{(d)}$ indicates the bias vector for the decoder layer. The activation function used for the decoder is represented by g , which is typically linear for reconstruction purposes.

The classification layer utilizes a specific activation function for binary classification output. This can be expressed as presented in Equation (16) [89].

$$\bar{y} = \sigma \left(W^{(out)} h^{(l)} + b^{(out)} \right) \quad (16)$$

In this framework, $\bar{y}$ denotes the predicted probability for the positive class. The weight matrix for the output layer is represented by $W^{(out)}$, while $b^{(out)}$ signifies the bias for the output layer. The sigmoid function, denoted as σ , is employed to map the output to a probability score between 0 and 1.

For multi-class classification output, the classification layer employs the softmax activation function, which enables the model to generate a probability distribution across multiple classes. This can be expressed as presented in Equation (17) [92].

$$\bar{y} = \text{softmax} \left(W^{(out)} h^{(l)} + b^{(out)} \right) \quad (17)$$

In this context, $\bar{y}$ represents the vector of predicted probabilities across multiple classes. The weight matrix $W^{(out)}$ and bias $b^{(out)}$ are associated with the output layer. The softmax function is utilized to convert the logits into probabilities, ensuring that the predicted values sum to one across all classes.

(i) Binary Classification

The architecture outlined in Table 21 presents the layers of the Auto Encoder model designed for binary classification, tailored for both the NF-UNSW-NB15-v2 and CICIDS2017 datasets. The input block is configured to accommodate the unique features of each dataset. The NF-UNSW-NB15-v2 dataset processes data with 25 features, while the CICIDS2017 dataset handles 69 features. This input layer serves as the entry point for data, providing the foundation for the model's operations. The encoder structure comprises three dense layers, with 128, 64, and 32 neurons, respectively. Each dense layer utilizes the ReLU activation function, which introduces non-linearity and facilitates the extraction of complex patterns within the data. These layers effectively compress the input data into a lower-dimensional latent space, capturing the most critical features necessary for effective classification. The final output block consists of a single neuron activated by a sigmoid function. This layer generates a probability score indicating the likelihood of the input data belonging to the positive class, enabling binary classification. The architecture is designed to distinguish effectively between the two classes, ensuring robust performance across both datasets. This carefully structured model leverages its shared architecture to handle the unique characteristics of the NF-UNSW-NB15-v2 and CICIDS2017 datasets, enhancing its overall classification effectiveness.

Table 21. Auto encoder model layers for binary classification.

Dataset	Block	Layers	Layer Size	Activation
NF-UNSW-NB15-v2	Input block	Input layer	25	-
CICIDS2017	Input block	Input layer	69	-
Shared Structure	Encoder	Dense layer	128	ReLU
	Encoder	Dense layer	64	ReLU
	Encoder	Dense layer	32	ReLU
	Output block	Output layer	1	Sigmoid

(ii) Multi-Class Classification

The architecture outlined in Table 22 showcases the auto encoder model specifically designed for multi-class classification, with tailored configurations for both the NF-UNSW-NB15-v2 and CICIDS2017 datasets. The model begins with an input layer that accommodates the unique features of each dataset. The NF-UNSW-NB15-v2 dataset processes data with 27 features, while the CICIDS2017 dataset handles 35 features. This input layer serves as the entry point, setting the foundation for the model's processing. The encoder consists of three dense layers, with 128, 64, and 32 neurons respectively. Each layer employs the ReLU activation function, which introduces non-linearity and enhances the model's ability to capture intricate patterns and relationships within the data. This structure efficiently compresses the input data into a lower-dimensional latent space, extracting the most essential features for multi-class classification. The architecture culminates in the output block, where the NF-UNSW-NB15-v2 dataset's output layer consists of 10 neurons, and the CICIDS2017 dataset's output layer has 15 neurons. Both layers are activated by the softmax function, which generates class probabilities, allowing the model to classify the input data into multiple distinct categories. This design enables the model to address multi-class classification tasks effectively, distinguishing between various classes with high accuracy across both datasets.

Table 22. Auto encoder model layers for multi-class classification.

Dataset	Block	Layers	Layer Size	Activation
NF-UNSW-NB15-v2	Input block	Input layer	27	-
CICIDS2017	Input block	Input layer	35	-
Shared Structure	Encoder	Dense layer	128	ReLU
	Encoder	Dense layer	64	ReLU
	Encoder	Dense layer	32	ReLU
NF-UNSW-NB15-v2	Output block	Output layer	10	Softmax
CICIDS2017	Output block	Output layer	15	Softmax

(iii) Hyperparameter Configuration for the Auto Encoder Model

The hyperparameters for the auto encoder model, as outlined in Table 23, are designed to suit both binary and multi-class classification tasks. In both cases, a batch size of 128 is used to streamline the training process. The learning rate for both classifiers is dynamically adjusted using the ReduceLROnPlateau scheduling mechanism. This technique monitors the validation loss during training, and if no improvement is observed over two consecutive epochs, the learning rate is reduced by a factor of 0.5. This gradual reduction enables more stable and refined parameter updates, particularly in the later stages of training, which

enhances the model's ability to converge effectively. Furthermore, the learning rate is capped with a minimum value of 1×10^{-5} to prevent it from becoming too small to produce meaningful updates. This strategy strikes a balance between accelerating convergence in the early stages and allowing for finer adjustments as the model nears optimal performance, ultimately leading to more reliable and efficient training. The Adam optimizer is employed for efficient weight updates, while the choice of loss function depends on the classification task. Binary cross-entropy is used for binary classification, and categorical cross-entropy is applied for multi-class classification. For performance evaluation, accuracy is chosen as the primary metric, offering a comprehensive assessment of the model's ability to classify data correctly in both contexts.

Table 23. Auto encoder model hyperparameters.

Parameter	Binary Classifier	Multi-Class Classifier
Batch size	128	128
Learning rate	Scheduled: Initial = 0.001, Factor = 0.5, Min = 1×10^{-5} (ReduceLROnPlateau)	Scheduled: Initial = 0.001, Factor = 0.5, Min = 1×10^{-5} (ReduceLROnPlateau)
Optimizer	Adam	Adam
Loss function	Binary cross-entropy	Categorical_crossentropy
Metric	Accuracy	Accuracy

3.3.3. Deep Neural Network (DNN)

The DNN model for binary and multi-class classification consists of several blocks, including the input block, two hidden blocks, and the output block. The model begins with an input layer, where the number of features varies depending on the dataset, and a dense layer utilizing ReLU activation to learn complex patterns. This is followed by the first hidden block, which includes a dropout layer with a very small rate to prevent overfitting, followed by a dense layer with ReLU activation, then batch normalization. The second hidden block includes another dropout layer with the same small rate and batch normalization to improve training stability. The output layer differs based on the classification type. For binary classification, it features a single neuron with a sigmoid activation function to produce a probability score, while for multi-class classification, it contains multiple neurons with a softmax activation function to provide class probabilities. The model is compiled using the Adam optimizer with a learning rate defined by an exponential decay schedule. It uses binary cross-entropy for binary tasks or categorical cross-entropy for multi-class tasks, enabling effective training. Additionally, a custom callback is implemented to visualize the confusion matrix at the end of each epoch, offering valuable insights into the model's classification performance by comparing predicted and actual labels.

The feed-forward operation in a DNN involves passing the input through multiple layers to produce the output. This can be expressed mathematically for a layer l as presented in Equation (18) [93].

$$a^{(l)} = f \left(W^{(l)} a^{(l-1)} + b^{(l)} \right) \quad (18)$$

In this context, $a^{(l)}$ denotes the activation of the current layer l . The weight matrix for this layer is represented by $W^{(l)}$, while $b^{(l)}$ signifies the bias vector for layer l . The activation function f is applied element-wise, which may include functions such as ReLU or sigmoid, to introduce non-linearity into the model.

The ReLU activation function, shown in Equation (19) [94], is a simple and efficient non-linear function that outputs zero for negative values, promotes sparse activations, and supports effective gradient flow, making it ideal for deep learning.

$$\text{ReLU}(x) = \max(0, x) \quad (19)$$

For binary classification tasks, the sigmoid function is utilized, which outputs a probability score indicating the likelihood of an instance belonging to the positive class. The sigmoid function transforms the raw score into a value between 0 and 1, effectively serving as a threshold for classification. This can be represented as presented in Equation (20) [92].

$$\sigma(Z) = \frac{1}{1 + e^{-z}} \quad (20)$$

where Z is the output from the last dense layer.

The output layer employs the softmax function for multi-class classification tasks, enabling the model to produce probability distributions across multiple classes. This function takes a vector of raw scores (logits) and normalizes them into a range between 0 and 1, where the sum of the probabilities equals 1. The softmax function can be mathematically expressed as shown in Equation (21) [92].

$$\text{Softmax}(Z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (21)$$

where Z_i is the output from the last dense layer for class i , and Z_j represents the raw score for class j .

(i) Binary Classification

The architecture detailed in Table 24 presents the structure of the DNN model specifically designed for binary classification, with tailored configurations for the NF-UNSW-NB15-v2 and CICIDS2017 datasets. The input layer block begins by processing 25 features for the NF-UNSW-NB15-v2 dataset and 69 features for the CICIDS2017 dataset, and a dense layer with 1024 neurons, where the ReLU activation function is applied to introduce non-linearity and enhance the model's ability to learn complex representations from the data. The first hidden block includes a dropout layer with a very low dropout rate to mitigate overfitting, followed by a dense layer with 768 neurons. The dense layer is equipped with a ReLU activation function to introduce non-linearity, enhancing the model's ability to learn complex patterns. Batch normalization is applied after the dense layer to stabilize the learning process by normalizing the outputs, ensuring more effective and consistent training. The second hidden block contains another dropout layer and batch normalization, further refining the learning dynamics. Ultimately, the architecture concludes with an output layer featuring a single neuron activated by the sigmoid function. This configuration is meticulously crafted to enhance the model's effectiveness in binary classification tasks across both datasets.

Table 24. DNN model layers for binary classification.

Dataset	Block	Layers	Layer Size	Activation
NF-UNSW-NB15-v2	Input block	Input layer	25	ReLU
		Dense layer	1024	-
CICIDS2017	Hidden block 1	Dropout layer	0.0000001	ReLU
		Dense layer	768	-
	Hidden block 2	Batch normalization	-	-
		Dropout layer	0.0000001	-
Shared Structure	Output block	Batch normalization	-	-
		Output layer	1	Sigmoid

(ii) Multi-Class Classification

The structure outlined in Table 25 describes the architecture of the DNN model designed for multi-class classification, with specific configurations for the NF-UNSW-NB15-v2 and CICIDS2017 datasets. The input layer block begins by processing 27 features for the NF-UNSW-NB15-v2 dataset and 35 features for the CICIDS2017 dataset, and a dense layer with 1024 neurons, where the ReLU activation function is applied to introduce non-linearity and enhance the model's ability to learn complex representations from the data. The first hidden block incorporates a dropout layer with a minimal dropout rate to mitigate overfitting. This is followed by a dense layer containing 768 neurons, which is augmented with ReLU activation to introduce non-linearity. Batch normalization is applied after the dense layer to stabilize the learning process by normalizing the output, ensuring more effective and faster training. The second hidden block contains another dropout layer and batch normalization, further refining the learning dynamics. The architecture concludes with an output layer for each dataset. The NF-UNSW-NB15-v2 dataset's output layer consists of 10 neurons, while the CICIDS2017 dataset's output layer has 15 neurons. Both output layers are activated by a softmax function, generating class probabilities across their respective categories. This design enables the model to effectively handle multi-class classification tasks across both datasets with precision.

Table 25. DNN model layers for multi-class classification.

Dataset	Block	Layers	Layer Size	Activation
NF-UNSW-NB15-v2	Input block	Input layer	27	ReLU
CICIDS2017	Input block	Input layer	35	ReLU
Shared Structure	Hidden block 1	Dense layer	1024	-
		Dropout layer	0.0000001	ReLU
		Dense layer	768	-
		Batch normalization	-	-
	Hidden block 2	Dropout layer	0.0000001	-
		Batch normalization	-	-
NF-UNSW-NB15-v2	Output block	Output layer	10	Softmax
CICIDS2017	Output block	Output layer	15	Softmax

(iii) Hyperparameter Configuration for the DNN Model

The hyperparameters for the DNN models, detailed in Table 26, are tailored to accommodate both binary and multi-class classification tasks. Each classifier utilizes a consistent batch size of 128 and employs the Adam optimizer for efficient training. The learning rate for both the binary and multi-class classifiers is governed by an exponential decay schedule, which dynamically adjusts the learning rate throughout the training process. Initially, the learning rate is set to 0.0003. As training progresses, the learning rate undergoes a reduction by a factor of 0.9 after every 10,000 steps. This progressive decrease in the learning rate ensures that the model can make larger, more decisive updates in the early stages of training, followed by more refined and precise adjustments in the later stages. This method of adaptive learning rate adjustment is designed to promote a more stable and efficient optimization process, ultimately facilitating smoother convergence toward an optimal solution. For loss functions, binary cross-entropy is applied in the context of binary classification, while categorical cross-entropy is utilized for the multi-class classification

scenario. In both cases, accuracy serves as the primary evaluation metric, providing a clear measure of the models' effectiveness in classifying data accurately.

Table 26. DNN model hyperparameters.

Parameter	Binary Classifier	Multi-Class Classifier
Batch size	128	128
Learning rate	Scheduled: Initial = 0.0003, Factor = 0.9, Decay Steps = 10,000 (Exponential Decay)	Scheduled: Initial = 0.0003, Factor = 0.9, Decay Steps = 10,000 (Exponential Decay)
Optimizer	Adam	Adam
Loss function	Binary_crossentropy	Categorical_crossentropy
Metric	Accuracy	Accuracy

3.3.4. Transformer-Convolutional Neural Network (Transformer CNN)

The model architecture presented integrates both Transformer and CNN components to enhance classification performance for the given task. The input layer receives data in a structured format, setting the stage for the subsequent processing. The Transformer block plays a crucial role in capturing intricate relationships within the input data through its multi-head attention mechanism. This approach allows the model to weigh different parts of the input more dynamically, facilitating the identification of complex patterns and dependencies. To stabilize the learning process and improve gradient flow, a layer normalization and residual connection are employed. Following the attention mechanism, a feed-forward neural network (FFN) processes the output, enhancing the data representation by applying non-linear transformations and introducing dropout layers for regularization. The primary job of the Transformer is to provide a global context and highlight important features across the entire input sequence. Subsequently, the CNN blocks operate on the output of the Transformer, focusing on local feature extraction. Each convolutional layer applies filters to detect various features within the input data, while batch normalization and activation functions such as ReLU ensure that the model remains robust and learns effectively. Max pooling layers downsample the data, reducing its dimensionality and allowing the model to concentrate on the most salient features. The CNN's primary function is to capture spatial hierarchies and patterns within the input, making it particularly effective for tasks requiring detailed analysis of local structures. The architecture also includes a flattening step that prepares the output of the CNN blocks for further processing. This flattened representation is then passed through MLP blocks, which serve to learn high-level abstractions from the features extracted by the CNNs. The concatenation of the Transformer and CNN outputs at this stage enables the model to leverage both global context and local feature patterns for improved classification accuracy. Finally, the output layer employs a sigmoid or softmax activation function to generate class probabilities, completing the model's capability to classify inputs based on the rich representations learned throughout the architecture. This integrated approach harnesses the strengths of both Transformer and CNN architectures, providing a comprehensive framework for effective classification tasks.

The multi-head attention mechanism effectively captures complex relationships within the input data, as represented by Equation (22) [95].

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \quad (22)$$

In this context, Q represents the query matrix, K denotes the key matrix, and V signifies the value matrix. The variable d_k refers to the dimension of the keys, which plays a crucial role in the computation of attention scores within the model.

Each head performs this attention calculation independently and then concatenates the results, as detailed in Equation (23) [95].

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^0 \quad (23)$$

In this context, W^0 refers to the output weight matrix, which is utilized to transform the output of the preceding layer into the final output of the model.

Layer normalization stabilizes the output of each layer, using Equation (24) [96].

$$\text{LayerNorm}(x) = \frac{X - \mu}{\sigma} * \gamma + \beta \quad (24)$$

In this context, μ represents the mean of the inputs, while σ denotes the standard deviation. Additionally, γ and β are learnable parameters that are utilized in the normalization process.

The FFN processes the output from the attention mechanism as presented in Equation (25) [95].

$$\text{FFN}(X) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (25)$$

In this context, W_1 and W_2 refer to the weight matrices, while b_1 and b_2 represent the corresponding biases associated with the layers in the model.

The convolution operation in the CNN layers can be defined as in Equation (26) [89].

$$Z_{i,j} = (X * K)_{i,j} = \sum_m \sum_n Z_{i+m,j+n} k_{m,n} \quad (26)$$

In this scenario, Z denotes the output feature map resulting from the convolution process, while X represents the input feature map. The convolution kernel, denoted as K , is used to perform the convolution operation between the input and the output.

The ReLU activation function, presented in Equation (27) [90], is efficient and straightforward, outputting zero for negative inputs while passing positive values through unchanged. Its ability to promote sparse activations and facilitate gradient flow makes it particularly effective in deep learning applications.

$$\text{ReLU}(x) = \max(0, x) \quad (27)$$

The max pooling operation can be expressed as presented in Equation (28) [89].

$$P_{i,j} = \max(X_{i:i+p, j:j+q}) \quad (28)$$

In this context, P represents the pooled output generated from the pooling operation, while P and q denote the dimensions of the pooling window applied to the input feature map.

The dropout layer randomly sets a fraction p of input units to zero during training to prevent overfitting, as presented in Equation (29) [91].

$$\text{Dropout}(x) = \begin{cases} x & \text{with probability } 1 - p \\ 0 & \text{with probability } p \end{cases} \quad (29)$$

For binary classification tasks, the sigmoid function is utilized, which outputs a probability score indicating the likelihood of an instance belonging to the positive class, as presented in Equation (30) [92].

$$\sigma(Z) = \frac{1}{1 + e^{-Z}} \quad (30)$$

where Z is the output from the last dense layer.

The output layer employs the softmax function for multi-class classification tasks, enabling the model to produce probability distributions across multiple classes, as presented in Equation (31) [92].

$$\text{Softmax}(Z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (31)$$

where Z_i is the output for class i , and Z_j represents the raw score for class j .

(i) Binary Classification

The architecture of the Transformer model designed for binary classification is detailed in Table 27. The model begins with an input layer that processes data structured as (25, 1) for the NF-UNSW-NB15-v2 dataset and (69, 1) for the CICIDS2017 dataset, effectively accommodating input with 25 features for NF-UNSW-NB15-v2 and 69 features for CICIDS2017. Following this, the Transformer block employs a multi-head attention mechanism with eight heads and a key dimension of 128. This mechanism captures complex relationships within the input data, enhancing the model's ability to identify intricate patterns. The output from the attention layer is subsequently normalized using layer normalization with an epsilon value of 1×10^{-6} , which helps stabilize the output. A residual connection is implemented to add the original input data back to the attention output, promoting stability during training. The feed-forward block consists of a dense layer with 512 units and a ReLU activation function, applying a transformation to the data. This is followed by a dropout layer with a rate of 0.0000001, aimed at mitigating overfitting by regularizing the network. Another dense layer with 512 units is included without an activation function, allowing for additional transformations. A subsequent dropout layer with the same rate further reinforces regularization, enhancing model robustness. The output from the feed-forward network is then added back to the previous block's output via another residual connection, followed by another layer normalization step with $\epsilon = 1 \times 10^{-6}$ to normalize the combined output, ensuring stability in the model's learning process.

The architecture of the CNN model designed for binary classification utilizes the output of the Transformer model as its input and is tailored for datasets like NF-UNSW-NB15-v2 and CICIDS2017. The input block processes the Transformer output, providing structured input for the model. The first hidden block includes a 1D CNN layer with 512 filters and a ReLU activation function, which extracts essential features from the input data. This is followed by a 1D max pooling layer with a pool size of two, reducing the dimensionality of the feature maps, and a dropout layer with a rate of 0.0000001 to mitigate overfitting. The second hidden block repeats this structure with another 1D CNN layer with 512 filters and ReLU activation, followed by a 1D max pooling layer with a pool size of four and a dropout layer with the same dropout rate. In the third hidden block, the model incorporates a dense layer with 1024 units and a ReLU activation function, enhancing the model's representational capabilities. A dropout layer with a rate of 0.0000001 is again applied for additional regularization. The architecture concludes with a single-output layer employing a sigmoid activation function for binary classification, producing a probability score to determine class membership. The detailed structure, including layer sizes, activation functions, and dropout rates, is outlined in Table 28.

Table 27. Transformer model layers for binary classification.

Dataset	Block	Layer Type	Output Size	Activation Function	Parameters	Description
NF-UNSW-NB15-v2	Input block	Input layer	(25, 1)	-	-	Accepts input data with 25 features.
CICIDS2017	Input block	Input layer	(69, 1)	-	-	Accepts input data with 69 features.

Table 27. Cont.

Dataset	Block	Layer Type	Output Size	Activation Function	Parameters	Description
Shared Structure	Transformer block	Multi-head attention	-	-	num_heads = 8, key_dim = 128	Captures complex relationships within input data.
		Layer Normalization	-	-	epsilon = 1×10^{-6}	Normalizes the output from the attention layer.
		Add (Residual Connection)	-	-	-	Adds input data to the attention output for stability.
	Feed Forward block	Dense layer	512	ReLU	units = 512, activation = 'relu'	Applies a dense transformation with ReLU activation.
		Dropout layer	-	-	rate = 0.0000001	Regularizes the network to prevent overfitting ($p = 0.0000001$).
		Dense layer	512	-	units = 512	Another dense transformation without activation.
		Dropout layer	-	-	rate = 0.0000001	Further regularization ($p = 0.0000001$).
		Add (Residual Connection)	-	-	-	Adds feed-forward output to the previous block output.
		Layer Normalization	-	-	epsilon = 1×10^{-6}	Normalizes the combined output for stability.

Table 28. CNN model layers for binary classification.

Dataset	Block	Layers	Layer Size	Activation
Shared Structure	Input block	Input layer	Transformer output	-
	Hidden block 1	1D CNN layer	512	ReLU
		1D Max Pooling layer	2	-
		Dropout layer	0.0000001	-
	Hidden block 2	1D CNN layer	512	ReLU
		1D Max Pooling layer	4	-
		Dropout layer	0.0000001	-
	Hidden block 3	Dense layer	1024	ReLU
		Dropout layer	0.0000001	-
	Output block	Output layer	1	Sigmoid

(ii) Multi-Class Classification

The architecture of the Transformer model designed for multi-class classification is outlined in Table 29. The model starts with an input layer that processes data structured as (27, 1) for the NF-UNSW-NB15-v2 dataset and (35, 1) for the CICIDS2017 dataset, effectively accommodating input with 27 features for NF-UNSW-NB15-v2 and 35 features for CICIDS2017. Following this, the Transformer block utilizes a multi-head attention mechanism with eight heads and a key dimension of 128, which captures complex relationships within the input data and enhances the model's ability to identify intricate patterns. The output

from the attention layer is then normalized using layer normalization with an epsilon value of 1×10^{-6} , which contributes to stabilizing the output. A residual connection is established to add the original input data back to the attention output, promoting stability during training. The feed-forward block consists of a dense layer with 512 units and a ReLU activation function, which applies a transformation to the data. This is followed by a dropout layer with a rate of 0.0000001, designed to mitigate overfitting by regularizing the network. An additional dense layer with 512 units is included without an activation function, allowing for further transformations. A subsequent dropout layer with the same rate reinforces regularization, enhancing the model's robustness. The output from the feed-forward network is then added back to the previous block's output via another residual connection, followed by an additional layer normalization step with epsilon = 1×10^{-6} to normalize the combined output, ensuring stability in the model's learning process.

Table 29. Transformer model layers for multi-class classification.

Dataset	Block	Layer Type	Output Size	Activation Function	Parameters	Description
NF-UNSW-NB15-v2	Input block	Input layer	(27, 1)	-	-	Accepts input data with 27 features.
CICIDS2017	Input block	Input layer	(35, 1)	-	-	Accepts input data with 35 features.
Shared Structure	Transformer block	Multi-head attention	-	-	num_heads = 8, key_dim = 128	Captures complex relationships within input data.
		Layer Normalization	-	-	epsilon = 1×10^{-6}	Normalizes the output from the attention layer.
		Add (Residual Connection)	-	-	-	Adds input data to the attention output for stability.
	Feed Forward block	Dense layer	512	ReLU	units = 512, activation = 'relu'	Applies a dense transformation with ReLU activation.
		Dropout layer	-	-	rate = 0.0000001	Regularizes the network to prevent overfitting ($p = 0.0000001$).
		Dense layer	512	-	units = 512	Another dense transformation without activation.
		Dropout layer	-	-	rate = 0.0000001	Further regularization ($p = 0.0000001$).
		Add (Residual Connection)	-	-	-	Adds feed-forward output to the previous block output.
		Layer Normalization	-	-	epsilon = 1×10^{-6}	Normalizes the combined output for stability.

The architecture of the CNN model designed for multi-class classification leverages the output of the Transformer model as its input, tailored for both the NF-UNSW-NB15-v2 and CICIDS2017 datasets. The model starts with an input block that processes the Transformer output. The first hidden block incorporates a 1D CNN layer with 512 filters and a ReLU activation function, enabling the extraction of critical features from the data. This is followed by a 1D max pooling layer with a pool size of two, which reduces dimensionality, and a dropout layer with a rate of 0.0000001 to mitigate overfitting. In the second hidden block, another 1D CNN layer with 512 filters and a ReLU activation function is utilized, accompanied by a 1D max pooling layer with a pool size of 4 and another dropout layer with the same rate, reinforcing regularization. The third hidden block comprises a dense layer with 1024 units and a ReLU activation function, further enhancing the model's ability

to represent complex patterns. This block also includes a dropout layer with a rate of 0.0000001 for additional regularization. The output block varies based on the dataset. For the NF-UNSW-NB15-v2 dataset, the output layer consists of 10 units, while for the CICIDS2017 dataset, it includes 15 units. Both employ a softmax activation function to perform multi-class classification. The complete architecture, including layer specifications, is outlined in Table 30.

Table 30. CNN model layers for multi-class classification.

Dataset	Block	Layers	Layer Size	Activation
Shared Structure	Input block	Input layer	Transformer output	-
	Hidden block 1	1D CNN layer	512	ReLU
		1D Max Pooling layer	2	-
		Dropout layer	0.0000001	-
	Hidden block 2	1D CNN layer	512	ReLU
		1D Max Pooling layer	4	-
		Dropout layer	0.0000001	-
	Hidden block 3	Dense layer	1024	ReLU
		Dropout layer	0.0000001	-
NF-UNSW-NB15-v2	Output block	Output layer	10	Softmax
CICIDS2017	Output block	Output layer	15	Softmax

(iii) Hyperparameter Configuration for the Transformer-CNN Model

The hyperparameters for the Transformer-CNN model, detailed in Table 31, have been meticulously optimized for effectiveness in both binary and multi-class classification tasks. The model operates with a batch size of 128, which defines the number of samples processed before the model's weights are updated, ensuring consistency across both classification scenarios. The learning rate for both the binary and multi-class classifiers is dynamically adjusted using the ReduceLROnPlateau schedule. If the validation loss does not improve for a specified number of epochs (patience), the learning rate is reduced by a factor of 0.5. This strategy helps to fine-tune the model's learning process, allowing for smaller adjustments as training progresses, potentially leading to improved convergence. The learning rate is bounded below by a minimum value of 1×10^{-5} , preventing it from becoming so small that the model makes ineffective updates. This approach enhances training efficiency and stability, ensuring the model can reliably converge without overshooting the optimal solution. The Adam optimizer is utilized due to its robust adaptive learning features, demonstrating effectiveness in both binary and multi-class contexts. In the case of binary classification, the model leverages binary cross-entropy as its loss function, quantifying the divergence between predicted probabilities and actual binary outcomes. In contrast, the multi-class classification model employs categorical cross-entropy, assessing the difference between predicted class probabilities and the true class labels across multiple categories. For performance evaluation, both models utilize accuracy as their primary metric, which reflects the ratio of correctly predicted instances to the total number of predictions made. This metric serves as a straightforward indicator of model performance, illustrating the extent to which predicted labels correspond with actual labels.

Table 31. Hyperparameters of the Transformer-CNN model.

Parameter	Binary Classifier	Multi-Class Classifier
Batch size	128	128
Learning rate	Scheduled: Initial = 0.001, Factor = 0.5, Min = 1×10^{-5} (ReduceLROnPlateau)	Scheduled: Initial = 0.001, Factor = 0.5, Min = 1×10^{-5} (ReduceLROnPlateau)
Optimizer	Adam	Adam
Loss function	Binary cross-entropy	Categorical_crossentropy
Metric	Accuracy	Accuracy

4. Results and Experiments

In this section, we present a comprehensive evaluation of the proposed models, incorporating advanced data resampling techniques and class weight adjustments to address class imbalance effectively. To ensure a robust comparison, the performance of our approach is assessed alongside state-of-the-art intrusion detection methods. The experimental findings demonstrate that the proposed model achieves superior results, setting a benchmark in anomaly detection performance.

4.1. Dataset Description and Preprocessing Overview

The datasets utilized in this study, NF-UNSW-NB15-v2 and CICIDS2017, are among the most comprehensive benchmarks for evaluating IDS. These datasets capture diverse network behaviors and attack scenarios, offering a solid foundation for developing and assessing anomaly detection models. Despite their strengths, both datasets present challenges such as missing data, duplicates, outliers, and class imbalance, which necessitate rigorous preprocessing. This section provides an overview of the datasets, their suitability for binary and multi-class classification tasks, and their relevance to IDS research, along with the essential preprocessing steps. These steps address issues like missing values, eliminating duplicates, handling outliers, and balancing the class distribution to optimize the datasets for effective model evaluation.

4.1.1. NF-UNSW-NB15-v2 Dataset

The NF-UNSW-NB15-v2 dataset, as described in Section 3.1, captures diverse network behaviors, including normal and malicious traffic across various attack types, providing valuable features for IDS development. However, it faces challenges such as missing values, duplicates, and class imbalance, which are addressed through preprocessing outlined in Section 3.2. This included handling missing values, eliminating duplicates, applying outlier detection techniques like z-score and LOF, performing feature selection to reduce dimensionality, and normalizing numerical features using MinMaxScaler. Advanced resampling methods, such as ADASYN for oversampling and ENN for undersampling, were applied, along with dynamic class weights during training to improve class representation. This comprehensive preprocessing optimized the dataset for both binary and multi-class classification tasks.

4.1.2. CICIDS2017 Dataset

Certain aspects, such as data structure and labeling, are pivotal for effective intrusion detection in network-based datasets. Markus et al. [97] offer a thorough analysis of these factors in both supervised and unsupervised intrusion detection techniques. This section delves into the history and characteristics of the CICIDS2017 dataset, which is utilized in this study for intrusion detection. Released by the Canadian Institute for Cybersecurity, this dataset is publicly available for academic research purposes [98]. It is one of the most up-to-date datasets for network intrusion detection found in the literature, comprising 2,830,743 records, 79 network traffic features, and 15 classes, including 1 for Benign traffic

and 14 distinct attack types [12]. The dataset is organized into eight files representing five days of benign and attack traffic, with each file containing real-world network data [98,99]. In addition to the core traffic data, the records include supplementary metadata and are provided in packet-based and bifacial flow-based formats [97]. The dataset is fully labeled, making it suitable for both binary and multi-class classification tasks. For binary classification, all attack types are labeled as '1', while benign traffic is labeled as '0'. For multi-class classification, all attack types are considered individually, providing a comprehensive view of the different forms of network attacks. The CICIDS2017 dataset, while extensive, requires meticulous preprocessing to address missing data and enhance its quality for analysis. Preprocessing began by consolidating the dataset's eight constituent files into a single comprehensive dataset. Missing values, or NaNs, were systematically addressed to prevent data quality issues. Duplicates were eliminated, and columns with only a single unique value were removed to optimize feature relevance. Remaining NaN values were carefully imputed, and feature names were standardized by stripping leading spaces for uniformity. Sampling was then performed, and for multi-class classification, instances belonging to the 'Normal' class were excluded post-sampling. To eliminate extreme values that could bias model outcomes, outliers were identified and removed using the LOF. In multi-class classification, feature selection based on correlation is applied following outlier removal to refine the feature set further. Then, numerical features are normalized using MinMaxScaler to ensure consistent scaling across variables. After these steps, the dataset was partitioned into training and testing subsets. To address class imbalances during training, advanced resampling techniques were implemented. For binary classification, the enhanced hybrid ADASYN-SMOTE method was applied to generate synthetic samples within the training data, while for multi-class classification, an advanced cascaded SMOTE approach was utilized to balance the training dataset effectively. Additionally, the ENN technique was employed to undersample the training data, further refining class distribution and improving model robustness. Class weights were dynamically adjusted during the training process to ensure balanced learning across all classes. Collectively, these preprocessing strategies transformed the raw CICIDS2017 dataset into a well-balanced and optimized resource, tailored for binary and multi-class classification tasks.

4.2. Experiment's Establishment

The models were developed on the Kaggle platform 1.6.17 using TensorFlow 2.17.0 and Keras 3.4.1. The experimental configuration was equipped with hardware that included an Nvidia GeForce RTX 1050 graphics card and operated on Windows 10. Throughout the data resampling process, only the training set was utilized, while the evaluation dataset was reserved as the testing set. The training process involved executing the models for 500 epochs, with validation accuracy monitored throughout the training.

4.3. Evaluation Metrics

The confusion matrix is an essential tool for assessing the performance of machine learning models. It presents a structured table that juxtaposes the actual and predicted class labels, as detailed in reference [100]. This matrix facilitates the calculation of a range of performance metrics.

- True Positive (TP): These are the instances that the model correctly predicted to be positive. For example, if a spam filter correctly identified an email as spam, this is a true positive.
- False Negative (FN): These are the instances that the model incorrectly predicted to be negative. In the spam filter example, if it mistakenly classified a spam email as legitimate, this is a false negative.
- True Negative (TN): These are the instances that the model correctly predicted to be negative. Returning to our spam filter, if it accurately identified a non-spam email as non-spam, this is a true negative.

- False Positive (FP): These are the instances that the model incorrectly predicted to be positive. In the spam filter context, if it mistakenly classified a legitimate email as spam, this is a false positive.

Equation (32) [101] illustrates the most basic and fundamental metric, accuracy, which can be derived from the confusion matrix.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (32)$$

It is common to evaluate the model using a variety of additional metrics, including recall, precision, and the F-score. Precision is determined by dividing the number of true positive results by the total number of predicted positive results, encompassing both correct and incorrect identifications. This metric, also known as positive predictive value, is calculated using Equation (33) [101]. Recall, defined in Equation (34) [101], assesses the proportion of actual positive instances that the model correctly identifies among all instances that should have been recognized as positive. The F-score, computed using Equation (35) [102], serves as the harmonic mean of precision and recall, providing a balanced measure of the model's performance.

$$Precision = \frac{TP}{TP + FP} \quad (33)$$

$$Recall = \frac{TP}{TP + FN} \quad (34)$$

$$Fscore = \frac{2 * precision * recall}{precision + recall} \quad (35)$$

In this scenario, the goal is to enhance metrics including the F-score, accuracy, recall, and precision, as outlined by the evaluation criteria.

4.4. Results

The evaluation of the proposed models was conducted across two primary phases, training and testing, utilizing the train and test subsets of the NF-UNSW-NB15-v2 dataset, with additional evaluation on other datasets like CICIDS2017 to demonstrate the models' generalizability. These experiments targeted both binary and multi-class classification tasks, ensuring accurate detection of malicious activities and precise identification of various attack types. A comprehensive analysis was performed to assess the impact of data re-sampling techniques on the models' performance, offering a thorough comparison of their effectiveness. The models were also benchmarked against established intrusion detection systems from the literature, providing valuable insights into their relative strengths and weaknesses in a broader context. The results from both the NF-UNSW-NB15-v2 and CICIDS2017 datasets underscore the effectiveness and versatility of the proposed models in addressing complex classification challenges. Among the evaluated approaches, the Transformer-CNN model consistently emerged as the top performer, demonstrating exceptional accuracy in detecting malicious activities and classifying diverse attack types. While other models, such as auto encoder, DNN and CNN, delivered commendable results, the Transformer-CNN model proved to be the most resilient and reliable across all evaluation metrics, highlighting the critical role of applied preprocessing techniques and emphasizing the robustness and generalizability of the models.

(i) Binary Classification

The performance metrics presented in Table 32 illustrate the results of binary classification on the NF-UNSW-NB15-v2 and CICIDS2017 datasets, utilizing data resampling techniques and class weights. Each model demonstrated impressive performance across all metrics, highlighting their reliability and robustness in binary classification tasks. On

the NF-UNSW-NB15-v2 dataset, the CNN model achieved an accuracy of 99.69%, with precision, recall, and F-score all matching at 99.69%. The auto encoder reported an accuracy of 99.66%, and similarly, the DNN model achieved an accuracy of 99.68%, with corresponding precision, recall, and F-score values of 99.68%. The Transformer-CNN model outperformed the others, achieving the highest accuracy at 99.71%, along with matching precision, recall, and F-score metrics of 99.71%. On the CICIDS2017 dataset, the CNN model demonstrated outstanding performance, achieving an accuracy of 99.86%, with precision, recall, and F-score all equally high at 99.86%. The auto encoder model, while slightly lower, still achieved a strong accuracy of 99.73%, with corresponding precision, recall, and F-score values matching at 99.73%, suggesting it is effective in identifying anomalies and classifying the data accurately. The DNN model reported an impressive accuracy of 99.88%, with precision, recall, and F-score values consistently high at 99.88%, indicating that it is highly reliable in distinguishing between the different classes within the dataset. However, The Transformer-CNN model stood out as the best performer, achieving the highest accuracy of 99.93%, with precision, recall, and F-score all at 99.93%. These results highlight the impressive performance of each model in binary classification tasks across both datasets, showcasing their reliability and robustness for real-world applications. The Transformer-CNN model, in particular, emerged as the most effective, achieving the highest performance in binary classification on both datasets.

Table 32. Performance metrics in binary classification using data resampling and class weights.

Dataset	Metric	Accuracy	Precision	Recall	F-Score
NF-UNSW-NB15-v2	CNN	99.69%	99.69%	99.69%	99.69%
	Auto Encoder	99.66%	99.66%	99.66%	99.66%
	DNN	99.68%	99.68%	99.68%	99.68%
	Transformer-CNN	99.71%	99.71%	99.71%	99.71%
CICIDS2017	CNN	99.86%	99.86%	99.86%	99.86%
	Auto Encoder	99.73%	99.73%	99.73%	99.73%
	DNN	99.88%	99.88%	99.88%	99.88%
	Transformer-CNN	99.93%	99.93%	99.93%	99.93%

(ii) Multi-Class Classification

The performance metrics for various models in multi-class classification on the NF-UNSW-NB15-v2 and CICIDS2017 datasets, utilizing data resampling techniques and class weights, are summarized in Table 33. On the NF-UNSW-NB15-v2 dataset, the CNN model achieved an accuracy of 98.36%, with precision at 98.66%, recall at 98.36%, and F-score at 98.46%. The auto encoder showed slightly lower performance, with an accuracy of 95.57%, precision of 96.54%, recall of 95.57%, and an F-score of 95.77%. The DNN model attained an accuracy of 97.65%, with precision at 98.09%, recall at 97.65%, and F-score at 97.77%. The Transformer-CNN model stood out with the highest performance, achieving an accuracy of 99.02%, precision of 99.30%, recall of 99.02%, and F-score of 99.13%. On the CICIDS2017 dataset, the CNN model achieved an accuracy of 99.05%, with precision at 99.12%, recall at 99.05%, and F-score at 99.07%. The Auto Encoder performed similarly, with an accuracy of 99.09%, precision of 99.12%, recall of 99.09%, and F-score of 99.09%. The DNN model reported an accuracy of 99.11%, with precision at 99.20%, recall at 99.11%, and F-score at 99.14%. The Transformer-CNN model once again outperformed the others, achieving the highest accuracy of 99.13%, precision of 99.22%, recall of 99.13%, and F-score of 99.16%. These results emphasize the strong performance of each model in multi-class classification tasks across both datasets, showcasing their reliability and robustness in real-world applications. Notably, the Transformer-CNN model demonstrated the highest effectiveness, standing out as the most proficient model for multi-class classification on both datasets.

Table 33. Performance metrics in multi-class classification using data resampling and class weights.

Dataset	Metric	Accuracy	Precision	Recall	F-Score
NF-UNSW-NB15-v2	CNN	98.36%	98.66%	98.36%	98.46%
	Auto Encoder	95.57%	96.54%	95.57%	95.77%
	DNN	97.65%	98.09%	97.65%	97.77%
	Transformer-CNN	99.02%	99.30%	99.02%	99.13%
CICIDS2017	CNN	99.05%	99.12%	99.05%	99.07%
	Auto Encoder	99.09%	99.12%	99.09%	99.09%
	DNN	99.11%	99.20%	99.11%	99.14%
	Transformer-CNN	99.13%	99.22%	99.13%	99.16%

5. Discussion

This section provides a comprehensive evaluation of the Transformer-CNN model's performance in comparison to other classification methods, such as CNN, auto encoder, and DNN, across both binary and multi-class classification tasks. We conduct a detailed analysis of the confusion matrices and key performance metrics, including accuracy, precision, recall, and F1-score, to offer a comparative assessment of each model's strengths and weaknesses. Results obtained from the NF-UNSW-NB15-v2 dataset, along with additional evaluation on other datasets like CICIDS2017 to demonstrate the models' generalizability, reveal how the Transformer-CNN model's innovative integration of Transformer and CNN architectures enhances its ability to detect malicious activities and classify various attack types. This analysis not only highlights the model's superior performance across multiple metrics but also underscores its robustness in real-world intrusion detection scenarios, emphasizing the practical implications of improving the accuracy and reliability of IDS systems.

(i) Binary Classification

In binary classification on both the NF-UNSW-NB15-v2 and CICIDS2017 datasets, the Transformer-CNN model demonstrated exceptional performance across critical metrics such as accuracy, precision, recall, and F1-score, outperforming previously proposed models. Its ability to extract and leverage essential features from the input data is evident in the classification outcomes. Figure 2 presents the confusion matrices for the Transformer-CNN model applied to the NF-UNSW-NB15-v2 and CICIDS2017 datasets. On the NF-UNSW-NB15-v2 dataset, the model achieved an accuracy of 99.71%, with precision, recall, and F1-score all at 99.71%. The confusion matrix shows that the model correctly identified 4342 normal instances and 1797 attack instances. However, 18 normal instances were misclassified as attacks, with no attack instances misclassified as normal. This performance underscores the model's robustness in handling imbalanced datasets and its precision in detecting attacks while minimizing false alarms. On the CICIDS2017 dataset, the Transformer-CNN model achieved an even higher accuracy of 99.93%, with precision, recall, and F1-score also at 99.93%. The confusion matrix reveals that the model correctly classified 13,939 normal instances and 11,033 attack instances. However, 15 normal instances were misclassified as attacks, and 3 attack instances were misclassified as normal. This result highlights the model's exceptional ability to distinguish between normal and malicious traffic effectively, ensuring reliability and precision in real-world intrusion detection scenarios. These results confirm the Transformer-CNN model's capability to address critical challenges in intrusion detection, including managing imbalanced datasets and reducing false positives and false negatives, making it a highly reliable tool for deployment in real-world network security applications.

The comparative performance of the proposed Transformer-CNN model against other binary classifiers, including a standalone CNN, auto encoder, and DNN, is depicted in Figures 3 and 4. The evaluation metrics displayed include accuracy, precision, recall, and F1-score. The results indicate that the Transformer-CNN model excelled, with an accuracy

of 99.71%, a precision of 99.71%, a recall of 99.71%, and an F1-score of 99.71% on the NF-UNSW-NB15-v2 dataset. This underscores its exceptional capability in detecting intrusions. The high precision score of 99.71% indicates that the Transformer-CNN model effectively identified true positives with very few false positives, while the 99.71% recall score shows that it captured nearly all true positive instances, minimizing false negatives. The F1-score of 99.71% reflects a nearly perfect balance between precision and recall, showcasing the model's overall effectiveness and reliability. On the CICIDS2017 dataset, the Transformer-CNN model demonstrated even greater performance, achieving an accuracy of 99.93%, along with matching precision, recall, and F1-score metrics of 99.93%. In contrast, the standalone auto encoder exhibited lower performance metrics on both datasets, with accuracy, precision, recall, and F1-score around 99.66% on NF-UNSW-NB15-v2 and 99.73% on CICIDS2017. The standalone CNN achieved slightly better metrics of 99.69% on NF-UNSW-NB15-v2 and 99.86% on CICIDS2017. The DNN model had metrics of 99.68% on NF-UNSW-NB15-v2 and 99.88% on CICIDS2017. Ultimately, the Transformer-CNN model stands out due to its robust overall performance on both datasets, reinforcing its suitability for binary classification tasks.

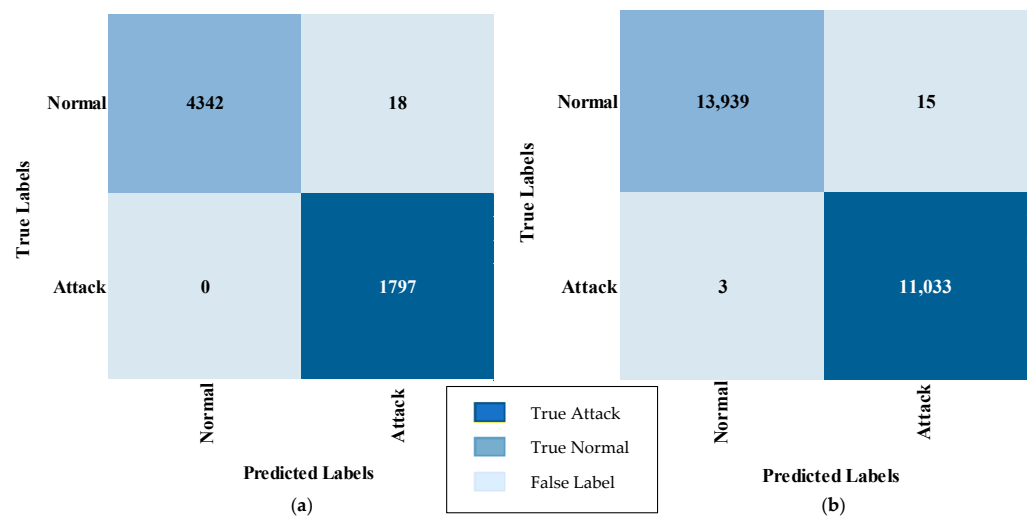


Figure 2. Confusion matrix for binary classification using Transformer-CNN on (a) NF-UNSW-NB15-v2 dataset and (b) CICIDS2017 dataset.

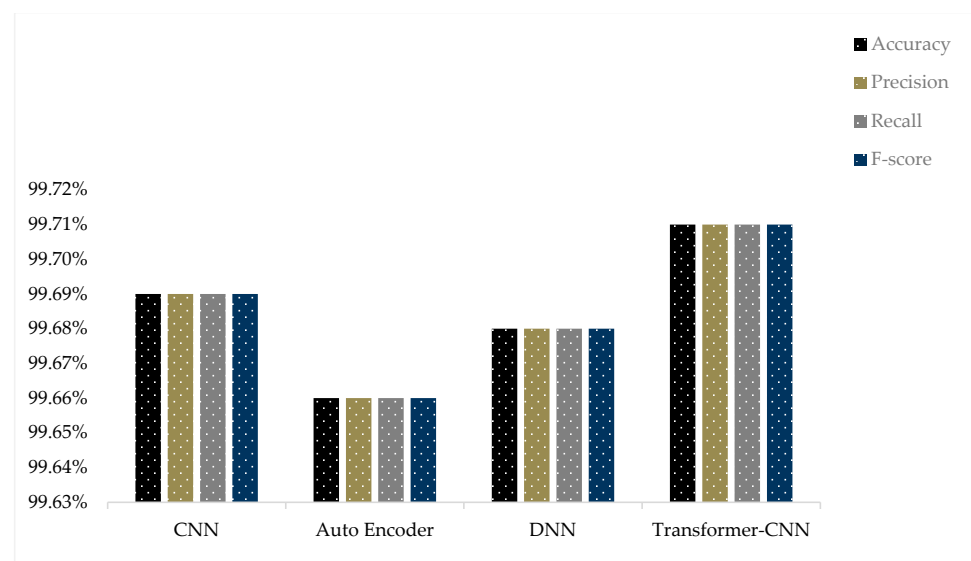


Figure 3. Proposed Transformer-CNN versus binary classifiers on NF-UNSW-NB15-v2 dataset.

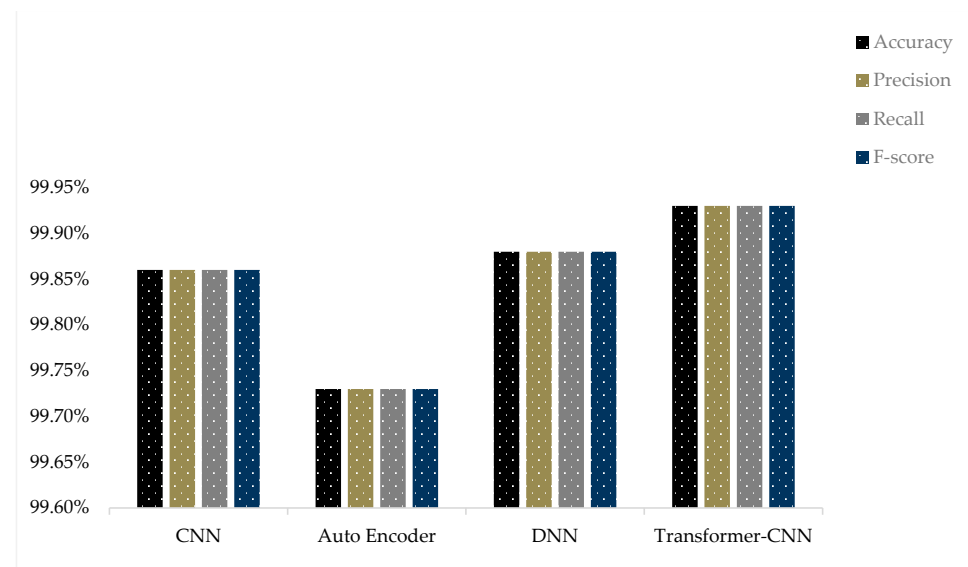


Figure 4. Proposed Transformer-CNN versus binary classifiers on CICIDS2017 dataset.

The effectiveness of the Transformer-CNN model in binary classification is further validated by its exemplary performance metrics across different classes on the NF-UNSW-NB15-v2 and CICIDS2017 datasets. For the NF-UNSW-NB15-v2 dataset, the model achieved an overall accuracy of 99.71%, along with precision, recall, and F1-score all at 99.71%. Specifically, for the ‘Normal’ class, it recorded an accuracy of 99.59%, a perfect precision of 100%, a recall of 99.59%, and an F1-score of 99.79%, showcasing its ability to accurately identify benign traffic. In the ‘Attack’ class, it achieved a perfect accuracy of 100%, precision of 99.01%, recall of 100%, and an F1-score of 99.50%, underscoring its effectiveness in detecting malicious traffic while minimizing false positives and false negatives. On the CICIDS2017 dataset, the model also demonstrated outstanding results, achieving an overall accuracy of 99.93%, precision of 99.93%, recall of 99.93%, and an F1-score of 99.93%. For the ‘Normal’ class, it attained an accuracy of 99.89%, precision of 99.98%, recall of 99.89%, and an F1-score of 99.94%, highlighting its precision in identifying benign traffic. For the ‘Attack’ class, the model achieved an accuracy of 99.97%, precision of 99.86%, recall of 99.97%, and an F1-score of 99.92%, validating its robustness in distinguishing attack traffic with high reliability. The results summarized in Tables 34 and 35 illustrate the Transformer-CNN model’s ability to perform consistently across diverse datasets. The detailed performance metrics for individual classes further emphasize the model’s precision and reliability, making it well-suited for deployment in real-world intrusion detection systems where the consequences of misclassification can be critical.

Table 34. Performance metrics for Transformer-CNN across several classes in binary classification on NF-UNSW-NB15-v2 dataset.

Label	Accuracy	Precision	Recall	F-Score
Normal	99.59%	100%	99.59%	99.79%
Attack	100%	99.01%	100%	99.50%

Table 35. Performance metrics for Transformer-CNN across several classes in binary classification on CICIDS2017 dataset.

Label	Accuracy	Precision	Recall	F-Score
Normal	99.89%	99.98%	99.89%	99.94%
Attack	99.97%	99.86%	99.97%	99.92%

(ii) Multi-Class Classification

In multi-class classification on the NF-UNSW-NB15-v2 dataset, the Transformer-CNN model demonstrated exceptional performance across key metrics such as accuracy, precision, recall, and F1-score compared to other models. The model's ability to accurately distinguish between different types of attacks is clearly reflected in the confusion matrix, as shown in Figure 5. This matrix highlights the model's effectiveness in correctly classifying a wide range of attack classes with minimal misclassification. For instance, the model successfully identified 4294 instances of Benign traffic, 720 instances of Exploits, 474 instances of Fuzzers, 344 instances of Reconnaissance, and 132 instances of Generic attacks. In addition, it correctly recognized 76 instances of DoS, 25 instances of Shellcode, 14 instances of Backdoor, 8 instances of Analysis, and 5 instances of Worms. Few misclassifications were observed, including some false positives and negatives across various attack classes, underscoring the model's overall reliability and precision in distinguishing these attacks. The comprehensive accuracy of 99.02%, precision of 99.30%, recall of 99.02%, and F1-score of 99.13%, as detailed in the confusion matrix, confirm the model's capability in managing the complexities of multi-class classification in real-world scenarios, particularly when dealing with diverse and imbalanced datasets.

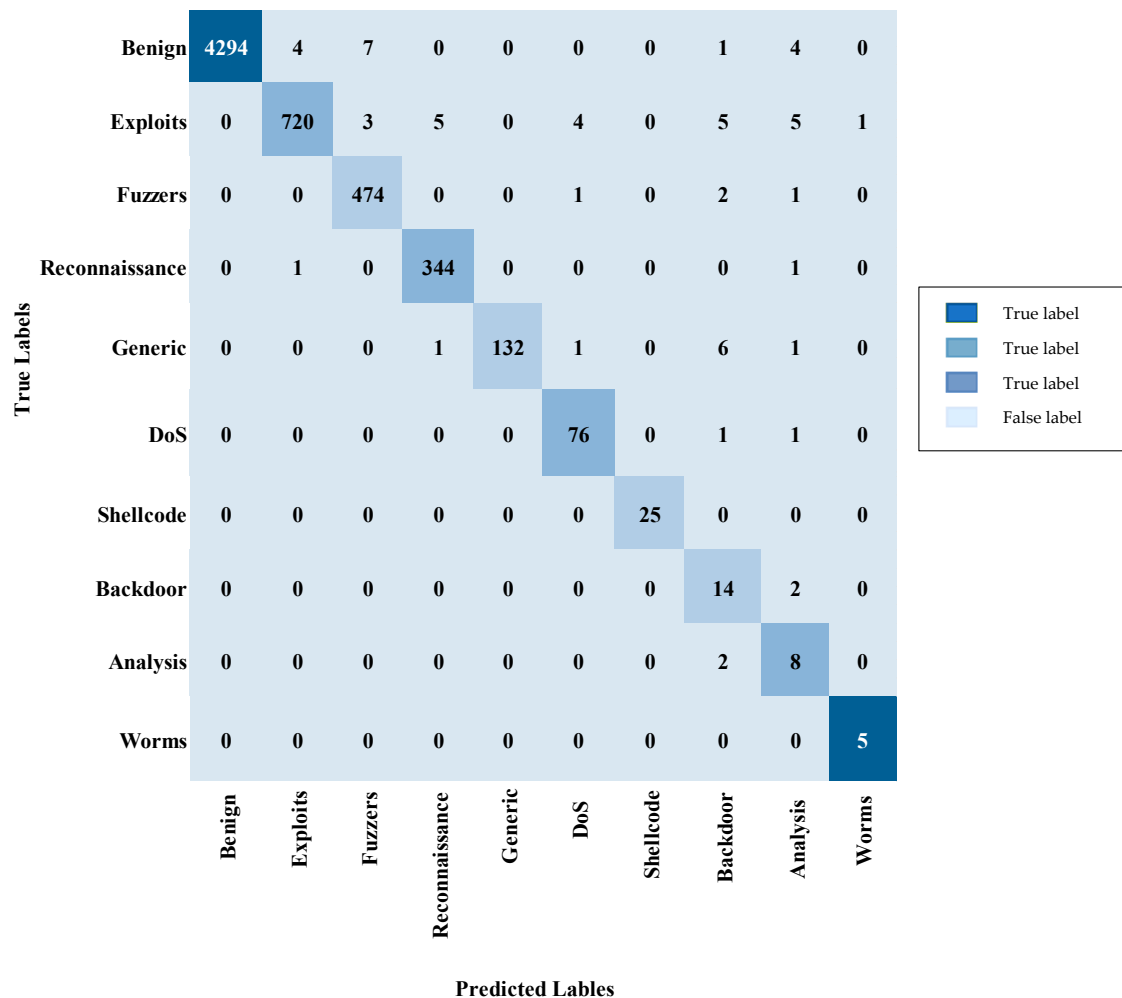


Figure 5. Confusion matrix for multi-class classification using Transformer-CNN on NF-UNSW-NB15-v2 dataset.

In multi-class classification on the CICIDS2017 dataset, the Transformer-CNN model demonstrates remarkable effectiveness, as illustrated by the confusion matrix shown in Figure 6. The model achieves outstanding accuracy, precision, recall, and F1-scores across

various attack classes, effectively distinguishing between diverse at-tack types with minimal misclassifications. For instance, the Benign class achieved 13,773 correct classifications, with only a few instances misclassified into other categories, such as 60 instances as PortScan and 34 as DoS Hulk. The PortScan attack class was classified with high precision, correctly identifying 1,806 out of 1,808 instances, with just 2 instance misclassified. Similarly, the model correctly classified 2,080 instances of DDoS, with 2 instances misclassified into other categories. For DoS Hulk, the model correctly classified 5,609 instances, with only two minor misclassifications. In the DoS GoldenEye class, all 480 instances were correctly identified, showing perfect performance. For FTP-Patator, 255 instances were correctly classified, with just 2 misclassified as DoS Slowloris. The model maintained strong accuracy for the SSH-Patator class, correctly identifying 112 instances with minimal errors. For more challenging attack types, such as DoS Slowloris and DoS Slowhttptest, the model achieved excellent results, correctly classifying 261 and 160 instances, respectively, without any misclassifications. The model also handled the Bot attack class effectively, correctly classifying 104 instances, with only 3 misclassified into the Benign category. The Web Attack - Brute Force class was classified with perfect precision and recall, correctly identifying all 69 instances without any errors, while the Web Attack - XSS class achieved near-perfect performance, correctly identifying 55 instances with minimal errors. The Transformer-CNN model demonstrated strong performance across the Infiltration, Web Attack - SQL Injection, and Heartbleed classes. For Infiltration, it correctly identified 3 instances, but misclassified 2 instances as Heartbleed. In the Web Attack - SQL Injection class, the model classified all 2 instances correctly, achieving perfect accuracy. Similarly, for Heartbleed, the model exhibited flawless performance, correctly identifying all 4 instances with no errors. These results further emphasize the model's ability to handle less frequent and challenging attack classes with high precision. With an overall accuracy of 99.13%, precision of 99.22%, recall of 99.13%, and an F1-score of 99.16%, the Transformer-CNN model demonstrates robust capability in handling multi-class classification challenges. Its ability to classify a wide range of attack types accurately and reliably underscores its potential for real-world deployment in intrusion detection systems, where precision and reliability are paramount.

The comparative performance of the proposed Transformer-CNN model against other multi-class classifiers, including a standalone CNN, auto encoder, and DNN, highlights the Transformer-CNN's remarkable capability in managing complex classification tasks on both the NF-UNSW-NB15-v2 and CICIDS2017 datasets, as shown in Figures 7 and 8. The evaluation metrics, including accuracy, precision, recall, and F1-score, show that the Transformer-CNN consistently outperforms the other classifiers across both datasets. On the NF-UNSW-NB15-v2 dataset, the Transformer-CNN achieved an accuracy of 99.02%, a precision of 99.30%, a recall of 99.02%, and an F1-score of 99.13%, underscoring its effectiveness in handling multi-class classification with high performance. In addition to its high accuracy, the model excelled in precision, recall, and F1-score, which are essential for assessing performance in imbalanced datasets. Specifically, it achieved a precision of 99.30% and a recall of 99.02%, underscoring its effectiveness in identifying true positives while minimizing false positives. In contrast, the CNN achieved an accuracy of 98.36%, with precision, recall, and F1-score values of 98.66%, 98.36%, and 98.46%, respectively. The DNN recorded an accuracy of 97.65%, with precision, recall, and F1-score values of 98.09%, 97.65%, and 97.77%, respectively. The auto encoder exhibited comparatively lower metrics, achieving 95.57% accuracy, 96.54% precision, 95.57% recall, and 95.77% F1-score. On the CICIDS2017 dataset, the Transformer-CNN also led with an accuracy of 99.13%, a precision of 99.22%, a recall of 99.13%, and an F1-score of 99.16%. The CNN achieved an accuracy of 99.05%, with precision, recall, and F1-score values of 99.12%, 99.05%, and 99.07%, respectively. The DNN recorded an accuracy of 99.11%, with precision, recall, and F1-score values of 99.20%, 99.11%, and 99.14%, respectively. The auto encoder achieved an accuracy of 99.09%, with precision, recall, and F1-score values of 99.12%, 99.09%, and 99.09%. These results emphasize the significant improvement offered by the Transformer-CNN model for multi-class classification tasks across both datasets.

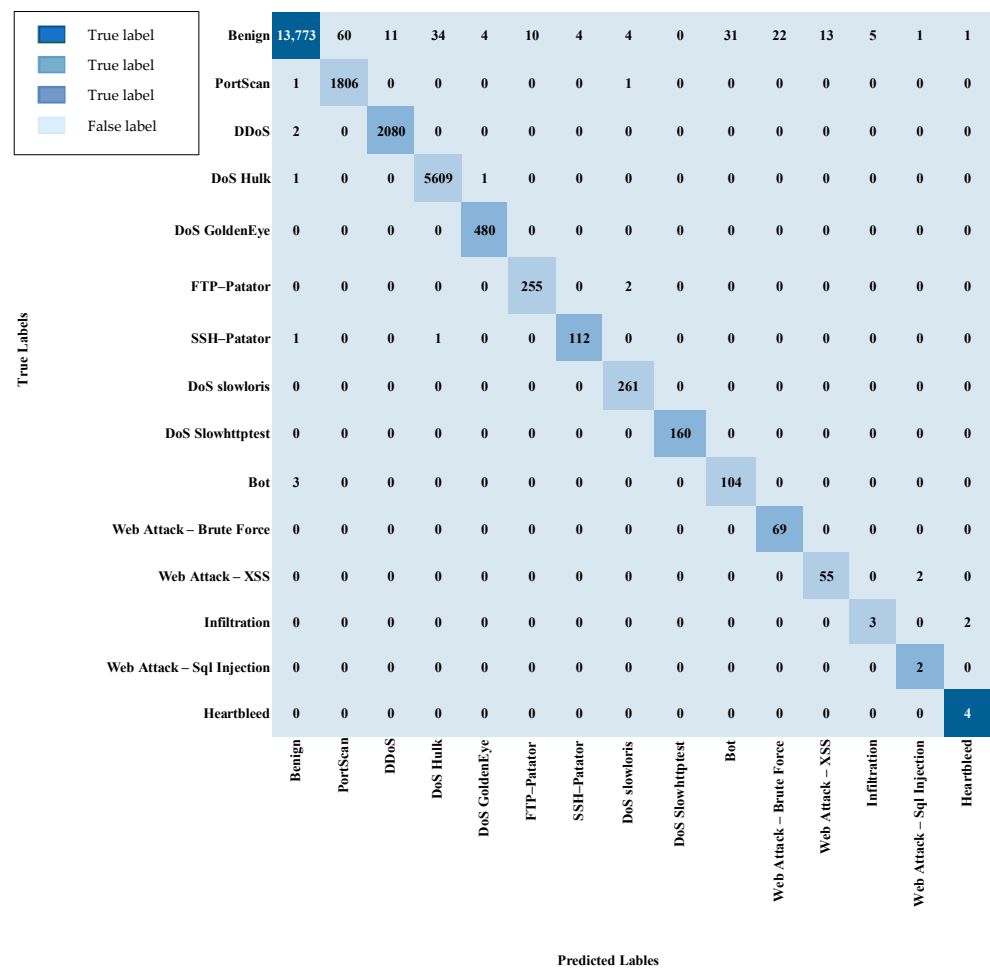


Figure 6. Confusion matrix for multi-class classification using Transformer-CNN on CICIDS2017 dataset.

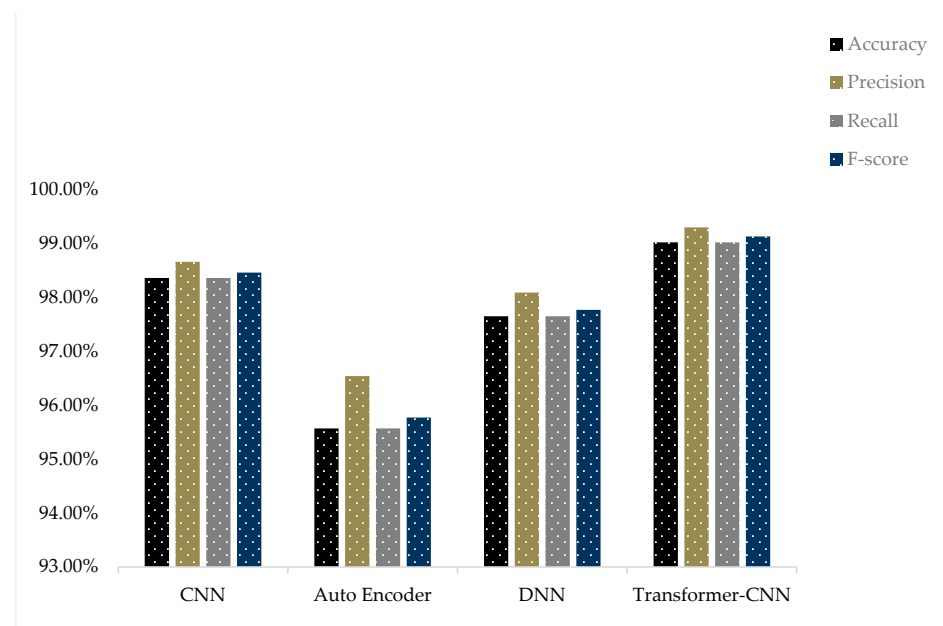


Figure 7. Proposed Transformer-CNN versus multi-class classifiers on NF-UNSW-NB15-v2 dataset.

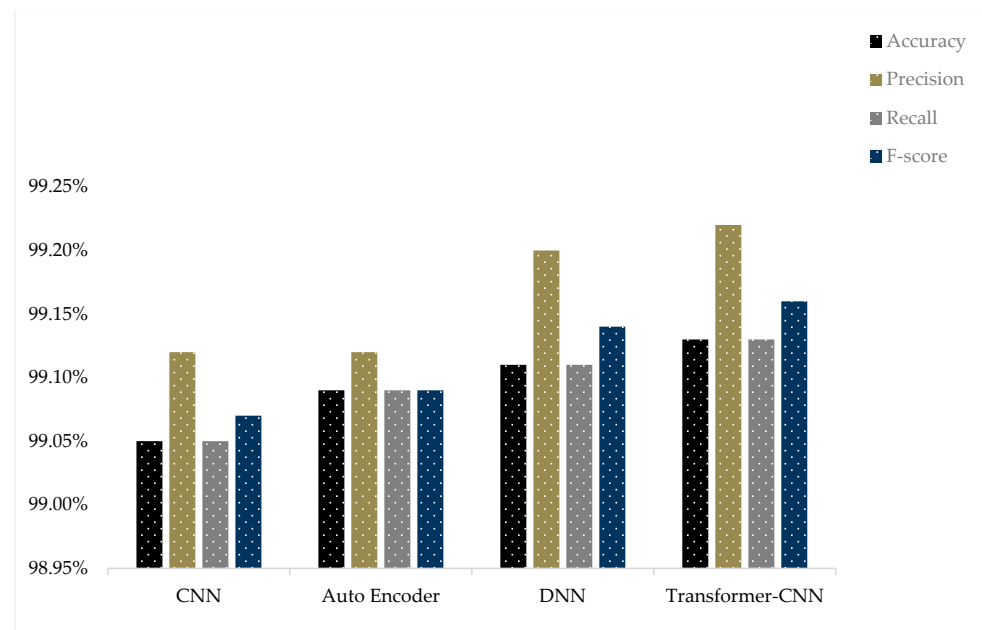


Figure 8. Proposed Transformer-CNN versus multi-class classifiers on CICIDS2017 dataset.

The Transformer-CNN model demonstrated remarkable effectiveness in multi-class classification, as evidenced by its performance metrics across various attack classes. The model achieved exceptional results, recording 100% accuracy, precision, recall, and F1-score for the Shellcode class, reflecting its outstanding capability to accurately identify this specific attack type without errors. For other classes such as Benign, Exploits, and Reconnaissance, the model maintained high performance, with metrics consistently exceeding 96%. For example, the Benign class achieved an accuracy of 99.63%, a precision of 100%, a recall of 99.63%, and an F1-score of 99.81%. The DoS class recorded an accuracy of 97.44% with a precision of 92.68%, while the Fuzzers class achieved an accuracy of 99.16% and an F1-score of 98.54%. Even for more challenging attack types like Backdoor and Analysis, the model performed robustly, attaining F1-scores of 59.57% and 48.48%, respectively. These comprehensive metrics, detailed in Table 36, highlight the model's ability to effectively manage the complexities of multi-class classification. Its precision in distinguishing between various attack types further emphasizes its potential for real-world deployment in intrusion detection systems, where accurate and reliable classification is crucial.

Table 36. Performance metrics for Transformer-CNN across several classes in multi-class classification on NF-UNSW-NB15-v2 dataset.

Label	Accuracy	Precision	Recall	F-Score
Benign	99.63%	100%	99.63%	99.81%
Exploits	96.90%	99.31%	96.90%	98.09%
Fuzzers	99.16%	97.93%	99.16%	98.54%
Reconnaissance	99.42%	98.29%	99.42%	98.85%
Generic	93.62%	100%	93.62%	96.70%
DoS	97.44%	92.68%	97.44%	95%
Shellcode	100%	100%	100%	100%
Backdoor	87.50%	45.16%	87.50%	59.57%
Analysis	80%	34.78%	80%	48.48%
Worms	100%	83.33%	100%	90.91%

The Transformer-CNN model exhibited exceptional effectiveness in multi-class classification, as evidenced by its performance metrics across various attack classes on the CICIDS2017 dataset. The model achieved outstanding results, particularly for certain attack types. For instance, the DoS Slowhttptest class recorded perfect scores, with 100% accuracy, precision, recall, and F1-score, demonstrating the model's capability to accurately classify this attack type without any errors. Similarly, the DoS GoldenEye class achieved 100% accuracy and recall, along with precision and an F1-score exceeding 98%. The model also excels in distinguishing between other classes. For example, the PortScan class recorded an accuracy of 99.89%, precision of 96.78%, recall of 99.89%, and an F1-score of 98.31%. The "DDoS" class similarly performed exceptionally, with accuracy and recall of 99.90% and an F1-score of 99.69%. Despite the inherent complexity of multi-class classification, the Transformer-CNN model maintained high metrics for a majority of the attack types, such as FTP-Patator and SSH-Patator, which achieved F1-scores of 97.70% and 97.39%, respectively. However, for more challenging attack classes like Infiltration and Web Attack-SQL Injection, the model's performance was relatively lower, recording F1-scores of 46.15% and 57.14%, respectively. These results highlight potential areas for improvement in handling low-frequency or highly complex attack types. Overall, the comprehensive metrics detailed in Table 37 underscore the Transformer-CNN model's ability to manage the complexities of multi-class classification effectively. Its precision in distinguishing between various attack types emphasizes its robustness and potential for real-world deployment in intrusion detection systems, where accurate and reliable classification across a wide range of threats is essential.

Table 37. Performance metrics for Transformer-CNN across several classes in multi-class classification on CICIDS2017 dataset.

Label	Accuracy	Precision	Recall	F-Score
Benign	98.57%	99.94%	98.57%	99.25%
PortScan	99.89%	96.78%	99.89%	98.31%
DDoS	99.90%	99.47%	99.90%	99.69%
DoS Hulk	99.96%	99.38%	99.96%	99.67%
DoS GoldenEye	100%	98.97%	100%	99.48%
FTP-Patator	99.22%	96.23%	99.22%	97.70%
SSH-Patator	98.25%	96.55%	98.25%	97.39%
DoS slowloris	100%	97.39%	100%	98.68%
DoS Slowhttptest	100%	100%	100%	100%
Bot	97.20%	77.04%	97.20%	85.95%
Web Attack-Brute Force	100%	75.82%	100%	86.25%
Web Attack-XSS	96.49%	80.88%	96.49%	88%
Infiltration	60%	37.50%	60%	46.15%
Web Attack-Sql Injection	100%	40%	100%	57.14%
Heartbleed	100%	57.14%	100%	72.73%

Case Study for Zero-Day Attack

In today's rapidly evolving cyber threat landscape, zero-day attacks pose a significant challenge to network security. These attacks exploit unknown vulnerabilities, often bypassing traditional security measures. To address this challenge, this case study examines the application of an advanced deep learning model, specifically a Transformer-CNN for effective zero-day attack detection. In the realm of zero-day attack detection, our Transformer-CNN model has proven to be highly effective, especially in the context of the "Reconnaissance" category within the NF-UNSW-NB15-v2 dataset. To rigorously test

the model's ability to detect previously unseen threats, we deliberately omitted this attack class from the training dataset, reserving it solely for evaluation during the testing phase. Remarkably, the model was able to accurately identify 293 out of the 299 instances of this attack, as illustrated in Figure 9. This outcome highlights the model's strong capacity for generalization, allowing it to recognize and respond to novel attack patterns it had not encountered before. The model's success in handling such sophisticated and unknown attack vectors underscores its robustness and positions it as a powerful asset in real-world cyber security defense mechanisms.

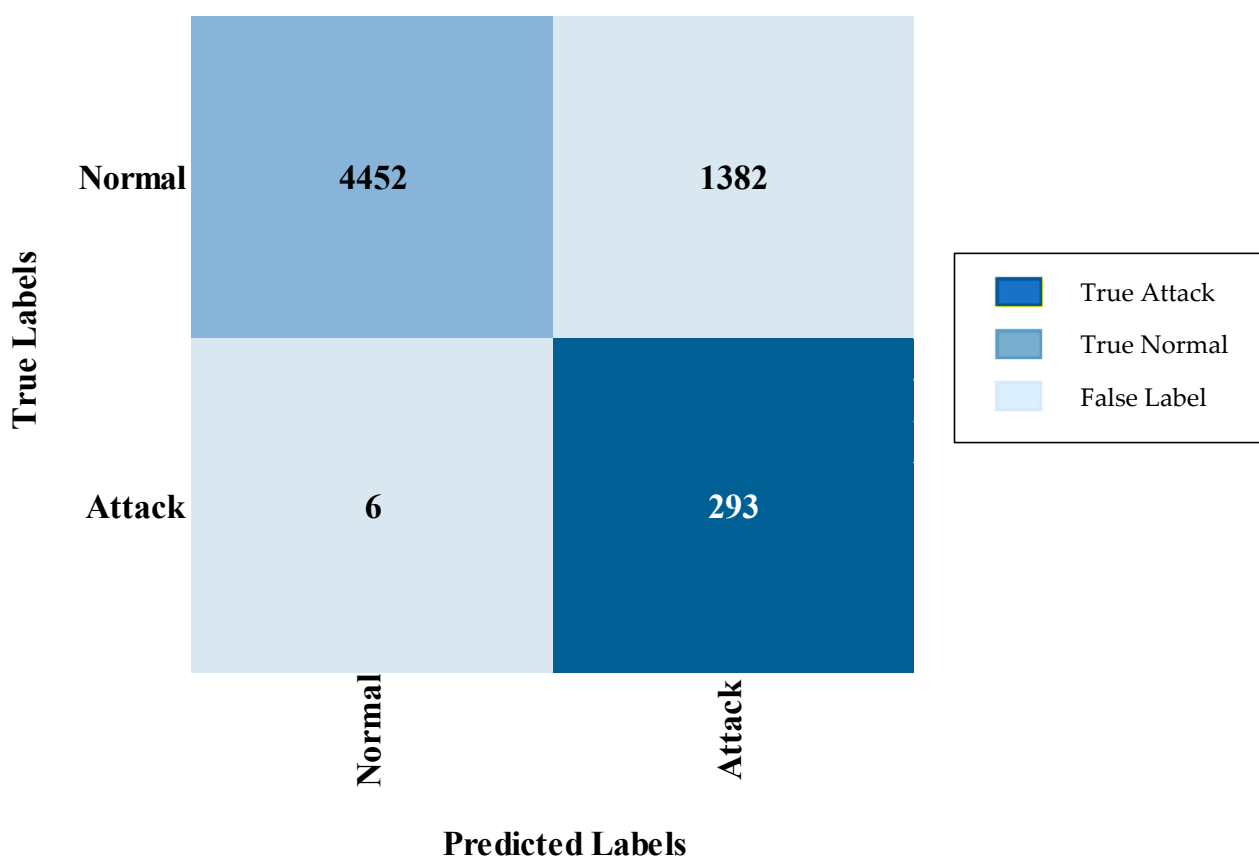


Figure 9. Confusion matrix of the Transformer-CNN model on the NF-UNSW-NB15-v2 dataset, demonstrating its effectiveness in detecting zero-day attacks.

6. Limitations

The Transformer-CNN architecture exemplifies a sophisticated deep learning framework that combines the capabilities of Transformers and CNNs to bolster performance in classification tasks. Although this innovative approach effectively tackles key issues in intrusion detection systems, such as enhancing accuracy and addressing class imbalances, it is essential to acknowledge various limitations and challenges that may arise:

- **Scalability:** As the volume of datasets or the complexity of network traffic grows, the computational demands on the model can intensify, which may hinder its efficiency and its capacity to manage larger datasets or adapt to changing network environments.
- **Generalization:** Although the Transformer-CNN exhibits impressive performance on the NF-UNSW-NB15-v2 and CICIDS2017 datasets, its efficacy across diverse types of network traffic or newly emerging attack vectors is not yet fully established. To assess its robustness and generalization capabilities, it is crucial to evaluate the model against a wider array of datasets, including KDDCup99 [36], NSL KDD [29], and more recent collections like CSE-CIC-IDS2018 [34], and IoT23 [16].

- **Data Preprocessing:** The execution of data preprocessing across various datasets is a vital stage that encompasses activities like addressing missing values, encoding categorical variables, normalizing or standardizing numerical features, and eliminating extraneous information. The model's performance is significantly influenced by the quality and thoroughness of these preprocessing procedures.
- **Model Adaptation:** Adjusting the model for various datasets necessitates a trial-and-error approach to hyperparameter optimization. This iterative process is essential for refining the model to better match the specific characteristics and nuances of new datasets.

7. Conclusions

In this paper, we proposed an advanced hybrid Transformer-CNN deep learning model designed to address the challenges of zero-day attack detection and class imbalance in IDS. The transformer component of our model is employed for contextual feature extraction, enabling the system to analyze relationships and patterns in the data effectively. In contrast, the CNN is responsible for final classification, processing the extracted features to accurately identify specific attack types. By integrating data resampling techniques such as ADASYN, SMOTE and ENN, we effectively address class imbalance in the training data. Additionally, utilizing class weights further enhances our model's performance by balancing the influence of different classes during training. As a result, our model significantly improves detection accuracy while reducing false positives and negatives. The results of our evaluation demonstrate the model's remarkable performance across both the NF-UNSW-NB15-v2 and CICIDS2017 datasets. On the NF-UNSW-NB15-v2 dataset, the model achieved an impressive 99.71% accuracy in binary classification and 99.02% accuracy in multi-class classification. Similarly, on the CICIDS2017 dataset, the model attained 99.93% accuracy in binary classification and 99.13% accuracy in multi-class classification, showcasing its effectiveness across diverse datasets and classification tasks. This performance surpasses that of existing models in both known and unknown threat detection. This research highlights the potential of hybrid deep learning models in fortifying network and cloud environments against increasingly sophisticated cyber threats. Our approach not only enhances real-time detection capabilities but also proves effective in handling imbalanced datasets, a common challenge in IDS development.

8. Future Work

To address the limitations and challenges outlined in Section 6, future research should prioritize exploration in the following domains:

- **Broader Dataset Evaluation:** Future investigations should involve testing the Transformer-CNN across a more diverse range of datasets, including KDDCup99 [36], NSL KDD [29], and newer datasets such as CSE-CIC-IDS2018 [34], and IoT23 [16]. This approach will provide insights into its robustness, generalization potential, and effectiveness in addressing emerging attack vectors.
- **Data Preprocessing Refinement:** The data preprocessing procedures should be meticulously refined and customized for each dataset to achieve optimal model performance. This entails experimenting with various preprocessing techniques and analyzing their effects on model results. Comprehensive discussions of these preprocessing strategies are extensively covered in Section 3.2 and 4.1 of the manuscript.
- **Model Adaptation and Hyperparameter Optimization:** Ongoing investigation into model adaptation techniques is essential, emphasizing the refinement of the hyperparameter optimization process tailored to various datasets. This process should undergo systematic analysis to uncover best practices for effectively adapting the model to different data environments. Detailed discussions of these aspects are presented in Section 3, specifically in Section 3.3.4.

- **Scalability and Computational Efficiency:** It is imperative to enhance the model's computational efficiency and scalability, enabling it to effectively manage larger datasets and more intricate network traffic scenarios without sacrificing performance.

Author Contributions: Conceptualization, H.K. and M.M.; Methodology, H.K. and M.M.; Software, H.K. and M.M.; Validation, H.K. and M.M.; Writing—original draft, H.K. and M.M.; Supervision, M.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The datasets used in our study, NF-UNSW-NB15-v2 and CICIDS2017, are publicly available. Below are the URLs for the datasets: NF-UNSW-NB15-v2: https://staff.itee.uq.edu.au/marius/NIDS_datasets/ (accessed on 15 December 2024); CICIDS2017: <https://www.unb.ca/cic/datasets/ids-2017.html> (accessed on 15 December 2024).

Conflicts of Interest: The authors declare no conflict of interest.

References

- Conti, M.; Dargahi, T.; Dehghantanha, A. *Cyber Threat Intelligence: Challenges and Opportunities*; Springer: Berlin/Heidelberg, Germany, 2018; pp. 1–6. [\[CrossRef\]](#)
- Faker, O.; Dogdu, E. Intrusion detection using big data and deep learning techniques. In Proceedings of the 2019 ACM Southeast Conference. ACM SE'19, Kennesaw, GA, USA, 18–20 April 2019; Association for Computing Machinery: New York, NY, USA, 2019; pp. 86–93. [\[CrossRef\]](#)
- Kaur, G.; Habibi Lashkari, A.; Rahali, A. Intrusion traffic detection and characterization using deep image learning. In Proceedings of the 2020 IEEE International Conference on Dependable, Autonomic and Secure Computing, International Conference on Pervasive Intelligence and Computing, International Conference on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCoM/CyberSciTech), Calgary, AB, Canada, 17–22 August 2020; pp. 55–62. [\[CrossRef\]](#)
- Internet Security Threat Report. Available online: <https://docs.broadcom.com/doc/istr-23-2018-en> (accessed on 18 July 2022).
- Cyberattacks Now Cost Companies \$200,000 on Average, Putting Many out of Business. Available online: <https://www.cnbc.com/2019/10/13/cyberattacks-cost-small-companies-200k-putting-many-out-of-business.html> (accessed on 13 October 2019).
- Kumar, M.; Singh, A.K. Distributed intrusion detection system using blockchain and cloud computing infrastructure. In Proceedings of the 2020 4th International Conference on Trends in Electronics and Informatics (ICOEI)(48184), Tirunelveli, India, 15–17 June 2020; pp. 248–252.
- Zhang, X.; Xie, J.; Huang, L. Real-Time Intrusion Detection Using Deep Learning Techniques. *J. Netw. Comput. Appl.* **2020**, *140*, 45–53.
- Kumar, S.; Kumar, R. A Review of Real-Time Intrusion Detection Systems Using Machine Learning Approaches. *Comput. Secur.* **2020**, *95*, 101944.
- Smith, A.; Jones, B.; Taylor, C. Enhancing Network Security with Real-Time Intrusion Detection Systems. *Int. J. Inf. Secur.* **2021**, *21*, 123–135.
- Sarhan, M.; Layeghy, S.; Portmann, M. Towards a standard feature set for network intrusion detection system datasets. *Mob. Netw. Appl.* **2022**, *27*, 357–370. [\[CrossRef\]](#)
- Sarhan, M.; Layeghy, S.; Moustafa, N.; Portmann, M. Cyber threat intelligence sharing scheme based on federated learning for network intrusion detection. *J. Netw. Syst. Manag.* **2023**, *31*, 3. [\[CrossRef\]](#)
- UNB. Intrusion Detection Evaluation Dataset (CICIDS2017), University of New Brunswick. Available online: <https://www.unb.ca/cic/datasets/ids-2017.html> (accessed on 30 October 2024).
- Panigrahi, R.; Borah, S. A detailed analysis of CICIDS2017 dataset for designing Intrusion Detection Systems. *Int. J. Eng. Technol.* **2018**, *7*, 479–482.
- Anderson, J.P. Computer security threat monitoring and surveillance. In *Technical Report*; James P. Anderson Company: Washington, DC, USA, 1980.
- Mahalingam, A.; Perumal, G.; Subburayalu, G.; Albathan, M.; Altameem, A.; Almakki, R.S.; Hussain, A.; Abbas, Q. ROAST-IoT: A novel range-optimized attention convolutional scattered technique for intrusion detection in IoT networks. *Sensors* **2023**, *23*, 8044. [\[CrossRef\]](#)
- ElKashlan, M.; Elsayed, M.S.; Jurcut, A.D.; Azer, M. A machine learning-based intrusion detection system for iot electric vehicle charging stations (evcss). *Electronics* **2023**, *12*, 1044. [\[CrossRef\]](#)
- Al Nuaimi, T.; Al Zaabi, S.; Alyilieli, M.; AlMaskari, M.; Alblooshi, S.; Alhabsi, F.; Yusof, M.F.B.; Al Badawi, A. A comparative evaluation of intrusion detection systems on the edge-IIoT-2022 dataset. *Intell. Syst. Appl.* **2023**, *20*, 200298. [\[CrossRef\]](#)
- Gad, A.R.; Nashat, A.A.; Barkat, T.M. Intrusion detection system using machine learning for vehicular ad hoc networks based on ToN-IoT dataset. *IEEE Access* **2021**, *9*, 142206–142217. [\[CrossRef\]](#)

19. Al-Daweri, M.S.; Ariffin, K.A.Z.; Abdullah, S.; Senan, M.F.E.M. An analysis of the KDD99 and UNSW-NB15 datasets for the intrusion detection system. *Symmetry* **2020**, *12*, 1666. [\[CrossRef\]](#)
20. Vitorino, J.; Praça, I.; Maia, E. Towards adversarial realism and robust learning for IoT intrusion detection and classification. *Ann. Telecommun.* **2023**, *78*, 401–412. [\[CrossRef\]](#)
21. Othman, T.S.; Abdullah, S.M. An intelligent intrusion detection system for internet of things attack detection and identification using machine learning. *Aro-Sci. J. Koya Univ.* **2023**, *11*, 126–137. [\[CrossRef\]](#)
22. Yaras, S.; Dener, M. IoT-Based Intrusion Detection System Using New Hybrid Deep Learning Algorithm. *Electronics* **2024**, *13*, 1053. [\[CrossRef\]](#)
23. Vinayakumar, R.; Alazab, M.; Soman, K.P.; Poornachandran, P.; Al-Nemrat, A.; Venkatraman, S. Deep learning approach for intelligent intrusion detection system. *IEEE Access* **2019**, *7*, 41525–41550. [\[CrossRef\]](#)
24. Farhana, K.; Rahman, M.; Ahmed, M.T. An intrusion detection system for packet and flow based networks using deep neural network approach. *Int. J. Electr. Comput. Eng.* **2020**, *10*, 5514–5525. [\[CrossRef\]](#)
25. Zhang, C.; Chen, Y.; Meng, Y.; Ruan, F.; Chen, R.; Li, Y.; Yang, Y. A novel framework design of network intrusion detection based on machine learning techniques. *Secur. Commun. Netw.* **2021**, *2021*, 6610675. [\[CrossRef\]](#)
26. Alsharaiah, M.; Abualhaj, M.; Baniata, L.; Al-saaidah, A.; Kharma, Q.; Al-Zyoud, M. An innovative network intrusion detection system (NIDS): Hierarchical deep learning model based on Unsw-Nb15 dataset. *Int. J. Data Netw. Sci.* **2024**, *8*, 709–722. [\[CrossRef\]](#)
27. Jouhari, M.; Benaddi, H.; Ibrahim, K. Efficient Intrusion Detection: Combining χ^2 Feature Selection with CNN-BiLSTM on the UNSW-NB15 Dataset. *arXiv* **2024**, arXiv:2407.14945.
28. Türk, F. Analysis of intrusion detection systems in UNSW-NB15 and NSL-KDD datasets with machine learning algorithms. *Bitlis Eren Üniversitesi Fen Bilim. Derg.* **2023**, *12*, 465–477. [\[CrossRef\]](#)
29. Muhuri, P.; Chatterjee, P.; Yuan, X.; Roy, K.; Esterline, A. Using a long short-term memory recurrent neural network (lstm-rnn) to classify network attacks. *Information* **2020**, *11*, 243. [\[CrossRef\]](#)
30. Fu, Y.; Du, Y.; Cao, Z.; Li, Q.; Xiang, W. A deep learning model for network intrusion detection with imbalanced data. *Electronics* **2022**, *11*, 898. [\[CrossRef\]](#)
31. Yin, Y.; Jang-Jaccard, J.; Xu, W.; Singh, A.; Zhu, J.; Sabrina, F.; Kwak, J. IGRF-RFE: A hybrid feature selection method for MLP-based network intrusion detection on UNSW-NB15 dataset. *J. Big Data* **2023**, *10*, 15. [\[CrossRef\]](#)
32. Yoo, J.; Min, B.; Kim, S.; Shin, D.; Shin, D. Study on network intrusion detection method using discrete pre-processing method and convolution neural network. *IEEE Access* **2021**, *9*, 142348–142361. [\[CrossRef\]](#)
33. Alzughaibi, S.; El Khediri, S. A cloud intrusion detection systems based on dnn using backpropagation and pso on the cse-cic-ids2018 dataset. *Appl. Sci.* **2023**, *13*, 2276. [\[CrossRef\]](#)
34. Basnet, R.B.; Shash, R.; Johnson, C.; Walgren, L.; Doleck, T. Towards Detecting and Classifying Network Intrusion Traffic Using Deep Learning Frameworks. *J. Internet Serv. Inf. Secur.* **2019**, *9*, 1–17.
35. Thilagam, T.; Aruna, R. Intrusion detection for network based cloud computing by custom RC-NN and optimization. *ICT Express* **2021**, *7*, 512–520. [\[CrossRef\]](#)
36. Farahnakian, F.; Heikkonen, J. A deep auto-encoder based approach for intrusion detection system. In Proceedings of the 2018 20th International Conference on Advanced Communication Technology (ICACT), Chuncheon, Republic of Korea, 11–14 February 2018; IEEE: Piscataway, NJ, USA, 2018; pp. 178–183.
37. Mahmood, H.A.; Hashem, S.H. Network intrusion detection system (NIDS) in cloud environment based on hid-den Naïve Bayes multiclass classifier. *Al-Mustansiriyah J. Sci.* **2018**, *28*, 134–142. [\[CrossRef\]](#)
38. Baig, M.M.; Awais, M.M.; El-Alfy, E.S.M. A multiclass cascade of artificial neural network for network intrusion detection. *J. Intell. Fuzzy Syst.* **2017**, *32*, 2875–2883. [\[CrossRef\]](#)
39. Mohy-Eddine, M.; Guezaz, A.; Benkirane, S.; Azrou, M.; Farhaoui, Y. An ensemble learning based intrusion detection model for industrial IoT security. *Big Data Min. Anal.* **2023**, *6*, 273–287. [\[CrossRef\]](#)
40. Nicolas-Alin, S. Machine Learning for Anomaly Detection in Iot Networks: Malware Analysis on the Iot-23 Data Set. Bachelor's Thesis, University of Twente, Enschede, The Netherlands, 2020.
41. Susilo, B.; Sari, R.F. Intrusion detection in IoT networks using deep learning algorithm. *Information* **2020**, *11*, 279. [\[CrossRef\]](#)
42. Szczepański, M.; Pawlicki, M.; Kozik, R.; Choraś, M. The application of deep learning imputation and other advanced methods for handling missing values in network intrusion detection. *Vietnam. J. Comput. Sci.* **2023**, *10*, 1–23. [\[CrossRef\]](#)
43. Kumar, P.; Bagga, H.; Netam, B.S.; Uduthalapally, V. Sad-iot: Security analysis of ddos attacks in iot networks. *Wirel. Pers. Commun.* **2022**, *122*, 87–108. [\[CrossRef\]](#)
44. Sarhan, M.; Layeghy, S.; Portmann, M. Feature analysis for machine learning-based IoT intrusion detection. *arXiv* **2021**, arXiv:2108.12732.
45. Ferrag, M.A.; Friha, O.; Hamouda, D.; Maglaras, L.; Janicke, H. Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning. *IEEE Access* **2022**, *10*, 40281–40306. [\[CrossRef\]](#)
46. Henry, A.; Gautam, S.; Khanna, S.; Rabie, K.; Shongwe, T.; Bhattacharya, P.; Sharma, B.; Chowdhury, S. Composition of hybrid deep learning model and feature optimization for intrusion detection system. *Sensors* **2023**, *23*, 890. [\[CrossRef\]](#) [\[PubMed\]](#)
47. Aleesa, A.; Mohammed, A.A.; Mohammed, A.A.; Sahar, N. Deep-intrusion detection system with enhanced UNSW-NB15 dataset based on deep learning techniques. *J. Eng. Sci. Technol.* **2021**, *16*, 711–727.

48. Ahmad, M.; Riaz, Q.; Zeeshan, M.; Tahir, H.; Haider, S.A.; Khan, M.S. Intrusion detection in internet of things using supervised machine learning based on application and transport layer features using UNSW-NB15 data-set. *EURASIP J. Wirel. Commun. Netw.* **2021**, 2021, 10. [\[CrossRef\]](#)
49. Mohammed, B.; Gbashi, E.K. Intrusion detection system for NSL-KDD dataset based on deep learning and recursive feature elimination. *Eng. Technol. J.* **2021**, 39, 1069–1079. [\[CrossRef\]](#)
50. Umair, M.B.; Iqbal, Z.; Faraz, M.A.; Khan, M.A.; Zhang, Y.D.; Razmjoo, N.; Kadry, S. A network intrusion detection system using hybrid multilayer deep learning model. *Big Data* **2022**, 12, 367–376. [\[CrossRef\]](#)
51. Choobdar, P.; Naderan, M.; Naderan, M. Detection and multi-class classification of intrusion in software defined networks using stacked auto-encoders and CICIDS2017 dataset. *Wirel. Pers. Commun.* **2022**, 123, 437–471. [\[CrossRef\]](#)
52. Shende, S.; Thorat, S. Long short-term memory (LSTM) deep learning method for intrusion detection in network security. *Int. J. Eng. Res.* **2020**, 9, 1615–1620.
53. Farhan, B.I.; Jasim, A.D. Performance analysis of intrusion detection for deep learning model based on CSE-CIC-IDS2018 dataset. *Indones. J. Electr. Eng. Comput. Sci.* **2022**, 26, 1165–1172. [\[CrossRef\]](#)
54. Farhan, R.I.; Maolood, A.T.; Hassan, N. Performance analysis of flow-based attacks detection on CSE-CIC-IDS2018 dataset using deep learning. *Indones. J. Electr. Eng. Comput. Sci.* **2020**, 20, 1413–1418. [\[CrossRef\]](#)
55. Lin, P.; Ye, K.; Xu, C.Z. Dynamic network anomaly detection system by using deep learning techniques. In Proceedings of the Cloud Computing–CLOUD 2019: 12th International Conference, Held as Part of the Services Conference Federation, SCF 2019, San Diego, CA, USA, 25–30 June 2019; Proceedings 12. Springer International Publishing: Berlin/Heidelberg, Germany, 2019; pp. 161–176.
56. Liu, G.; Zhang, J. CNID: Research of network intrusion detection based on convolutional neural network. *Discret. Dyn. Nat. Soc.* **2020**, 2020, 4705982. [\[CrossRef\]](#)
57. Li, F.; Shen, H.; Mai, J.; Wang, T.; Dai, Y.; Miao, X. Pre-trained language model-enhanced conditional generative adversarial networks for intrusion detection. *Peer-to-Peer Netw. Appl.* **2024**, 17, 227–245. [\[CrossRef\]](#)
58. Wang, S.; Yao, X. Multiclass imbalance problems: Analysis and potential solutions. *IEEE Trans. Syst. Man Cybern. Part B* **2012**, 42, 1119–1130. [\[CrossRef\]](#) [\[PubMed\]](#)
59. Abdelkhalek, A.; Mashaly, M. Addressing the class imbalance problem in network intrusion detection systems using data resampling and deep learning. *J. Supercomput.* **2023**, 79, 10611–10644. [\[CrossRef\]](#)
60. Yang, H.; Xu, J.; Xiao, Y.; Hu, L. SPE-ACGAN: A resampling approach for class imbalance problem in network intrusion detection systems. *Electronics* **2023**, 12, 3323. [\[CrossRef\]](#)
61. Zakariah, M.; AlQahtani, S.A.; Al-Rakhami, M.S. Machine learning-based adaptive synthetic sampling technique for intrusion detection. *Appl. Sci.* **2023**, 13, 6504. [\[CrossRef\]](#)
62. Thiya, B.; Dey, S. Efficient feature evaluation approach for a class-imbalanced dataset using machine learning. *Procedia Comput. Sci.* **2023**, 218, 2520–2532. [\[CrossRef\]](#)
63. AlbAlbasheer, F.O.; Haibatti, R.R.; Agarwal, M.; Nam, S.Y. A Novel IDS Based on Jaya Optimizer and Smote-ENN for Cyberattacks Detection. *IEEE Access* **2024**, 12, 101506–101527. [\[CrossRef\]](#)
64. Arik, A.O.; Cavdaroglu, G.C. An Intrusion Detection Approach based on the Combination of Oversampling and Undersampling Algorithms. *Acta Infologica* **2023**, 7, 125–138. [\[CrossRef\]](#)
65. Rao, Y.N.; Suresh Babu, K. An imbalanced generative adversarial network-based approach for network intrusion detection in an imbalanced dataset. *Sensors* **2023**, 23, 550. [\[CrossRef\]](#)
66. Jamoos, M.; Mora, A.M.; AlKhanafseh, M.; Surakhi, O. A new data-balancing approach based on generative adversarial network for network intrusion detection system. *Electronics* **2023**, 12, 2851. [\[CrossRef\]](#)
67. Xu, B.; Sun, L.; Mao, X.; Ding, R.; Liu, C. IoT Intrusion Detection System Based on Machine Learning. *Electronics* **2023**, 12, 4289. [\[CrossRef\]](#)
68. Assy, A.T.; Mostafa, Y.; Abd El-khaleq, A.; Mashaly, M. Anomaly-based intrusion detection system using one-dimensional convolutional neural network. *Procedia Comput. Sci.* **2023**, 220, 78–85. [\[CrossRef\]](#)
69. Elghalhoud, O.; Naik, K.; Zaman, M.; Manzano, R. *Data Balancing and cnn Based Network Intrusion Detection System*; IEEE: Piscataway, NJ, USA, 2023.
70. Almarshdi, R.; Nassef, L.; Fadel, E.; Alowidi, N. Hybrid Deep Learning Based Attack Detection for Imbalanced Data Classification. *Intell. Autom. Soft Comput.* **2023**, 35, 297–320. [\[CrossRef\]](#)
71. Thockchom, N.; Singh, M.M.; Nandi, U. A novel ensemble learning-based model for network intrusion detection. *Complex Intell. Syst.* **2023**, 9, 5693–5714. [\[CrossRef\]](#)
72. Jumabek, A.; Yang, S.S.; Noh, Y.T. CatBoost-based network intrusion detection on imbalanced CIC-IDS-2018 dataset. *Korean Soc. Commun. Commun. J.* **2021**, 46, 2191–2197. [\[CrossRef\]](#)
73. Zhu, Y.; Liang, J.; Chen, J.; Ming, Z. An improved nsga-iii algorithm for feature selection used in intrusion detection. *Knowl.-Based Syst.* **2017**, 116, 74–85. [\[CrossRef\]](#)
74. Jiang, J.; Wang, Q.; Shi, Z.; Lv, B.; Qi, B. Rst-rf: A hybrid model based on rough set theory and random forest for network intrusion detection. In Proceedings of the 2nd International Conference on Cryptography, Security and Privacy, Guiyang, China, 16–18 March 2018.

75. Chawla, N.; Bowyer, K.; Hall, L.; Kegelmeyer, W. Smote: Synthetic minority over-sampling technique. *J. Artif. Intell. Res.* **2002**, *16*, 321–357. [\[CrossRef\]](#)
76. Alikhanov, J.; Jang, R.; Abuhamad, M.; Mohaisen, D.; Nyang, D.; Noh, Y. Investigating the effect of traffic sampling on machine learning-based network intrusion detection approaches. *IEEE Access* **2022**, *10*, 5801–5823. [\[CrossRef\]](#)
77. Zhang, X.; Ran, J.; Mi, J. An intrusion detection system based on convolutional neural network for imbalanced network traffic. In Proceedings of the 2019 IEEE 7th International Conference on Computer Science and Network Technology (ICCSNT), Dalian, China, 19–20 October 2019; pp. 456–460.
78. Gupta, N.; Jindal, V.; Bedi, P. CSE-IDS: Using cost-sensitive deep learning and ensemble algorithms to handle class imbalance in Network-based intrusion detection systems. *Comput. Secur.* **2021**, *112*, 102499. [\[CrossRef\]](#)
79. Mbow, M.; Koide, H.; Sakurai, K. Handling class imbalance problem in intrusion detection system based on deep learning. *Int. J. Netw. Comput.* **2022**, *12*, 467–492. [\[CrossRef\]](#) [\[PubMed\]](#)
80. Patro, S.G.; Sahu, D.-K.K. Normalization: A preprocessing stage. *arXiv* **2015**, arXiv:1503.06462. [\[CrossRef\]](#)
81. Bagui, S.; Li, K. Resampling imbalanced data for network intrusion detection datasets. *J. Big Data* **2021**, *8*, 6. [\[CrossRef\]](#)
82. Elmasry, W.; Akbulut, A.; Zaim, A.H. Empirical study on multiclass classification-based network intrusion detection. *Comput. Intell.* **2019**, *35*, 919–954. [\[CrossRef\]](#)
83. El-Habil, B.Y.; Abu-Naser, S.S. Global climate prediction using deep learning. *J. Theor. Appl. Inf. Technol.* **2022**, *100*, 4824–4838.
84. He, H.; Wu, D. ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning. In Proceedings of the 2008 Fourth International Conference on Natural Computation, Jinan, China, 18–20 October 2008.
85. Wilson, D.L. Asymptotic properties of nearest neighbor rules using edited data. *IEEE Trans. Syst. Man Cybern.* **1972**, *3*, 408–421. [\[CrossRef\]](#)
86. He, H.; Garcia, E. Learning from imbalanced data. In *IEEE Transactions on Knowledge and Data Engineering*; IEEE: Piscataway, NJ, USA, 2009.
87. Zhendong, S.; Jinping, M. Deep learning-driven MIMO: Data encoding and processing mechanism. *Phys. Commun.* **2022**, *57*, 101976. [\[CrossRef\]](#)
88. Xin, Z.; Chunjiang, Z.; Jun, S.; Kunshan, Y.; Min, X. Detection of lead content in oilseed rape leaves and roots based on deep transfer learning and hyperspectral imaging technology. *Spectrochim. Acta Part A Mol. Biomol. Spectrosc.* **2022**, *290*, 122288. [\[CrossRef\]](#)
89. Goodfellow, I.; Bengio, Y.; Courville, A. *Deep Learning*; MIT Press: Cambridge, MA, USA, 2016.
90. Nair, V.; Hinton, G.E. Rectified linear units improve restricted boltzmann machines. In Proceedings of the 27th International Conference on Machine Learning (ICML-10), Haifa, Israel, 21–24 June 2010; pp. 807–814.
91. Srivastava, N.; Hinton, G.; Krizhevsky, A.; Sutskever, I.; Salakhutdinov, R. Dropout: A simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.* **2014**, *15*, 1929–1958.
92. Bishop, C.M.; Nasrabadi, N.M. *Pattern Recognition And Machine Learning*; Springer: New York, NY, USA, 2006; Volume 4.
93. Nielsen, M.A. Neural Networks and Deep Learning. In *Chapter 1 Explains the Basics of Feedforward Operations in Neural Networks*; Determination Press: San Francisco, CA, USA, 2015.
94. Glorot, X.; Bordes, A.; Bengio, Y. Deep Sparse Rectifier Neural Networks. In Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, Fort Lauderdale, FL, USA, 11–13 April 2011.
95. Vaswani, A.; Noam, S.; Niki, P.; Jakob, U.; Llion, J.; Aidan, N.G.; Lukasz, K.; Illia, P. Attention Is All You Need.(Nips), 2017. *arXiv* **2017**, arXiv:1706.03762.
96. Lei Ba, J.; Kiros, J.R.; Hinton, G.E. Layer normalization. *arXiv* **2016**, arXiv:1607.06450.
97. Ring, M.; Wunderlich, S.; Scheuring, D.; Landes, D.; Hotho, A. A survey of network-based intrusion detection data sets. *Comput. Secur.* **2019**, *86*, 147–167. [\[CrossRef\]](#)
98. Sharafaldin, I.; Lashkari, A.H.; Ghorbani, A.A. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp* **2018**, *1*, 108–116.
99. Sharafaldin, I.; Habibi Lashkari, A.; Ghorbani, A.A. A detailed analysis of the cids2017 data set. In Proceedings of the Information Systems Security and Privacy: 4th International Conference, ICISSP 2018, Funchal-Madeira, Portugal, 22–24 January 2018; Revised Selected Papers 4. Springer International Publishing: Berlin/Heidelberg, Germany, 2019; pp. 172–188.
100. Jyothsna, V.; Prasad, K.M. Anomaly-based intrusion detection system. In *Computer and Network Security*; Intech: Houston, TX, USA, 2019; Volume 10.
101. Chen, C.; Song, Y.; Yue, S.; Xu, X.; Zhou, L.; Lv, Q.; Yang, L. FCNN-SE: An Intrusion Detection Model Based on a Fusion CNN and Stacked Ensemble. *Appl. Sci.* **2022**, *12*, 8601. [\[CrossRef\]](#)
102. Powers, D.M.W. Evaluation: From Precision, Recall, and F-Measure to ROC, Informedness, Markedness & Correlation. *J. Mach. Learn. Technol.* **2011**, *2*, 37–63.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.